

22. Delegates

[*Note:* Delegates enable scenarios that other languages—such as C++, Pascal, and Modula—have addressed with function pointers. Unlike C++ function pointers, however, delegates are fully object oriented, and unlike C++ pointers to member functions, delegates encapsulate both an object instance and a method. *end note*]

A delegate declaration defines a class that is derived from the class `System.Delegate`. A delegate instance encapsulates one or more methods, each of which is referred to as a **callable entity**. For instance methods, a callable entity consists of an instance and a method on that instance. For static methods, a callable entity consists of just a method. Given a delegate instance and an appropriate set of arguments, one can invoke all of that delegate instance’s methods with that set of arguments.

An interesting and useful property of a delegate instance is that it does not know or care about the classes of the methods it encapsulates; all that matters is that those methods be compatible (§22.1) with the delegate’s type. This makes delegates perfectly suited for “anonymous” invocation.

22.1 Delegate declarations

A *delegate-declaration* is a *type-declaration* (§16.5) that declares a new delegate type.

delegate-declaration:

```
attributesopt delegate-modifiersopt delegate return-type identifier
    ( formal-parameter-listopt ) ;
```

delegate-modifiers:

```
delegate-modifier
delegate-modifiers delegate-modifier
```

delegate-modifier:

```
new
public
protected
internal
private
```

It is a compile-time error for the same modifier to appear multiple times in a delegate declaration.

The **new** modifier is only permitted on delegates declared within another type, in which case it specifies that such a delegate hides an inherited member by the same name, as described in §17.2.2.

The **public**, **protected**, **internal**, and **private** modifiers control the accessibility of the delegate type. Depending on the context in which the delegate declaration occurs, some of these modifiers may not be permitted (§10.5.1).

The delegate’s type name is *identifier*.

The optional *formal-parameter-list* specifies the parameters of the delegate, and *return-type* indicates the return type of the delegate. A method and a delegate type are **compatible** if both of the following are true:

- They have the same number or parameters, with the same types, in the same order, with the same parameter modifiers.
- Their *return-types* are the same.

Delegate types in C# are name equivalent, not structurally equivalent. [*Note:* However, instances of two distinct but structurally equivalent delegate types may compare as equal (§14.9.8). *end note*] Specifically,

two different delegate types that have the same parameter lists and return type are considered **different** delegate types. [Example: For example:

```

delegate int D1(int i, double d);

class A
{
    public static int M1(int a, double b) {...}
}

class B
{
    delegate int D2(int c, double d);
    public static int M1(int f, double g) {...}
    public static void M2(int k, double l) {...}
    public static int M3(int g) {...}
    public static void M4(int g) {...}
}

```

The delegate types D1 and D2 are both compatible with the methods A.M1 and B.M1, since they have the same return type and parameter list; however, these delegate types are two different types, so they are not interchangeable. The delegate types D1 and D2 are incompatible with the methods B.M2, B.M3, and B.M4, since they have different return types or parameter lists. *end example*]

The only way to declare a delegate type is via a *delegate-declaration*. A delegate type is a class type that is derived from System.Delegate. Delegate types are implicitly **sealed**, so it is not permissible to derive any type from a delegate type. It is also not permissible to derive a non-delegate class type from System.Delegate. Note that System.Delegate is not itself a delegate type; it is a class type from which all delegate types are derived.

C# provides special syntax for delegate instantiation and invocation. Except for instantiation, any operation that can be applied to a class or class instance can also be applied to a delegate class or instance, respectively. In particular, it is possible to access members of the System.Delegate type via the usual member access syntax.

The set of methods encapsulated by a delegate instance is called an **invocation list**. When a delegate instance is created (§22.2) from a single method, it encapsulates that method, and its invocation list contains only one entry. However, when two non-null delegate instances are combined, their invocation lists are concatenated—in the order left operand then right operand—to form a new invocation list, which contains two or more entries.

Delegates are combined using the binary + (§14.7.4) and += operators (§14.13.2). A delegate can be removed from a combination of delegates, using the binary - (§14.7.5) and -= operators (§14.13.2). Delegates can be compared for equality (§14.9.8).

[Example: The following example shows the instantiation of a number of delegates, and their corresponding invocation lists:

```

delegate void D(int x);

class Test
{
    public static void M1(int i) {...}
    public static void M2(int i) {...}
}

class Demo
{
    static void Main() {
        D cd1 = new D(Test.M1); // M1
        D cd2 = new D(Test.M2); // M2
        D cd3 = cd1 + cd2;      // M1 + M2
        D cd4 = cd3 + cd1;      // M1 + M2 + M1
        D cd5 = cd4 + cd3;      // M1 + M2 + M1 + M1 + M2
    }
}

```

When `cd1` and `cd2` are instantiated, they each encapsulate one method. When `cd3` is instantiated, it has an invocation list of two methods, `M1` and `M2`, in that order. `cd4`'s invocation list contains `M1`, `M2`, and `M1`, in that order. Finally, `cd5`'s invocation list contains `M1`, `M2`, `M1`, `M1`, and `M2`, in that order.

For more examples of combining (as well as removing) delegates, see §22.3. *end example*

22.2 Delegate instantiation

An instance of a delegate is created by a *delegate-creation-expression* (§14.5.10.3). The newly created delegate instance then refers to either:

- The static method referenced in the *delegate-creation-expression*, or
- The target object (which cannot be `null`) and instance method referenced in the *delegate-creation-expression*, or
- Another delegate

[*Example:* For example:

```

delegate void D(int x);
class Test
{
    public static void M1(int i) {...}
    public void M2(int i) {...}
}
class Demo
{
    static void Main() {
        D cd1 = new D(Test.M1); // static method
        Test t = new Test();
        D cd2 = new D(t.M2);     // instance method
        D cd3 = new D(cd2);      // another delegate
    }
}

```

end example

Once instantiated, delegate instances always refer to the same target object and method. [*Note:* Remember, when two delegates are combined, or one is removed from another, a new delegate results with its own invocation list; the invocation lists of the delegates combined or removed remain unchanged. *end note*]

22.3 Delegate invocation

C# provides special syntax for invoking a delegate. When a non-`null` delegate instance whose invocation list contains one entry, is invoked, it invokes the one method with the same arguments it was given, and returns the same value as the referred to method. (See §14.5.5.2 for detailed information on delegate invocation.) If an exception occurs during the invocation of such a delegate, and that exception is not caught within the method that was invoked, the search for an exception catch clause continues in the method that called the delegate, as if that method had directly called the method to which that delegate referred.

Invocation of a delegate instance whose invocation list contains multiple entries, proceeds by invoking each of the methods in the invocation list, synchronously, in order. Each method so called is passed the same set of arguments as was given to the delegate instance. If such a delegate invocation includes reference parameters (§17.5.1.2), each method invocation will occur with a reference to the same variable; changes to that variable by one method in the invocation list will be visible to methods further down the invocation list. If the delegate invocation includes output parameters or a return value, their final value will come from the invocation of the last delegate in the list. If an exception occurs during processing of the invocation of such a delegate, and that exception is not caught within the method that was invoked, the search for an exception catch clause continues in the method that called the delegate, and any methods further down the invocation list are **not** invoked.

Attempting to invoke a delegate instance whose value is `null` results in an exception of type `System.NullReferenceException`.

[Example: The following example shows how to instantiate, combine, remove, and invoke delegates:

```
using System;
delegate void D(int x);
class Test
{
    public static void M1(int i) {
        Console.WriteLine("Test.M1: " + i);
    }
    public static void M2(int i) {
        Console.WriteLine("Test.M2: " + i);
    }
    public void M3(int i) {
        Console.WriteLine("Test.M3: " + i);
    }
}
class Demo
{
    static void Main() {
        D cd1 = new D(Test.M1);
        cd1(-1);    // call M1
        D cd2 = new D(Test.M2);
        cd2(-2);    // call M2
        D cd3 = cd1 + cd2;
        cd3(10);    // call M1 then M2
        cd3 += cd1;
        cd3(20);    // call M1, M2, then M1
        Test t = new Test();
        D cd4 = new D(t.M3);
        cd3 += cd4;
        cd3(30);    // call M1, M2, M1, then M3
        cd3 -= cd1; // remove last M1
        cd3(40);    // call M1, M2, then M3
        cd3 -= cd4;
        cd3(50);    // call M1 then M2
        cd3 -= cd2;
        cd3(60);    // call M1
        cd3 -= cd2; // impossible removal is benign
        cd3(60);    // call M1
        cd3 -= cd1; // invocation list is empty
        // cd3(70); // System.NullReferenceException thrown
        cd3 -= cd1; // impossible removal is benign
    }
}
```

As shown in the statement `cd3 += cd1;`, a delegate can be present in an invocation list multiple times. In this case, it is simply invoked once per occurrence. In an invocation list such as this, when that delegate is removed, the last occurrence in the invocation list is the one actually removed.

Immediately prior to the execution of the final statement, `cd3 -= cd1;`, the delegate `cd3` refers to an empty invocation list. Attempting to remove a delegate from an empty list (or to remove a non-existent delegate from a non-empty list) is not an error.

The output produced is:

```
Test.M1: -1
Test.M2: -2
```

```
1      Test.M1: 10
2      Test.M2: 10

3      Test.M1: 20
4      Test.M2: 20
5      Test.M1: 20

6      Test.M1: 30
7      Test.M2: 30
8      Test.M1: 30
9      Test.M3: 30

10     Test.M1: 40
11     Test.M2: 40
12     Test.M3: 40

13     Test.M1: 50
14     Test.M2: 50

15     Test.M1: 60
16     Test.M1: 60

17  end example]
```


23. Exceptions

Exceptions in C# provide a structured, uniform, and type-safe way of handling both system level and application-level error conditions. [Note: The exception mechanism in C# is quite similar to that of C++, with a few important differences:

- In C#, all exceptions must be represented by an instance of a class type derived from `System.Exception`. In C++, any value of any type can be used to represent an exception.
- In C#, a finally block (§15.10) can be used to write termination code that executes in both normal execution and exceptional conditions. Such code is difficult to write in C++ without duplicating code.
- In C#, system-level exceptions such as overflow, divide-by-zero, and null dereferences have well defined exception classes and are on a par with application-level error conditions.

end note]

23.1 Causes of exceptions

Exception can be thrown in two different ways.

- A `throw` statement (§15.9.5) throws an exception immediately and unconditionally. Control never reaches the statement immediately following the `throw`.
- Certain exceptional conditions that arise during the processing of C# statements and expressions cause an exception in certain circumstances when the operation cannot be completed normally. For example, an integer division operation (§14.7.2) throws a `System.DivideByZeroException` if the denominator is zero. See §23.4 for a list of the various exceptions that can occur in this way.

23.2 The `System.Exception` class

The `System.Exception` class is the base type of all exceptions. This class has a few notable properties that all exceptions share:

- `Message` is a read-only property of type `string` that contains a human-readable description of the reason for the exception.
- `InnerException` is a read-only property of type `Exception`. If its value is non-`null`, it refers to the exception that caused the current exception. (That is, the current exception was raised in a catch block handling the type `InnerException`.) Otherwise, its value is `null`, indicating that this exception was not caused by another exception. (The number of exception objects chained together in this manner can be arbitrary.)

The value of these properties can be specified in calls to the instance constructor for `System.Exception`.

23.3 How exceptions are handled

Exceptions are handled by a `try` statement (§15.10).

When an exception occurs, the system searches for the nearest `catch` clause that can handle the exception, as determined by the run-time type of the exception. First, the current method is searched for a lexically enclosing `try` statement, and the associated `catch` clauses of the `try` statement are considered in order. If that fails, the method that called the current method is searched for a lexically enclosing `try` statement that encloses the point of the call to the current method. This search continues until a `catch` clause is found that can handle the current exception, by naming an exception class that is of the same class, or a base class, of

the run-time type of the exception being thrown. A `catch` clause that doesn't name an exception class can handle any exception.

Once a matching `catch` clause is found, the system prepares to transfer control to the first statement of the `catch` clause. Before execution of the `catch` clause begins, the system first executes, in order any `finally` clauses that were associated with `try` statements more nested than the one that caught the exception.

If no matching `catch` clause is found, one of two things occurs:

- If the search for a matching `catch` clause reaches a static constructor (§17.11) or static field initializer, then a `System.TypeInitializationException` is thrown at the point that triggered the invocation of the static constructor. The inner exception of the `System.TypeInitializationException` contains the exception that was originally thrown.
- If the search for matching `catch` clauses reaches the code that initially started the thread, then execution of the thread is terminated. The impact of such termination is implementation-defined.

Exceptions that occur during destructor execution are worth special mention. If an exception occurs during destructor execution, and that exception is not caught, then the execution of that destructor is terminated and the destructor of the base class (if any) is called. If there is no base class (as in the case of the `object` type) or if there is no base class destructor, then the exception is discarded.

23.4 Common Exception Classes

The following exceptions are thrown by certain C# operations.

<code>System.ArithmeticException</code>	A base class for exceptions that occur during arithmetic operations, such as <code>System.DivideByZeroException</code> and <code>System.OverflowException</code> .
<code>System.ArrayTypeMismatchException</code>	Thrown when a store into an array fails because the actual type of the stored element is incompatible with the actual type of the array.
<code>System.DivideByZeroException</code>	Thrown when an attempt to divide an integral value by zero occurs.
<code>System.IndexOutOfRangeException</code>	Thrown when an attempt to index an array via an index that is less than zero or outside the bounds of the array.
<code>System.InvalidCastException</code>	Thrown when an explicit conversion from a base type or interface to a derived type fails at run time.
<code>System.NullReferenceException</code>	Thrown when a <code>null</code> reference is used in a way that causes the referenced object to be required.
<code>System.OutOfMemoryException</code>	Thrown when an attempt to allocate memory (via <code>new</code>) fails.
<code>System.OverflowException</code>	Thrown when an arithmetic operation in a checked context overflows.
<code>System.StackOverflowException</code>	Thrown when the execution stack is exhausted by having too many pending method calls; typically indicative of very deep or unbounded recursion.
<code>System.TypeInitializationException</code>	Thrown when a static constructor throws an exception, and no <code>catch</code> clauses exists to catch it.

24. Attributes

[*Note:* Much of the C# language enables the programmer to specify declarative information about the entities defined in the program. For example, the accessibility of a method in a class is specified by decorating it with the *method-modifiers* `public`, `protected`, `internal`, and `private`. *end note*]

C# enables programmers to invent new kinds of declarative information, called **attributes**. Programmers can then attach attributes to various program entities, and retrieve attribute information in a run-time environment. [*Note:* For instance, a framework might define a `HelpAttribute` attribute that can be placed on certain program elements (such as classes and methods) to provide a mapping from those program elements to their documentation. *end note*]

Attributes are defined through the declaration of attribute classes (§24.1), which may have positional and named parameters (§24.1.2). Attributes are attached to entities in a C# program using attribute specifications (§24.2), and can be retrieved at run-time as attribute instances (§24.3).

24.1 Attribute classes

A class that derives from the abstract class `System.Attribute`, whether directly or indirectly, is an **attribute class**. The declaration of an attribute class defines a new kind of attribute that can be placed on a declaration. By convention, attribute classes are named with a suffix of `Attribute`. Uses of an attribute may either include or omit this suffix.

24.1.1 Attribute usage

The attribute `AttributeUsage` (§24.4.1) is used to describe how an attribute class can be used.

`AttributeUsage` has a positional parameter (§24.1.2) that enables an attribute class to specify the kinds of declarations on which it can be used. [*Example:* The example

```
using System;
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Interface)]
public class SimpleAttribute: Attribute
{}
```

defines an attribute class named `SimpleAttribute` that can be placed on *class-declarations* and *interface-declarations* only. The example

```
[Simple] class Class1 {...}
[Simple] interface Interface1 {...}
```

shows several uses of the `Simple` attribute. Although this attribute is defined with the name `SimpleAttribute`, when this attribute is used, the `Attribute` suffix may be omitted, resulting in the short name `Simple`. Thus, the example above is semantically equivalent to the following

```
[SimpleAttribute] class Class1 {...}
[SimpleAttribute] interface Interface1 {...}
```

end example]

`AttributeUsage` has a named parameter (§24.1.2), called `AllowMultiple`, which indicates whether the attribute can be specified more than once for a given entity. If `AllowMultiple` for an attribute class is true, then that class is a **multi-use attribute class**, and can be specified more than once on an entity. If `AllowMultiple` for an attribute class is false or it is unspecified, then that class is a **single-use attribute class**, and can be specified at most once on an entity.

[*Example:* The example

```

1      using System;
2      [AttributeUsage(AttributeTargets.Class, AllowMultiple = true)]
3      public class AuthorAttribute: Attribute {
4          public AuthorAttribute(string name) {
5              this.name = name;
6          }
7          public string Name { get { return name;} }
8          private string name;
9      }

```

10 defines a multi-use attribute class named `AuthorAttribute`. The example

```

11      [Author("Brian Kernighan"), Author("Dennis Ritchie")]
12      class Class1 {...}

```

13 shows a class declaration with two uses of the `Author` attribute. *end example*

14 `AttributeUsage` has another named parameter (§24.1.2), called `Inherited`, which indicates whether the
 15 attribute, when specified on a base class, is also inherited by classes that derive from that base class. If
 16 `Inherited` for an attribute class is true, then that attribute is inherited. If `Inherited` for an attribute class
 17 is false or it is unspecified, then that attribute is not inherited.

18 An attribute class `X` not having an `AttributeUsage` attribute attached to it, as in

```

19      using System;
20      class X: Attribute { ... }

```

21 is equivalent to the following:

```

22      using System;
23      [AttributeUsage(AttributeTargets.All, AllowMultiple = false, Inherited =
24      true)]
25      class X: Attribute { ... }

```

26 24.1.2 Positional and named parameters

27 Attribute classes can have *positional parameters* and *named parameters*. Each public instance constructor
 28 for an attribute class defines a valid sequence of positional parameters for that attribute class. Each non-
 29 static public read-write field and property for an attribute class defines a named parameter for the attribute
 30 class.

31 [*Example:* The example

```

32      using System;
33      [AttributeUsage(AttributeTargets.Class)]
34      public class HelpAttribute: Attribute
35      {
36          public HelpAttribute(string url) { // url is a positional parameter
37              ...
38          }
39          public string Topic { // Topic is a named parameter
40              get {...}
41              set {...}
42          }
43          public string Url { get {...} }
44      }

```

45 defines an attribute class named `HelpAttribute` that has one positional parameter (`string url`) and one
 46 named parameter (`string Topic`). Although it is non-static and public, the property `Url` does not define a
 47 named parameter, since it is not read-write.

48 This attribute class might be used as follows:

```

49      [Help("http://www.mycompany.com/.../Class1.htm")]
50      class Class1 {
51      }

```

```

1      [Help("http://www.mycompany.com/.../Misc.htm", Topic ="Class2")]
2      class Class2 {
3      }
4  end example]

```

24.1.3 Attribute parameter types

The types of positional and named parameters for an attribute class are limited to the **attribute parameter types**, which are:

- One of the following types: `bool`, `byte`, `char`, `double`, `float`, `int`, `long`, `short`, `string`.
- The type `object`.
- The type `System.Type`.
- An enum type, provided it has public accessibility and the types in which it is nested (if any) also have public accessibility.
- Single-dimensional arrays of the above types.

24.2 Attribute specification

Attribute specification is the application of a previously defined attribute to a declaration. An attribute is a piece of additional declarative information that is specified for a declaration. Attributes can be specified at global scope (to specify attributes on the containing assembly) and for *type-declarations* (§16.5), *class-member-declarations* (§17.2), *interface-member-declarations* (§20.2), *enum-member-declarations* (§21.1), *accessor-declarations* (§17.6.2), *event-accessor-declarations* (§17.7), and *formal-parameter-lists* (§17.5.1).

Attributes are specified in **attribute sections**. An attribute section consists of a pair of square brackets, which surround a comma-separated list of one or more attributes. The order in which attributes are specified in such a list, and the order in which sections attached to the same program entity are arranged, is not significant. For instance, the attribute specifications `[A][B]`, `[B][A]`, `[A, B]`, and `[B, A]` are equivalent.

```

24      global-attributes:
25          global-attribute-sections
26
27      global-attribute-sections:
28          global-attribute-section
29          global-attribute-sections global-attribute-section
30
31      global-attribute-section:
32          [ global-attribute-target-specifier attribute-list ]
33          [ global-attribute-target-specifier attribute-list ,]
34
35      global-attribute-target-specifier:
36          global-attribute-target :
37
38      global-attribute-target:
39          assembly
40
41      attributes:
42          attribute-sections
43
44      attribute-sections:
45          attribute-section
46          attribute-sections attribute-section
47
48      attribute-section:
49          [ attribute-target-specifieropt attribute-list ]
50          [ attribute-target-specifieropt attribute-list ,]
51
52      attribute-target-specifier:
53          attribute-target :

```

```

1      attribute-target:
2          field
3          event
4          method
5          param
6          property
7          return
8          type
9
10     attribute-list:
11         attribute
12         attribute-list , attribute
13
14     attribute:
15         attribute-name attribute-argumentsopt
16
17     attribute-name:
18         type-name
19
20     attribute-arguments:
21         ( positional-argument-listopt )
22         ( positional-argument-list , named-argument-list )
23         ( named-argument-list )
24
25     positional-argument-list:
26         positional-argument
27         positional-argument-list , positional-argument
28
29     positional-argument:
30         attribute-argument-expression
31
32     named-argument-list:
33         named-argument
34         named-argument-list , named-argument
35
36     named-argument:
37         identifier = attribute-argument-expression
38
39     attribute-argument-expression:
40         expression

```

An attribute consists of an *attribute-name* and an optional list of positional and named arguments. The positional arguments (if any) precede the named arguments. A positional argument consists of an *attribute-argument-expression*; a named argument consists of a name, followed by an equal sign, followed by an *attribute-argument-expression*, which, together, are constrained by the same rules as simple assignment.) The order of named arguments is not significant.

The *attribute-name* identifies an attribute class. If the form of *attribute-name* is *type-name* then this name must refer to an attribute class. Otherwise, a compile-time error occurs. [Example: The example

```

class Class1 {}
[Class1] class Class2 {}    // Error

```

results in a compile-time error because it attempts to use Class1 as an attribute class when Class1 is not an attribute class. end example]

Certain contexts permit the specification of an attribute on more than one target. A program can explicitly specify the target by including an *attribute-target-specifier*. When an attribute is placed at the global level, a *global-attribute-target-specifier* is required. In all other locations, a reasonable default is applied, but an *attribute-target-specifier* can be used to affirm or override the default in certain ambiguous cases (or to just affirm the default in non-ambiguous cases). Thus, typically, *attribute-target-specifiers* can be omitted except at the global level. The potentially ambiguous contexts are resolved as follows:

- An attribute specified on a delegate declaration can apply either to the delegate being declared or to its return value. In the absence of an *attribute-target-specifier*, the attribute applies to the delegate. The presence of the `type` *attribute-target-specifier* indicates that the attribute applies to the delegate; the presence of the `return` *attribute-target-specifier* indicates that the attribute applies to the return value.
- An attribute specified on a method declaration can apply either to the method being declared or to its return value. In the absence of an *attribute-target-specifier*, the attribute applies to the method. The presence of the `method` *attribute-target-specifier* indicates that the attribute applies to the method; the presence of the `return` *attribute-target-specifier* indicates that the attribute applies to the return value.
- An attribute specified on an operator declaration can apply either to the operator being declared or to its return value of this declaration. In the absence of an *attribute-target-specifier*, the attribute applies to the operator. The presence of the `type` *attribute-target-specifier* indicates that the attribute applies to the operator; the presence of the `return` *attribute-target-specifier* indicates that the attribute applies to the return value.
- An attribute specified on an event declaration that omits event accessors can apply to the event being declared, to the associated field (if the event is not abstract), or to the associated add and remove methods. In the absence of an *attribute-target-specifier*, the attribute applies to the event declaration. The presence of the `event` *attribute-target-specifier* indicates that the attribute applies to the event; the presence of the `field` *attribute-target-specifier* indicates that the attribute applies to the field; and the presence of the `method` *attribute-target-specifier* indicates that the attribute applies to the methods.
- An attribute specified on a get accessor declaration for a property or indexer declaration can apply either to the associated method or to its return value. In the absence of an *attribute-target-specifier*, the attribute applies to the method. The presence of the `method` *attribute-target-specifier* indicates that the attribute applies to the method; the presence of the `return` *attribute-target-specifier* indicates that the attribute applies to the return value.
- An attribute specified on a set accessor for a property or indexer declaration can apply either to the associated method or to its lone implicit parameter. In the absence of an *attribute-target-specifier*, the attribute applies to the method. The presence of the `method` *attribute-target-specifier* indicates that the attribute applies to the method; the presence of the `param` *attribute-target-specifier* indicates that the attribute applies to the parameter.
- An attribute specified on an add or remove accessor declaration for an event declaration can apply either to the associated method or to its lone parameter. In the absence of an *attribute-target-specifier*, the attribute applies to the method. The presence of the `method` *attribute-target-specifier* indicates that the attribute applies to the method; the presence of the `param` *attribute-target-specifier* indicates that the attribute applies to the parameter.

An implementation may accept other attribute target specifiers, the purpose of which is implementation-defined. However, an implementation that does not recognize such a target, shall issue a warning.

By convention, attribute classes are named with a suffix of `Attribute`. An *attribute-name* of the form *type-name* may either include or omit this suffix. If an attribute class is found both with and without this suffix, an ambiguity is present, and a compile-time error shall be issued. If the *attribute-name* is spelled using a verbatim identifier (§9.4.2), then only an attribute without a suffix is matched, thus enabling such an ambiguity to be resolved. [Example: The example

```
using System;
[AttributeUsage(AttributeTargets.All)]
public class X: Attribute
{
}

[AttributeUsage(AttributeTargets.All)]
public class XAttribute: Attribute
{
}

[X]
class Class1 {}           // error: ambiguity
```

```

1      [XAttribute]      // refers to XAttribute
2      class Class2 {}
3      [@X]              // refers to X
4      class Class3 {}
5      [@XAttribute]     // refers to XAttribute
6      class Class4 {}

```

7 shows two attribute classes named `X` and `XAttribute`. The attribute `[X]` is ambiguous, since it could refer
8 to either `X` or `XAttribute`. Using a verbatim identifier allows the exact intent to be specified in such rare
9 cases. The attribute `[XAttribute]` is not ambiguous (although it would be if there was an attribute class
10 named `XAttributeAttribute!`). If the declaration for class `X` is removed, then both attributes refer to the
11 attribute class named `XAttribute`, as follows:

```

12      using System;
13      [AttributeUsage(AttributeTargets.All)]
14      public class XAttribute: Attribute
15      {}
16      [X]              // refers to XAttribute
17      class Class1 {}
18      [XAttribute]     // refers to XAttribute
19      class Class2 {}
20      [@X]             // error: no attribute named "X"
21      class Class3 {}

```

22 *end example]*

23 It is a compile-time error to use a single-use attribute class more than once on the same entity. *[Example:*
24 The example

```

25      using System;
26      [AttributeUsage(AttributeTargets.Class)]
27      public class HelpStringAttribute: Attribute
28      {
29          string value;
30          public HelpStringAttribute(string value) {
31              this.value = value;
32          }
33          public string value { get {...} }
34      }
35      [HelpString("Description of Class1")]
36      [HelpString("Another description of Class1")]
37      public class Class1 {}

```

38 results in a compile-time error because it attempts to use `HelpString`, which is a single-use attribute class,
39 more than once on the declaration of `Class1`. *end example]*

40 An expression `E` is an *attribute-argument-expression* if all of the following statements are true:

- 41 • The type of `E` is an attribute parameter type (§24.1.3).
- 42 • At compile-time, the value of `E` can be resolved to one of the following:
 - 43 ○ A constant value.
 - 44 ○ A `System.Type` object.
 - 45 ○ A one-dimensional array of *attribute-argument-expressions*.

46 *[Example: For example:*

```

1      using System;
2      [AttributeUsage(AttributeTargets.Class)]
3      public class MyAttribute: Attribute
4      {
5          public int P1 {
6              get {...}
7              set {...}
8          }
9          public Type P2 {
10             get {...}
11             set {...}
12         }
13         public object P3 {
14             get {...}
15             set {...}
16         }
17     }
18     [My(P1 = 1234, P3 = new int[]{1, 3, 5}, P2 = typeof(float))]
19     class MyClass {}
20 end example]

```

24.3 Attribute instances

An **attribute instance** is an instance that represents an attribute at run-time. An attribute is defined with an attribute class, positional arguments, and named arguments. An attribute instance is an instance of the attribute class that is initialized with the positional and named arguments.

Retrieval of an attribute instance involves both compile-time and run-time processing, as described in the following sections.

24.3.1 Compilation of an attribute

The compilation of an *attribute* with attribute class *T*, *positional-argument-list* *P* and *named-argument-list* *N*, consists of the following steps:

- Follow the compile-time processing steps for compiling an *object-creation-expression* of the form `new T(P)`. These steps either result in a compile-time error, or determine an instance constructor on *T* that can be invoked at run-time. Call this instance constructor *C*.
- If *C* does not have public accessibility, then a compile-time error occurs.
- For each *named-argument* *Arg* in *N*:
 - Let *Name* be the *identifier* of the *named-argument* *Arg*.
 - *Name* must identify a non-static read-write public field or property on *T*. If *T* has no such field or property, then a compile-time error occurs.
- Keep the following information for run-time instantiation of the attribute: the attribute class *T*, the instance constructor *C* on *T*, the *positional-argument-list* *P* and the *named-argument-list* *N*.

24.3.2 Run-time retrieval of an attribute instance

Compilation of an *attribute* yields an attribute class *T*, an instance constructor *C* on *T*, a *positional-argument-list* *P*, and a *named-argument-list* *N*. Given this information, an attribute instance can be retrieved at run-time using the following steps:

- Follow the run-time processing steps for executing an *object-creation-expression* of the form `new T(P)`, using the instance constructor `C` as determined at compile-time. These steps either result in an exception, or produce an instance of `T`. Call this instance `O`.
- For each *named-argument* `Arg` in `N`, in order:
 - Let `Name` be the *identifier* of the *named-argument* `Arg`. If `Name` does not identify a non-static public read-write field or property on `O`, then an exception is thrown.
 - Let `Value` be the result of evaluating the *attribute-argument-expression* of `Arg`.
 - If `Name` identifies a field on `O`, then set this field to the value `Value`.
 - Otherwise, `Name` identifies a property on `O`. Set this property to the value `Value`.
 - The result is `O`, an instance of the attribute class `T` that has been initialized with the *positional-argument-list* `P` and the *named-argument-list* `N`.

24.4 Reserved attributes

A small number of attributes affect the language in some way. These attributes include:

- `System.AttributeUsageAttribute` (§24.4.1), which is used to describe the ways in which an attribute class can be used.
- `System.ConditionalAttribute` (§24.4.2), which is used to define conditional methods.
- `System.ObsoleteAttribute` (§24.4.3), which is used to mark a member as obsolete.

24.4.1 The `AttributeUsage` attribute

The attribute `AttributeUsage` is used to describe the manner in which the attribute class can be used.

A class that is decorated with the `AttributeUsage` attribute must derive from `System.Attribute`, either directly or indirectly. Otherwise, a compile-time error occurs.

[*Note:* For an example of using this attribute, see §24.1.1. *end note*]

24.4.2 The `Conditional` attribute

The attribute `Conditional` enables the definition of *conditional methods*. The `Conditional` attribute indicates a condition by testing a conditional compilation symbol. Calls to a conditional method are either included or omitted depending on whether this symbol is defined at the point of the call. If the symbol is defined, the call is included; otherwise, the call is omitted.

A conditional method is subject to the following restrictions:

- The conditional method must be a method in a *class-declaration*. A compile-time error occurs if the `Conditional` attribute is specified on an interface method.
- The conditional method must have a return type of `void`.
- The conditional method must not be marked with the `override` modifier. A conditional method may be marked with the `virtual` modifier, however. Overrides of such a method are implicitly conditional, and must not be explicitly marked with a `Conditional` attribute.
- The conditional method must not be an implementation of an interface method. Otherwise, a compile-time error occurs.

In addition, a compile-time error occurs if a conditional method is used in a *delegate-creation-expression*.

[*Example:* The example

```
#define DEBUG
```



```

1      using System;
2      using System.Diagnostics;
3      class Class1
4      {
5          [Conditional("DEBUG")]
6          public static void M() {
7              Console.WriteLine("Executed Class1.M");
8          }
9      }
10     class Class2
11     {
12         public static void Test() {
13             Class1.M();
14         }
15     }

```

16 declares `Class1.M` as a conditional method. `Class2`'s `Test` method calls this method. Since the
17 conditional compilation symbol `DEBUG` is defined, if `Class2.Test` is called, it will call `M`. If the symbol
18 `DEBUG` had not been defined, then `Class2.Test` would not call `Class1.M`. *end example*

19 It is important to note that the inclusion or exclusion of a call to a conditional method is controlled by the
20 conditional compilation symbols at the point of the call. [*Example:* In the example

```

21     // Begin class1.cs
22     using System;
23     using System.Diagnostics;
24     class Class1
25     {
26         [Conditional("DEBUG")]
27         public static void F() {
28             Console.WriteLine("Executed Class1.F");
29         }
30     }
31     // End class1.cs
32
33     // Begin class2.cs
34     #define DEBUG
35     class Class2
36     {
37         public static void G() {
38             Class1.F();           // F is called
39         }
40     }
41     // End class2.cs
42
43     // Begin class3.cs
44     #undef DEBUG
45     class Class3
46     {
47         public static void H() {
48             Class1.F();           // F is not called
49         }
50     }
51     // End class3.cs

```

52 the classes `Class2` and `Class3` each contain calls to the conditional method `Class1.F`, which is
53 conditional based on whether or not `DEBUG` is defined. Since this symbol is defined in the context of `Class2`
54 but not `Class3`, the call to `F` in `Class2` is included, while the call to `F` in `Class3` is omitted. *end example*

55 The use of conditional methods in an inheritance chain can be confusing. Calls made to a conditional
56 method through base, of the form `base.M`, are subject to the normal conditional method call rules.
57 [*Example:* In the example

```

1      // Begin class1.cs
2      using System;
3      using System.Diagnostics;
4      class Class1
5      {
6          [Conditional("DEBUG")]
7          public virtual void M() {
8              Console.WriteLine("Class1.M executed");
9          }
10     }
11     // End class1.cs
12
13     // Begin class2.cs
14     using System;
15     class Class2: Class1
16     {
17         public override void M() {
18             Console.WriteLine("Class2.M executed");
19             base.M();           // base.M is not called!
20         }
21     }
22     // End class2.cs
23
24     // Begin class3.cs
25     #define DEBUG
26
27     using System;
28     class Class3
29     {
30         public static void Test() {
31             Class2 c = new Class2();
32             c.M();               // M is called
33         }
34     }
35     // End class3.cs

```

Class2 includes a call to the M defined in its base class. This call is omitted because the base method is conditional based on the presence of the symbol `DEBUG`, which is undefined. Thus, the method writes to the console “Class2.M executed” only. Judicious use of *pp-declarations* can eliminate such problems. *end example*

24.4.3 The `Obsolete` attribute

The attribute `Obsolete` is used to mark types and members of types that should no longer be used.

```

41     using System;
42     [AttributeUsage(AttributeTargets.Class | AttributeTargets.Struct |
43         AttributeTargets.Enum | AttributeTargets.Interface |
44         AttributeTargets.Delegate | AttributeTargets.Method |
45         AttributeTargets.Constructor | AttributeTargets.Property |
46         AttributeTargets.Field | AttributeTargets.Event)]
47     public class ObsoleteAttribute: Attribute
48     {
49         public ObsoleteAttribute() {...}
50         public ObsoleteAttribute(string message) {...}
51         public ObsoleteAttribute(string message, bool error) {...}
52         public string Message { get {...} }
53         public bool IsError { get {...} }
54     }

```

If a program uses a type or member that is decorated with the `Obsolete` attribute, then the compiler shall issue a warning or error in order to alert the developer, so the offending code can be fixed. Specifically, the compiler shall issue a warning if no error parameter is provided, or if the error parameter is provided and has the value `false`. The compiler shall issue a compile-time error if the error parameter is specified and has the value `true`.

```
1  [Example: In the example
2      [Obsolete("This class is obsolete; use class B instead")]
3      class A
4      {
5          public void F() {}
6      }
7
8      class B
9      {
10         public void F() {}
11     }
12
13     class Test
14     {
15         static void Main() {
16             A a = new A(); // warning
17             a.F();
18         }
19     }
20 ]
```

the class A is decorated with the `Obsolete` attribute. Each use of A in `Main` results in a warning that includes the specified message, "This class is obsolete; use class B instead."

end example]

25. Unsafe code

An implementation that does not support unsafe code is required to diagnose any usage of the keyword `unsafe`.

This remainder of this clause is conditionally normative.

[*Note:* The core C# language, as defined in the preceding chapters, differs notably from C and C++ in its omission of pointers as a data type. Instead, C# provides references and the ability to create objects that are managed by a garbage collector. This design, coupled with other features, makes C# a much safer language than C or C++. In the core C# language it is simply not possible to have an uninitialized variable, a “dangling” pointer, or an expression that indexes an array beyond its bounds. Whole categories of bugs that routinely plague C and C++ programs are thus eliminated.

While practically every pointer type construct in C or C++ has a reference type counterpart in C#, nonetheless, there are situations where access to pointer types becomes a necessity. For example, interfacing with the underlying operating system, accessing a memory-mapped device, or implementing a time-critical algorithm may not be possible or practical without access to pointers. To address this need, C# provides the ability to write **unsafe code**.

In unsafe code it is possible to declare and operate on pointers, to perform conversions between pointers and integral types, to take the address of variables, and so forth. In a sense, writing unsafe code is much like writing C code within a C# program.

Unsafe code is in fact a “safe” feature from the perspective of both developers and users. Unsafe code must be clearly marked with the modifier `unsafe`, so developers can’t possibly use unsafe features accidentally, and the execution engine works to ensure that unsafe code cannot be executed in an untrusted environment. *end note*]

25.1 Unsafe contexts

The unsafe features of C# are available only in *unsafe contexts*. An unsafe context is introduced by including an `unsafe` modifier in the declaration of a type or member, or by employing an *unsafe-statement*:

- A declaration of a class, struct, interface, or delegate may include an `unsafe` modifier, in which case the entire textual extent of that type declaration (including the body of the class, struct, or interface) is considered an unsafe context.
- A declaration of a field, method, property, event, indexer, operator, instance constructor, destructor, or static constructor may include an `unsafe` modifier, in which case, the entire textual extent of that member declaration is considered an unsafe context.
- An *unsafe-statement* enables the use of an unsafe context within a *block*. The entire textual extent of the associated *block* is considered an unsafe context.

The associated grammar extensions are shown below. For brevity, ellipses (...) are used to represent productions that appear in preceding chapters.

class-modifier:

...

`unsafe`

struct-modifier:

...

`unsafe`

interface-modifier:

...

unsafe

delegate-modifier:

...

unsafe

field-modifier:

...

unsafe

method-modifier:

...

unsafe

property-modifier:

...

unsafe

event-modifier:

...

unsafe

indexer-modifier:

...

unsafe

operator-modifier:

...

unsafe

constructor-modifier:

...

unsafe

destructor-declaration:

*attributes*_{opt} **extern**_{opt} **unsafe**_{opt} ~ *identifier* () *destructor-body*

*attributes*_{opt} **unsafe**_{opt} **extern**_{opt} ~ *identifier* () *destructor-body*

static-constructor-declaration:

*attributes*_{opt} **extern**_{opt} **unsafe**_{opt} **static** *identifier* () *static-constructor-body*

*attributes*_{opt} **unsafe**_{opt} **extern**_{opt} **static** *identifier* () *static-constructor-body*

embedded-statement:

...

unsafe-statement

unsafe-statement:

unsafe *block*

- [Example: In the example

```
public unsafe struct Node
{
    public int value;
    public Node* Left;
    public Node* Right;
}
```

the **unsafe** modifier specified in the struct declaration causes the entire textual extent of the struct declaration to become an unsafe context. Thus, it is possible to declare the `Left` and `Right` fields to be of a pointer type. The example above could also be written

```

1      public struct Node
2      {
3          public int Value;
4          public unsafe Node* Left;
5          public unsafe Node* Right;
6      }

```

7 Here, the `unsafe` modifiers in the field declarations cause those declarations to be considered unsafe contexts. *end example*

9 Other than establishing an unsafe context, thus permitting the use of pointer types, the `unsafe` modifier has no effect on a type or a member. [*Example:* In the example

```

11     public class A
12     {
13         public unsafe virtual void F() {
14             char* p;
15             ...
16         }
17     }
18     public class B: A
19     {
20         public override void F() {
21             base.F();
22             ...
23         }
24     }

```

25 the `unsafe` modifier on the `F` method in `A` simply causes the textual extent of `F` to become an unsafe context in which the unsafe features of the language can be used. In the override of `F` in `B`, there is no need to re-specify the `unsafe` modifier—unless, of course, the `F` method in `B` itself needs access to unsafe features.

29 The situation is slightly different when a pointer type is part of the method’s signature

```

30     public unsafe class A
31     {
32         public virtual void F(char* p) {...}
33     }
34     public class B: A
35     {
36         public unsafe override void F(char* p) {...}
37     }

```

38 Here, because `F`’s signature includes a pointer type, it can only be written in an unsafe context. However, the unsafe context can be introduced by either making the entire class unsafe, as is the case in `A`, or by including an `unsafe` modifier in the method declaration, as is the case in `B`. *end example*

41 25.2 Pointer types

42 In an unsafe context, a *type* (§11) may be a *pointer-type* as well as a *value-type* or a *reference-type*.

```

43     type:
44         value-type
45         reference-type
46         pointer-type

```

47 A *pointer-type* is written as an *unmanaged-type* or the keyword `void`, followed by a `*` token:

```

48     pointer-type:
49         unmanaged-type *
50         void *
51
52     unmanaged-type:
53         type

```

The type specified before the `*` in a pointer type is called the **referent type** of the pointer type. It represents the type of the variable to which a value of the pointer type points.

Unlike references (values of reference types), pointers are not tracked by the garbage collector—the garbage collector has no knowledge of pointers and the data to which they point. For this reason a pointer is not permitted to point to a reference or to a struct that contains references, and the referent type of a pointer must be an *unmanaged-type*.

An *unmanaged-type* is any type that isn't a *reference-type* and doesn't contain *reference-type* fields at any level of nesting. In other words, an *unmanaged-type* is one of the following:

- `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, `decimal`, or `bool`.
- Any *enum-type*.
- Any *pointer-type*.
- Any user-defined *struct-type* that contains fields of *unmanaged-types* only.

The intuitive rule for mixing of pointers and references is that referents of references (objects) are permitted to contain pointers, but referents of pointers are not permitted to contain references.

[*Example:* Some examples of pointer types are given in the table below:

Example	Description
<code>byte*</code>	Pointer to <code>byte</code>
<code>char*</code>	Pointer to <code>char</code>
<code>int**</code>	Pointer to pointer to <code>int</code>
<code>int*[]</code>	Single-dimensional array of pointers to <code>int</code>
<code>void*</code>	Pointer to unknown type

end example]

For a given implementation, all pointer types must have the same size and representation.

[*Note:* Unlike C and C++, when multiple pointers are declared in the same declaration, in C# the `*` is written along with the underlying type only, not as a prefix punctuator on each pointer name. For example:

```
int* pi, pj;    // NOT as int *pi, *pj;
```

end note]

The value of a pointer having type `T*` represents the *address* of a variable of type `T`. The pointer indirection operator `*` (§25.5.1) may be used to access this variable. For example, given a variable `P` of type `int*`, the expression `*P` denotes the `int` variable found at the address contained in `P`.

Like an object reference, a pointer may be `null`. Applying the indirection operator to a `null` pointer results in implementation-defined behavior. A pointer with value `null` is represented by all-bits-zero.

The `void*` type represents a pointer to an unknown type. Because the referent type is unknown, the indirection operator cannot be applied to a pointer of type `void*`, nor can any arithmetic be performed on such a pointer. However, a pointer of type `void*` can be cast to any other pointer type (and vice versa).

Pointer types are a separate category of types. Unlike reference types and value types, pointer types do not inherit from `object` and no conversions exist between pointer types and `object`. In particular, boxing and unboxing (§11.3) are not supported for pointers. However, conversions are permitted between different pointer types and between pointer types and the integral types. This is described in §25.4.

A *pointer-type* may be used as the type of a volatile field (§17.4.3).

[Note: Although pointers can be passed as `ref` or `out` parameters, doing so can cause undefined behavior, since the pointer may well be set to point to a local variable which no longer exists when the called method returns, or the fixed object to which it used to point, is no longer fixed. For example:

```

using System;
class Test
{
    static int value = 20;

    unsafe static void F(out int* pi1, ref int* pi2) {
        int i = 10;
        pi1 = &i;

        fixed (int* pj = &value) {
            // ...
            pi2 = pj;
        }
    }

    static void Main() {
        int i = 10;
        unsafe {
            int* px1;
            int* px2 = &i;

            F(out px1, ref px2);
            Console.WriteLine("*px1 = {0}, *px2 = {1}",
                *px1, *px2); // undefined behavior
        }
    }
}

```

end note

A method can return a value of some type, and that type can be a pointer. [Example: For example, when given a pointer to a contiguous sequence of `ints`, that sequence's element count, and some other `int` value, the following method returns the address of that value in that sequence, if a match occurs; otherwise it returns `null`:

```

unsafe static int* Find(int* pi, int size, int value) {
    for (int i = 0; i < size; ++i) {
        if (*pi == value) {
            return pi;
        }
        ++pi;
    }
    return null;
}

```

end example

In an unsafe context, several constructs are available for operating on pointers:

- The `*` operator may be used to perform pointer indirection (§25.5.1).
- The `->` operator may be used to access a member of a struct through a pointer (§25.5.2).
- The `[]` operator may be used to index a pointer (§25.5.3).
- The `&` operator may be used to obtain the address of a variable (§25.5.4).
- The `++` and `--` operators may be used to increment and decrement pointers (§25.5.5).
- The `+` and `-` operators may be used to perform pointer arithmetic (§25.5.6).
- The `==`, `!=`, `<`, `>`, `<=`, and `=>` operators may be used to compare pointers (§25.5.7).
- The `stackalloc` operator may be used to allocate memory from the call stack (§25.7).
- The `fixed` statement may be used to temporarily fix a variable so its address can be obtained (§25.6).

25.3 Fixed and moveable variables

The address-of operator (§25.5.4) and the `fixed` statement (§25.6) divide variables into two categories: ***Fixed variables*** and ***moveable variables***.

Fixed variables reside in storage locations that are unaffected by operation of the garbage collector. (Examples of fixed variables include local variables, value parameters, and variables created by dereferencing pointers.) On the other hand, moveable variables reside in storage locations that are subject to relocation or disposal by the garbage collector. (Examples of moveable variables include fields in objects and elements of arrays.)

The `&` operator (§25.5.4) permits the address of a fixed variable to be obtained without restrictions. However, because a moveable variable is subject to relocation or disposal by the garbage collector, the address of a moveable variable can only be obtained using a `fixed` statement (§25.6), and that address remains valid only for the duration of that `fixed` statement.

In precise terms, a fixed variable is one of the following:

- A variable resulting from a *simple-name* (§14.5.2) that refers to a local variable or a value parameter.
- A variable resulting from a *member-access* (§14.5.4) of the form `V.I`, where `V` is a fixed variable of a *struct-type*.
- A variable resulting from a *pointer-indirection-expression* (§25.5.1) of the form `*P`, a *pointer-member-access* (§25.5.2) of the form `P->I`, or a *pointer-element-access* (§25.5.3) of the form `P[E]`.

All other variables are classified as moveable variables.

Note that a static field is classified as a moveable variable. Also note that a `ref` or `out` parameter is classified as a moveable variable, even if the argument given for the parameter is a fixed variable. Finally, note that a variable produced by dereferencing a pointer is always classified as a fixed variable.

25.4 Pointer conversions

In an unsafe context, the set of available implicit conversions (§13.1) is extended to include the following implicit pointer conversions:

- From any *pointer-type* to the type `void*`.
- From the null type to any *pointer-type*.

Additionally, in an unsafe context, the set of available explicit conversions (§13.2) is extended to include the following explicit pointer conversions:

- 1 • From any *pointer-type* to any other *pointer-type*.
 - 2 • From `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, or `ulong` to any *pointer-type*.
 - 3 • From any *pointer-type* to `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, or `ulong`.
- 4 Finally, in an unsafe context, the set of standard implicit conversions (§13.3.1) includes the following
5 pointer conversion:

- 6 • From any *pointer-type* to the type `void*`.

7 Conversions between two pointer types never change the actual pointer value. In other words, a conversion
8 from one pointer type to another has no effect on the underlying address given by the pointer.

9 When one pointer type is converted to another, if the resulting pointer is not correctly aligned for the
10 pointed-to type, the behavior is undefined if the result is dereferenced. In general, the concept “correctly
11 aligned” is transitive: if a pointer to type A is correctly aligned for a pointer to type B, which, in turn, is
12 correctly aligned for a pointer to type C, then a pointer to type A is correctly aligned for a pointer to type C.
13 [Example: Consider the following case in which a variable having one type is accessed via a pointer to a
14 different type:

```
15     char c = 'A';
16     char* pc = &c;
17     void* pv = pc;
18     int* pi = (int*)pv;
19     int i = *pi;           // undefined
20     *pi = 123456;         // undefined
```

21 *end example]*

22 When a pointer type is converted to a pointer to `byte`, the result points to the lowest addressed byte of the
23 variable. Successive increments of the result, up to the size of the variable, yield pointers to the remaining
24 bytes of that variable. [Example: For example, the following method displays each of the eight bytes in a
25 `double` as a hexadecimal value:

```
26     using System;
27     class Test
28     {
29         static void Main() {
30             double d = 123.456e23;
31             unsafe {
32                 byte* pb = (byte*)&d;
33
34                 for (int i = 0; i < sizeof(double); ++i)
35                     Console.Write("{0,2:X}", (uint)(*pb++));
36                 Console.WriteLine();
37             }
38         }
39     }
```

40 Of course, the output produced depends on endianness. *end example]*

41 Mappings between pointers and integers are implementation-defined. [Note: However, on 32- and 64-bit
42 CPU architectures with a linear address space, conversions of pointers to or from integral types typically
43 behave exactly like conversions of `uint` or `ulong` values, respectively, to or from those integral types. *end*
44 *note]*

45 25.5 Pointers in expressions

46 In an unsafe context, an expression may yield a result of a pointer type, but outside an unsafe context it is a
47 compile-time error for an expression to be of a pointer type. In precise terms, outside an unsafe context a
48 compile-time error occurs if any *simple-name* (§14.5.2), *member-access* (§14.5.4), *invocation-expression*
49 (§14.5.5), or *element-access* (§14.5.6) is of a pointer type.

50 In an unsafe context, the *primary-no-array-creation-expression* (§14.5) and *unary-expression* (§14.6)
51 productions permit the following additional constructs:

primary-no-array-creation-expression:

...
pointer-member-access
pointer-element-access
sizeof-expression

unary-expression:

...
pointer-indirection-expression
addressof-expression

These constructs are described in the following sections.

[*Note:* The precedence and associativity of the unsafe operators is implied by the grammar. *end note*]

25.5.1 Pointer indirection

A *pointer-indirection-expression* consists of an asterisk (*) followed by a *unary-expression*.

pointer-indirection-expression:
 * *unary-expression*

The unary * operator denotes *pointer indirection* and is used to obtain the variable to which a pointer points. The result of evaluating *P, where P is an expression of a pointer type T*, is a variable of type T. It is a compile-time error to apply the unary * operator to an expression of type void* or to an expression that isn't of a pointer type.

The effect of applying the unary * operator to a null pointer is implementation-defined. In particular, there is no guarantee that this operation throws a `System.NullReferenceException`.

If an invalid value has been assigned to the pointer, the behavior of the unary * operator is undefined. [*Note:* Among the invalid values for dereferencing a pointer by the unary * operator are an address inappropriately aligned for the type pointed to (see example in §25.4), and the address of a variable after the end of its lifetime. *end note*]

For purposes of definite assignment analysis, a variable produced by evaluating an expression of the form *P is considered initially assigned (§12.3.1).

25.5.2 Pointer member access

A *pointer-member-access* consists of a *primary-expression*, followed by a “->” token, followed by an *identifier*.

pointer-member-access:
primary-expression -> *identifier*

In a pointer member access of the form P->I, P must be an expression of a pointer type other than void*, and I must denote an accessible member of the type to which P points.

A pointer member access of the form P->I is evaluated exactly as (*P).I. For a description of the pointer indirection operator (*), see §25.5.1. For a description of the member access operator (.), see §14.5.4.

[*Example:* In the example

```
struct Point
{
    public int x;
    public int y;
    public override string ToString() {
        return "(" + x + "," + y + ")";
    }
}
```

```

1      using System;
2      class Test
3      {
4          static void Main() {
5              Point point;
6              unsafe {
7                  Point* p = &point;
8                  p->x = 10;
9                  p->y = 20;
10                 Console.WriteLine(p->ToString());
11             }
12         }
13     }

```

the `->` operator is used to access fields and invoke a method of a struct through a pointer. Because the operation `P->I` is precisely equivalent to `(*P).I`, the `Main` method could equally well have been written:

```

16     using System;
17     class Test
18     {
19         static void Main() {
20             Point point;
21             unsafe {
22                 Point* p = &point;
23                 (*p).x = 10;
24                 (*p).y = 20;
25                 Console.WriteLine((*p).ToString());
26             }
27         }
28     }

```

29 *end example]*

30 25.5.3 Pointer element access

31 A *pointer-element-access* consists of a *primary-no-array-creation-expression* followed by an expression
32 enclosed in “[” and “]”.

33 *pointer-element-access:*
34 *primary-no-array-creation-expression* [*expression*]

35 In a pointer element access of the form `P[E]`, `P` must be an expression of a pointer type other than `void*`,
36 and `E` must be an expression of a type that can be implicitly converted to `int`, `uint`, `long`, or `ulong`.

37 A pointer element access of the form `P[E]` is evaluated exactly as `*(P + E)`. For a description of the pointer
38 indirection operator (`*`), see §25.5.1. For a description of the pointer addition operator (`+`), see §25.5.6.

39 [*Example:* In the example

```

40     class Test
41     {
42         static void Main() {
43             unsafe {
44                 char* p = stackalloc char[256];
45                 for (int i = 0; i < 256; i++) p[i] = (char)i;
46             }
47         }
48     }

```

49 a pointer element access is used to initialize the character buffer in a `for` loop. Because the operation `P[E]`
50 is precisely equivalent to `*(P + E)`, the example could equally well have been written:

```

1      class Test
2      {
3          static void Main() {
4              unsafe {
5                  char* p = stackalloc char[256];
6                  for (int i = 0; i < 256; i++) *(p + i) = (char)i;
7              }
8          }
9      }

```

10 *end example]*

11 The pointer element access operator does not check for out-of-bounds errors and the behavior when
 12 accessing an out-of-bounds element is undefined. [*Note:* This is the same as C and C++. *end note]*

13 25.5.4 The address-of operator

14 An *addressof-expression* consists of an ampersand (&) followed by a *unary-expression*.

15 *addressof-expression:*
 16 & *unary-expression*

17 Given an expression E which is of a type T and is classified as a fixed variable (§25.3), the construct &E
 18 computes the address of the variable given by E. The type of the result is T* and is classified as a value. A
 19 compile-time error occurs if E is not classified as a variable, if E is classified as a volatile field (§17.4.3), or
 20 if E denotes a moveable variable. In the last case, a fixed statement (§25.6) can be used to temporarily “fix”
 21 the variable before obtaining its address.

22 The & operator does not require its argument to be definitely assigned, but following an & operation, the
 23 variable to which the operator is applied is considered definitely assigned in the execution path in which the
 24 operation occurs. It is the responsibility of the programmer to ensure that correct initialization of the variable
 25 actually does take place in this situation.

26 [*Example:* In the example

```

27      using System;
28      class Test
29      {
30          static void Main() {
31              int i;
32              unsafe {
33                  int* p = &i;
34                  *p = 123;
35              }
36              Console.WriteLine(i);
37          }
38      }

```

39 i is considered definitely assigned following the &i operation used to initialize p. The assignment to *p in
 40 effect initializes i, but the inclusion of this initialization is the responsibility of the programmer, and no
 41 compile-time error would occur if the assignment was removed. *end example]*

42 [*Note:* The rules of definite assignment for the & operator exist such that redundant initialization of local
 43 variables can be avoided. For example, many external APIs take a pointer to a structure which is filled in by
 44 the API. Calls to such APIs typically pass the address of a local struct variable, and without the rule,
 45 redundant initialization of the struct variable would be required. *end note]*

46 [*Note:* As stated in §14.5.4, outside an instance constructor or static constructor for a struct or class that
 47 defines a readonly field, that field is considered a value, not a variable. As such, its address cannot be taken.
 48 Similarly, the address of a constant cannot be taken. *end note]*

49 25.5.5 Pointer increment and decrement

50 In an unsafe context, the ++ and -- operators (§14.5.9 and §14.6.5) can be applied to pointer variables of all
 51 types except void*. Thus, for every pointer type T*, the following operators are implicitly defined:

```

1      T* operator ++(T* x);
2      T* operator --(T* x);

```

3 The operators produce the same results as `x+1` and `x-1`, respectively (§25.5.6). In other words, for a pointer variable of type `T*`, the `++` operator adds `sizeof(T)` to the address contained in the variable, and the

4 `--` operator subtracts `sizeof(T)` from the address contained in the variable.

6 If a pointer increment or decrement operation overflows the domain of the pointer type, the result is implementation-defined, but no exceptions are produced.

8 25.5.6 Pointer arithmetic

9 In an unsafe context, the `+` operator (§14.7.4) and `-` operator (§14.7.5) can be applied to values of all pointer types except `void*`. Thus, for every pointer type `T*`, the following operators are implicitly defined:

```

11     T* operator +(T* x, int y);
12     T* operator +(T* x, uint y);
13     T* operator +(T* x, long y);
14     T* operator +(T* x, ulong y);

15     T* operator +(int x, T* y);
16     T* operator +(uint x, T* y);
17     T* operator +(long x, T* y);
18     T* operator +(ulong x, T* y);

19     T* operator -(T* x, int y);
20     T* operator -(T* x, uint y);
21     T* operator -(T* x, long y);
22     T* operator -(T* x, ulong y);

23     long operator -(T* x, T* y);

```

24 Given an expression `P` of a pointer type `T*` and an expression `N` of type `int`, `uint`, `long`, or `ulong`, the expressions `P + N` and `N + P` compute the pointer value of type `T*` that results from adding

26 `N * sizeof(T)` to the address given by `P`. Likewise, the expression `P - N` computes the pointer value of type `T*` that results from subtracting `N * sizeof(T)` from the address given by `P`.

28 Given two expressions, `P` and `Q`, of a pointer type `T*`, the expression `P - Q` computes the difference between the addresses given by `P` and `Q` and then divides that difference by `sizeof(T)`. The type of the result is always `long`. In effect, `P - Q` is computed as `((long)(P) - (long)(Q)) / sizeof(T)`.

31 [Example: For example:

```

32
33     using System;
34     class Test
35     {
36         static void Main() {
37             unsafe {
38                 int* values = stackalloc int[20];
39
40                 int* p = &values[1];
41                 int* q = &values[15];
42
43                 Console.WriteLine("p - q = {0}", p - q);
44                 Console.WriteLine("q - p = {0}", q - p);
45             }
46         }
47     }

```

48 which produces the output:

```

49     p - q = -14
50     q - p = 14

```

51 end example]

52 If a pointer arithmetic operation overflows the domain of the pointer type, the result is truncated in an implementation-defined fashion, but no exceptions are produced.

25.5.7 Pointer comparison

In an unsafe context, the `==`, `!=`, `<`, `>`, `<=`, and `>=` operators (§14.9) can be applied to values of all pointer types. The pointer comparison operators are:

```
bool operator ==(void* x, void* y);
bool operator !=(void* x, void* y);
bool operator <(void* x, void* y);

bool operator >(void* x, void* y);
bool operator <=(void* x, void* y);
bool operator >=(void* x, void* y);
```

Because an implicit conversion exists from any pointer type to the `void*` type, operands of any pointer type can be compared using these operators. The comparison operators compare the addresses given by the two operands as if they were unsigned integers.

25.5.8 The `sizeof` operator

The `sizeof` operator returns the number of bytes occupied by a variable of a given type. The type specified as an operand to `sizeof` must be an *unmanaged-type* (§25.2).

sizeof-expression:

```
sizeof ( unmanaged-type )
```

The result of the `sizeof` operator is a value of type `int`. For certain predefined types, the `sizeof` operator yields a constant value as shown in the table below.

Expression	Result
<code>sizeof(sbyte)</code>	1
<code>sizeof(byte)</code>	1
<code>sizeof(short)</code>	2
<code>sizeof(ushort)</code>	2
<code>sizeof(int)</code>	4
<code>sizeof(uint)</code>	4
<code>sizeof(long)</code>	8
<code>sizeof(ulong)</code>	8
<code>sizeof(char)</code>	2
<code>sizeof(float)</code>	4
<code>sizeof(double)</code>	8
<code>sizeof(bool)</code>	1

For all other types, the result of the `sizeof` operator is implementation-defined and is classified as a value, not a constant.

The order in which members are packed into a struct is unspecified.

For alignment purposes, there may be unnamed padding at the beginning of a struct, within a struct, and at the end of the struct. The contents of the bits used as padding are indeterminate.

When applied to an operand that has struct type, the result is the total number of bytes in a variable of that type, including any padding.

25.6 The `fixed` statement

In an unsafe context, the *embedded-statement* (§15) production permits an additional construct, the `fixed` statement, which is used to “fix” a moveable variable such that its address remains constant for the duration of the statement.


```

1      embedded-statement:
2          ...
3          fixed-statement
4
5      fixed-statement:
6          fixed ( pointer-type fixed-pointer-declarators ) embedded-statement
7
8      fixed-pointer-declarators:
9          fixed-pointer-declarator
10         fixed-pointer-declarators , fixed-pointer-declarator
11
12     fixed-pointer-declarator:
13         identifier = fixed-pointer-initializer
14
15     fixed-pointer-initializer:
16         & variable-reference
17         expression

```

Each *fixed-pointer-declarator* declares a local variable of the given *pointer-type* and initializes that local variable with the address computed by the corresponding *fixed-pointer-initializer*. A local variable declared in a **fixed** statement is accessible in any *fixed-pointer-initializers* occurring to the right of that variable's declaration, and in the *embedded-statement* of the **fixed** statement. A local variable declared by a **fixed** statement is considered read-only. A compile-time error occurs if the embedded statement attempts to modify this local variable (via assignment or the ++ and -- operators) or pass it as a **ref** or **out** parameter.

A *fixed-pointer-initializer* can be one of the following:

- The token "&" followed by a *variable-reference* (§12.3.3) to a moveable variable (§25.3) of an unmanaged type *T*, provided the type *T** is implicitly convertible to the pointer type given in the **fixed** statement. In this case, the initializer computes the address of the given variable, and the variable is guaranteed to remain at a fixed address for the duration of the **fixed** statement.
- An expression of an *array-type* with elements of an unmanaged type *T*, provided the type *T** is implicitly convertible to the pointer type given in the **fixed** statement. In this case, the initializer computes the address of the first element in the array, and the entire array is guaranteed to remain at a fixed address for the duration of the **fixed** statement. The behavior of the **fixed** statement is implementation-defined if the array expression is null or if the array has zero elements.
- An expression of type **string**, provided the type **char*** is implicitly convertible to the pointer type given in the **fixed** statement. In this case, the initializer computes the address of the first character in the string, and the entire string is guaranteed to remain at a fixed address for the duration of the **fixed** statement. The behavior of the **fixed** statement is implementation-defined if the string expression is null.

For each address computed by a *fixed-pointer-initializer* the **fixed** statement ensures that the variable referenced by the address is not subject to relocation or disposal by the garbage collector for the duration of the **fixed** statement. For example, if the address computed by a *fixed-pointer-initializer* references a field of an object or an element of an array instance, the **fixed** statement guarantees that the containing object instance is not relocated or disposed of during the lifetime of the statement.

It is the programmer's responsibility to ensure that pointers created by **fixed** statements do not survive beyond execution of those statements. For example, when pointers created by **fixed** statements are passed to external APIs, it is the programmer's responsibility to ensure that the APIs retain no memory of these pointers.

Fixed objects may cause fragmentation of the heap (because they can't be moved). For that reason, objects should be fixed only when absolutely necessary and then only for the shortest amount of time possible.[*Example*: The example

```

1      class Test
2      {
3          static int x;
4          int y;
5
6          unsafe static void F(int* p) {
7              *p = 1;
8          }
9
10         static void Main() {
11             Test t = new Test();
12             int[] a = new int[10];
13             unsafe {
14                 fixed (int* p = &x) F(p);
15                 fixed (int* p = &t.y) F(p);
16                 fixed (int* p = &a[0]) F(p);
17                 fixed (int* p = a) F(p);
18             }
19         }
20     }

```

demonstrates several uses of the `fixed` statement. The first statement fixes and obtains the address of a static field, the second statement fixes and obtains the address of an instance field, and the third statement fixes and obtains the address of an array element. In each case it would have been an error to use the regular `&` operator since the variables are all classified as moveable variables.

The third and fourth `fixed` statements in the example above produce identical results. In general, for an array instance `a`, specifying `&a[0]` in a `fixed` statement is the same as simply specifying `a`.

Here's another example of the `fixed` statement, this time using `string`:

```

26      class Test
27      {
28          static string name = "xx";
29
30          unsafe static void F(char* p) {
31              for (int i = 0; p[i] != '\0'; ++i)
32                  Console.WriteLine(p[i]);
33          }
34
35          static void Main() {
36              unsafe {
37                  fixed (char* p = name) F(p);
38                  fixed (char* p = "xx") F(p);
39              }
40          }
41      }
42

```

end example]

In an unsafe context array elements of single-dimensional arrays are stored in increasing index order, starting with index 0 and ending with index `Length - 1`. For multi-dimensional arrays, array elements are stored such that the indices of the rightmost dimension are increased first, then the next left dimension, and so on to the left.

Within a `fixed` statement that obtains a pointer `p` to an array instance `a`, the pointer values ranging from `p` to `p + a.Length - 1` represent addresses of the elements in the array. Likewise, the variables ranging from `p[0]` to `p[a.Length - 1]` represent the actual array elements. Given the way in which arrays are stored, we can treat an array of any dimension as though it were linear. [Example: For example.

```

1      using System;
2      class Test
3      {
4          static void Main() {
5              int[, ,] a = new int[2,3,4];
6              unsafe {
7                  fixed (int* p = a) {
8                      for (int i = 0; i < a.Length; ++i) // treat as linear
9                          p[i] = i;
10                 }
11             }
12
13             for (int i = 0; i < 2; ++i)
14                 for (int j = 0; j < 3; ++j) {
15                     for (int k = 0; k < 4; ++k)
16                         Console.WriteLine("[{0},{1},{2}] = {3,2} ", i, j, k,
17                         a[i,j,k]);
18                     Console.WriteLine();
19                 }
20             }
21         }

```

22 which produces the output:

```

23      [0,0,0] = 0 [0,0,1] = 1 [0,0,2] = 2 [0,0,3] = 3
24      [0,1,0] = 4 [0,1,1] = 5 [0,1,2] = 6 [0,1,3] = 7
25      [0,2,0] = 8 [0,2,1] = 9 [0,2,2] = 10 [0,2,3] = 11
26      [1,0,0] = 12 [1,0,1] = 13 [1,0,2] = 14 [1,0,3] = 15
27      [1,1,0] = 16 [1,1,1] = 17 [1,1,2] = 18 [1,1,3] = 19
28      [1,2,0] = 20 [1,2,1] = 21 [1,2,2] = 22 [1,2,3] = 23

```

29 *end example]*

30 *[Example:* In the example

```

31      class Test
32      {
33          unsafe static void Fill(int* p, int count, int value) {
34              for (; count != 0; count--) *p++ = value;
35          }
36
37          static void Main() {
38              int[] a = new int[100];
39              unsafe {
40                  fixed (int* p = a) Fill(p, 100, -1);
41              }
42          }

```

43 a *fixed* statement is used to fix an array so its address can be passed to a method that takes a pointer. *end*
44 *example]*

45 A *char** value produced by fixing a string instance always points to a null-terminated string. Within a *fixed*
46 statement that obtains a pointer *p* to a string instance *s*, the pointer values ranging from *p* to
47 *p + s.Length - 1* represent addresses of the characters in the string, and the pointer value
48 *p + s.Length* always points to a null character (the character with value *'\0'*).

49 Modifying objects of managed type through *fixed* pointers can result in undefined behavior. *[Note:* For
50 example, because strings are immutable, it is the programmer's responsibility to ensure that the characters
51 referenced by a pointer to a *fixed* string are not modified. *end note]*

52 *[Note:* The automatic null-termination of strings is particularly convenient when calling external APIs that
53 expect "C-style" strings. Note, however, that a string instance is permitted to contain null characters. If such
54 null characters are present, the string will appear truncated when treated as a null-terminated *char**. *end*
55 *note]*

25.7 Stack allocation

In an unsafe context, a local variable declaration (§15.5.1) may include a stack allocation initializer, which allocates memory from the call stack.

local-variable-initializer:

expression

array-initializer

stackalloc-initializer

stackalloc-initializer:

stackalloc *unmanaged-type* [*expression*]

The *unmanaged-type* indicates the type of the items that will be stored in the newly allocated location, and the *expression* indicates the number of these items. Taken together, these specify the required allocation size. Since the size of a stack allocation cannot be negative, it is a compile-time error to specify the number of items as a constant-expression that evaluates to a negative value.

A stack allocation initializer of the form **stackalloc** *T*[*E*] requires *T* to be an unmanaged type (§25.2) and *E* to be an expression of type **int**. The construct allocates *E* * **sizeof**(*T*) bytes from the call stack and returns a pointer, of type *T**, to the newly allocated block. If *E* is a negative value, then the behavior is undefined. If *E* is zero, then no allocation is made, and the pointer returned is implementation-defined. If there is not enough memory available to allocate a block of the given size, a **System.StackOverflowException** is thrown.

The content of the newly allocated memory is undefined.

Stack allocation initializers are not permitted in catch or finally blocks (§15.10).

[*Note:* There is no way to explicitly free memory allocated using **stackalloc**. *end note*] All stack-allocated memory blocks created during the execution of a function member are automatically discarded when that function member returns. [*Note:* This corresponds to the **alloca** function, an extension commonly found C and C++ implementations. *end note*]

[*Example:* In the example

```
using System;
class Test
{
    static string IntToString(int value) {
        int n = value >= 0 ? value : -value;
        unsafe {
            char* buffer = stackalloc char[16];
            char* p = buffer + 16;
            do {
                *--p = (char)(n % 10 + '0');
                n /= 10;
            } while (n != 0);
            if (value < 0) *--p = '-';
            return new string(p, 0, (int)(buffer + 16 - p));
        }
    }

    static void Main() {
        Console.WriteLine(IntToString(12345));
        Console.WriteLine(IntToString(-999));
    }
}
```

a **stackalloc** initializer is used in the **IntToString** method to allocate a buffer of 16 characters on the stack. The buffer is automatically discarded when the method returns. *end example*]

25.8 Dynamic memory allocation

Except for the **stackalloc** operator, C# provides no predefined constructs for managing non-garbage collected memory. Such services are typically provided by supporting class libraries or imported directly

from the underlying operating system. [Example: For example, the `Memory` class below illustrates how the heap functions of an underlying operating system might be accessed from C#:

```

1  using System;
2  using System.Runtime.InteropServices;
3
4  public unsafe class Memory
5  {
6      // Handle for the process heap. This handle is used in all calls to
7      the
8      // HeapXXX APIs in the methods below.
9
10     static int ph = GetProcessHeap();
11     // Private instance constructor to prevent instantiation.
12     private Memory() {}
13     // Allocates a memory block of the given size. The allocated memory is
14     // automatically initialized to zero.
15     public static void* Alloc(int size) {
16         void* result = HeapAlloc(ph, HEAP_ZERO_MEMORY, size);
17         if (result == null) throw new OutOfMemoryException();
18         return result;
19     }
20     // Copies count bytes from src to dst. The source and destination
21     // blocks are permitted to overlap.
22     public static void Copy(void* src, void* dst, int count) {
23         byte* ps = (byte*)src;
24         byte* pd = (byte*)dst;
25         if (ps > pd) {
26             for (; count != 0; count--) *pd++ = *ps++;
27         }
28         else if (ps < pd) {
29             for (ps += count, pd += count; count != 0; count--) *--pd = *--
30 ps;
31         }
32     }
33     // Frees a memory block.
34     public static void Free(void* block) {
35         if (!HeapFree(ph, 0, block)) throw new InvalidOperationException();
36     }
37     // Re-allocates a memory block. If the reallocation request is for a
38     // larger size, the additional region of memory is automatically
39     // initialized to zero.
40     public static void* ReAlloc(void* block, int size) {
41         void* result = HeapReAlloc(ph, HEAP_ZERO_MEMORY, block, size);
42         if (result == null) throw new OutOfMemoryException();
43         return result;
44     }
45     // Returns the size of a memory block.
46     public static int SizeOf(void* block) {
47         int result = HeapSize(ph, 0, block);
48         if (result == -1) throw new InvalidOperationException();
49         return result;
50     }
51     // Heap API flags
52     const int HEAP_ZERO_MEMORY = 0x00000008;
53     // Heap API functions
54     [DllImport("kernel32")]
55     static extern int GetProcessHeap();
56     [DllImport("kernel32")]
57     static extern void* HeapAlloc(int hHeap, int flags, int size);

```

```

1      [DllImport("kernel32")]
2      static extern bool HeapFree(int hHeap, int flags, void* block);
3      [DllImport("kernel32")]
4      static extern void* HeapReAlloc(int hHeap, int flags,
5          void* block, int size);
6      [DllImport("kernel32")]
7      static extern int HeapSize(int hHeap, int flags, void* block);
8  }

```

9 An example that uses the `Memory` class is given below:

```

10     class Test
11     {
12         static void Main() {
13             unsafe {
14                 byte* buffer = (byte*)Memory.Alloc(256);
15                 for (int i = 0; i < 256; i++) buffer[i] = (byte)i;
16                 byte[] array = new byte[256];
17                 fixed (byte* p = array) Memory.Copy(buffer, p, 256);
18                 Memory.Free(buffer);
19                 for (int i = 0; i < 256; i++) Console.WriteLine(array[i]);
20             }
21         }
22     }

```

23 The example allocates 256 bytes of memory through `Memory.Alloc` and initializes the memory block with
24 values increasing from 0 to 255. It then allocates a 256-element byte array and uses `Memory.Copy` to copy
25 the contents of the memory block into the byte array. Finally, the memory block is freed using
26 `Memory.Free` and the contents of the byte array are output on the console. *end example*

27 **End of conditionally normative text.**

A. Grammar

This clause is informative.

This appendix contains summaries of the lexical and syntactic grammars found in the main document, and of the grammar extensions for unsafe code. Grammar productions appear here in the same order that they appear in the main document.

A.1 Lexical grammar

```

input::
    input-sectionopt

input-section::
    input-section-part
    input-section input-section-part

input-section-part::
    input-elementsopt new-line
    pp-directive

input-elements::
    input-element
    input-elements input-element

input-element::
    whitespace
    comment
    token

```

A.1.1 Line terminators

```

new-line::
    Carriage return character (U+000D)
    Line feed character (U+000A)
    Carriage return character (U+000D) followed by line feed character (U+000A)
    Line separator character (U+2028)
    Paragraph separator character (U+2029)

```

A.1.2 White space

```

whitespace::
    Any character with Unicode class Zs
    Horizontal tab character (U+0009)
    Vertical tab character (U+000B)
    Form feed character (U+000C)

```

A.1.3 Comments

```

comment::
    single-line-comment
    delimited-comment

single-line-comment::
    // input-charactersopt

```

input-characters::
input-character
input-characters input-character
input-character::
 Any Unicode character except a *new-line-character*
new-line-character::
 Carriage return character (U+000D)
 Line feed character (U+000A)
 Line separator character (U+2028)
 Paragraph separator character (U+2029)
delimited-comment::
/ delimited-comment-characters_{opt} */*
delimited-comment-characters::
delimited-comment-character
delimited-comment-characters delimited-comment-character
delimited-comment-character::
not-asterisk
** not-slash*
not-asterisk::
 Any Unicode character except *
not-slash::
 Any Unicode character except /

A.1.4 Tokens

token::
identifier
keyword
integer-literal
real-literal
character-literal
string-literal
operator-or-punctuator

A.1.5 Unicode character escape sequences

unicode-character-escape-sequence::
\u hex-digit hex-digit hex-digit hex-digit
\U hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit hex-digit

A.1.6 Identifiers

identifier::
available-identifier
@ identifier-or-keyword
available-identifier::
 An *identifier-or-keyword* that is not a *keyword*
identifier-or-keyword::
identifier-start-character identifier-part-characters_{opt}
identifier-start-character::
letter-character
_ (the underscore character)

identifier-part-characters::
identifier-part-character
identifier-part-characters identifier-part-character

identifier-part-character::
letter-character
decimal-digit-character
connecting-character
combining-character
formatting-character

letter-character::
 A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nl
 A *unicode-character-escape-sequence* representing a character of classes Lu, Ll, Lt, Lm, Lo, or Nl

combining-character::
 A Unicode character of classes Mn or Mc
 A *unicode-character-escape-sequence* representing a character of classes Mn or Mc

decimal-digit-character::
 A Unicode character of the class Nd
 A *unicode-character-escape-sequence* representing a character of the class Nd

connecting-character::
 A Unicode character of the class Pc
 A *unicode-character-escape-sequence* representing a character of the class Pc

formatting-character::
 A Unicode character of the class Cf
 A *unicode-character-escape-sequence* representing a character of the class Cf

A.1.7 Keywords

keyword:: one of

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
while				

A.1.8 Literals*literal*::*boolean-literal**integer-literal**real-literal**character-literal**string-literal**null-literal**boolean-literal*::**true****false***integer-literal*::*decimal-integer-literal**hexadecimal-integer-literal**decimal-integer-literal*::*decimal-digits* *integer-type-suffix*_{opt}*decimal-digits*::*decimal-digit**decimal-digits* *decimal-digit**decimal-digit*:: one of**0 1 2 3 4 5 6 7 8 9***integer-type-suffix*:: one of**U u L l UL Ul uL ul LU Lu lU lu***hexadecimal-integer-literal*::**0x** *hex-digits* *integer-type-suffix*_{opt}**0X** *hex-digits* *integer-type-suffix*_{opt}*hex-digits*::*hex-digit**hex-digits* *hex-digit**hex-digit*:: one of**0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f***real-literal*::*decimal-digits* **.** *decimal-digits* *exponent-part*_{opt} *real-type-suffix*_{opt}**.** *decimal-digits* *exponent-part*_{opt} *real-type-suffix*_{opt}*decimal-digits* *exponent-part* *real-type-suffix*_{opt}*decimal-digits* *real-type-suffix**exponent-part*::**e** *sign*_{opt} *decimal-digits***E** *sign*_{opt} *decimal-digits**sign*:: one of**+ -***real-type-suffix*:: one of**F f D d M m***character-literal*::**'** *character* **'**

```

1  character::
2      single-character
3      simple-escape-sequence
4      hexadecimal-escape-sequence
5      unicode-character-escape-sequence
6
7  single-character::
8      Any character except ' (U+0027), \ (U+005C), and new-line-character
9
10 simple-escape-sequence:: one of
11     \' \" \\ \0 \a \b \f \n \r \t \v
12
13 hexadecimal-escape-sequence::
14     \x hex-digit hex-digitopt hex-digitopt hex-digitopt
15
16 string-literal::
17     regular-string-literal
18     verbatim-string-literal
19
20 regular-string-literal::
21     " regular-string-literal-charactersopt "
22
23 regular-string-literal-characters::
24     regular-string-literal-character
25     regular-string-literal-characters regular-string-literal-character
26
27 regular-string-literal-character::
28     single-regular-string-literal-character
29     simple-escape-sequence
30     hexadecimal-escape-sequence
31     unicode-character-escape-sequence
32
33 single-regular-string-literal-character::
34     Any character except " (U+0022), \ (U+005C), and new-line-character
35
36 verbatim-string-literal::
37     @" verbatim-string-literal-charactersopt "
38
39 verbatim-string-literal-characters::
40     verbatim-string-literal-character
41     verbatim-string-literal-characters verbatim-string-literal-character
42
43 verbatim-string-literal-character::
44     single-verbatim-string-literal-character
45     quote-escape-sequence
46
47 single-verbatim-string-literal-character::
48     any character except "
49
50 quote-escape-sequence::
51     ""
52
53 null-literal::
54     null

```

A.1.9 Operators and punctuators

```

operator-or-punctuator:: one of
{      }      [      ]      (      )      .      ,      :      ;
+      -      *      /      %      &      |      ^      !      ~
=      <      >      ?      ++     --     &&     ||     <<     >>
==     !=     <=     >=     +=     -=     *=     /=     %=     &=
|=     ^=     <<=    >>=    ->

```

A.1.10 Pre-processing directives

```

1  pp-directive::
2      pp-declaration
3      pp-conditional
4      pp-line
5      pp-diagnostic
6      pp-region
7
8  pp-new-line::
9      whitespaceopt single-line-commentopt new-line
10
11 conditional-symbol::
12     Any identifier-or-keyword except true or false
13
14 pp-expression::
15     whitespaceopt pp-or-expression whitespaceopt
16
17 pp-or-expression::
18     pp-and-expression
19     pp-or-expression whitespaceopt || whitespaceopt pp-and-expression
20
21 pp-and-expression::
22     pp-equality-expression
23     pp-and-expression whitespaceopt && whitespaceopt pp-equality-expression
24
25 pp-equality-expression::
26     pp-unary-expression
27     pp-equality-expression whitespaceopt == whitespaceopt pp-unary-expression
28     pp-equality-expression whitespaceopt != whitespaceopt pp-unary-expression
29
30 pp-unary-expression::
31     pp-primary-expression
32     ! whitespaceopt pp-unary-expression
33
34 pp-primary-expression::
35     true
36     false
37     conditional-symbol
38     ( whitespaceopt pp-expression whitespaceopt )
39
40
41 pp-declaration::
42     whitespaceopt # whitespaceopt define whitespace conditional-symbol pp-new-line
43     whitespaceopt # whitespaceopt undef whitespace conditional-symbol pp-new-line
44
45 pp-conditional::
46     pp-if-section pp-elif-sectionsopt pp-else-sectionopt pp-endif
47
48 pp-if-section::
49     whitespaceopt # whitespaceopt if whitespace pp-expression pp-new-line conditional-
50     sectionopt
51
52 pp-elif-sections::
53     pp-elif-section
54     pp-elif-sections pp-elif-section
55
56 pp-elif-section::
57     whitespaceopt # whitespaceopt elif whitespace pp-expression pp-new-line conditional-
58     sectionopt
59
60 pp-else-section::
61     whitespaceopt # whitespaceopt else pp-new-line conditional-sectionopt

```

```

1  pp-endif::
2      whitespaceopt # whitespaceopt endif pp-new-line
3
4  conditional-section::
5      input-section
6      skipped-section
7
8  skipped-section::
9      skipped-section-part
10     skipped-section skipped-section-part
11
12  skipped-section-part::
13     skipped-charactersopt new-line
14     pp-directive
15
16  skipped-characters::
17     whitespaceopt not-number-sign input-charactersopt
18
19  not-number-sign::
20     Any input-character except #
21
22  pp-line::
23     whitespaceopt # whitespaceopt line whitespaceopt line-indicator pp-new-line
24
25  line-indicator::
26     decimal-digits whitespace file-name
27     decimal-digits
28     default
29
30  file-name::
31     " file-name-characters "
32
33  file-name-characters::
34     file-name-character
35     file-name-characters file-name-character
36
37  file-name-character::
38     Any character except " (U+0022), and new-line
39
40  pp-diagnostic::
41     whitespaceopt # whitespaceopt error whitespaceopt pp-message
42     whitespaceopt # whitespaceopt warning whitespaceopt pp-message
43
44  pp-message::
45     input-charactersopt new-line
46
47  pp-region::
48     pp-start-region conditional-sectionopt pp-end-region
49
50  pp-start-region::
51     whitespaceopt # whitespaceopt region whitespaceopt pp-message
52
53  pp-end-region::
54     whitespaceopt # whitespaceopt endregion whitespaceopt pp-message

```

40 A.2 Syntactic grammar

41 A.2.1 Basic concepts

```

42     namespace-name:
43         namespace-or-type-name
44
45     type-name:
46         namespace-or-type-name

```

1 *namespace-or-type-name*:
 2 *identifier*
 3 *namespace-or-type-name* . *identifier*

4 **A.2.2 Types**

5 *type*:
 6 *value-type*
 7 *reference-type*

8 *value-type*:
 9 *struct-type*
 10 *enum-type*

11 *struct-type*:
 12 *type-name*
 13 *simple-type*

14 *simple-type*:
 15 *numeric-type*
 16 *bool*

17 *numeric-type*:
 18 *integral-type*
 19 *floating-point-type*
 20 *decimal*

21 *integral-type*:
 22 *sbyte*
 23 *byte*
 24 *short*
 25 *ushort*
 26 *int*
 27 *uint*
 28 *long*
 29 *ulong*
 30 *char*

31 *floating-point-type*:
 32 *float*
 33 *double*

34 *enum-type*:
 35 *type-name*

36 *reference-type*:
 37 *class-type*
 38 *interface-type*
 39 *array-type*
 40 *delegate-type*

41 *class-type*:
 42 *type-name*
 43 *object*
 44 *string*

45 *interface-type*:
 46 *type-name*

47 *array-type*:
 48 *non-array-type* *rank-specifiers*

```

1      non-array-type:
2          type
3
4      rank-specifiers:
5          rank-specifier
6          rank-specifiers rank-specifier
7
8      rank-specifier:
9          [ dim-separatorsopt ]
10
11     dim-separators:
12         ,
13         dim-separators ,
14
15     delegate-type:
16         type-name

```

13 A.2.3 Variables

```

14     variable-reference:
15         expression

```

16 A.2.4 Expressions

```

17     argument-list:
18         argument
19         argument-list , argument
20
21     argument:
22         expression
23         ref variable-reference
24         out variable-reference
25
26     primary-expression:
27         array-creation-expression
28         primary-no-array-creation-expression
29
30     primary-no-array-creation-expression:
31         literal
32         simple-name
33         parenthesized-expression
34         member-access
35         invocation-expression
36         element-access
37         this-access
38         base-access
39         post-increment-expression
40         post-decrement-expression
41         object-creation-expression
42         delegate-creation-expression
43
44         typeof-expression
45         sizeof-expression
46         checked-expression
47         unchecked-expression
48
49     simple-name:
50         identifier
51
52     parenthesized-expression:
53         ( expression )

```

```

1      member-access:
2          primary-expression . identifier
3          predefined-type . identifier
4
5      predefined-type: one of
6          bool      byte      char      decimal  double  float  int      long
7          object    sbyte     short     string   uint    ulong   ushort
8
9      invocation-expression:
10         primary-expression ( argument-listopt )
11
12     element-access:
13         primary-no-array-creation-expression [ expression-list ]
14
15     expression-list:
16         expression
17         expression-list , expression
18
19     this-access:
20         this
21
22     base-access:
23         base . identifier
24         base [ expression-list ]
25
26     post-increment-expression:
27         primary-expression ++
28
29     post-decrement-expression:
30         primary-expression --
31
32     object-creation-expression:
33         new type ( argument-listopt )
34
35     array-creation-expression:
36         new non-array-type [ expression-list ] rank-specifiersopt array-initializeropt
37         new array-type array-initializer
38
39     delegate-creation-expression:
40         new delegate-type ( expression )
41
42     typeof-expression:
43         typeof ( type )
44         typeof ( void )
45
46     checked-expression:
47         checked ( expression )
48
49     unchecked-expression:
50         unchecked ( expression )
51
52     unary-expression:
53         primary-expression
54         + unary-expression
55         - unary-expression
56         ! unary-expression
57         ~ unary-expression
58         * unary-expression
59         pre-increment-expression
60         pre-decrement-expression
61         cast-expression
62
63     pre-increment-expression:
64         ++ unary-expression

```



```

1  pre-decrement-expression:
2      -- unary-expression
3
4  cast-expression:
5      ( type ) unary-expression
6
7  multiplicative-expression:
8      unary-expression
9      multiplicative-expression * unary-expression
10     multiplicative-expression / unary-expression
11     multiplicative-expression % unary-expression
12
13 additive-expression:
14     multiplicative-expression
15     additive-expression + multiplicative-expression
16     additive-expression - multiplicative-expression
17
18 shift-expression:
19     additive-expression
20     shift-expression << additive-expression
21     shift-expression >> additive-expression
22
23 relational-expression:
24     shift-expression
25     relational-expression < shift-expression
26     relational-expression > shift-expression
27     relational-expression <= shift-expression
28     relational-expression >= shift-expression
29     relational-expression is type
30     relational-expression as type
31
32 equality-expression:
33     relational-expression
34     equality-expression == relational-expression
35     equality-expression != relational-expression
36
37 and-expression:
38     equality-expression
39     and-expression & equality-expression
40
41 exclusive-or-expression:
42     and-expression
43     exclusive-or-expression ^ and-expression
44
45 inclusive-or-expression:
46     exclusive-or-expression
47     inclusive-or-expression | exclusive-or-expression
48
49 conditional-and-expression:
50     inclusive-or-expression
51     conditional-and-expression && inclusive-or-expression
52
53 conditional-or-expression:
54     conditional-and-expression
55     conditional-or-expression || conditional-and-expression
56
57 conditional-expression:
58     conditional-or-expression
59     conditional-or-expression ? expression : expression
60
61 assignment:
62     unary-expression assignment-operator expression

```

1 *assignment-operator*: one of
 2 = += -= *= /= %= &= |= ^= <<= >>=
 3 *expression*:
 4 *conditional-expression*
 5 *assignment*
 6 *constant-expression*:
 7 *expression*
 8 *boolean-expression*:
 9 *expression*

10 A.2.5 Statements

11 *statement*:
 12 *labeled-statement*
 13 *declaration-statement*
 14 *embedded-statement*
 15 *embedded-statement*:
 16 *block*
 17 *empty-statement*
 18 *expression-statement*
 19 *selection-statement*
 20 *iteration-statement*
 21 *jump-statement*
 22 *try-statement*
 23 *checked-statement*
 24 *unchecked-statement*
 25 *lock-statement*
 26 *using-statement*
 27 *block*:
 28 { *statement-list*_{opt} }
 29 *statement-list*:
 30 *statement*
 31 *statement-list* *statement*
 32 *empty-statement*:
 33 ;
 34 *labeled-statement*:
 35 *identifier* : *statement*
 36 *declaration-statement*:
 37 *local-variable-declaration* ;
 38 *local-constant-declaration* ;
 39 *local-variable-declaration*:
 40 *type* *local-variable-declarators*
 41 *local-variable-declarators*:
 42 *local-variable-declarator*
 43 *local-variable-declarators* , *local-variable-declarator*
 44 *local-variable-declarator*:
 45 *identifier*
 46 *identifier* = *local-variable-initializer*

```

1  local-variable-initializer:
2      expression
3      array-initializer
4
5  local-constant-declaration:
6      const type constant-declarators
7
8  constant-declarators:
9      constant-declarator
10     constant-declarators , constant-declarator
11
12 constant-declarator:
13     identifier = constant-expression
14
15 expression-statement:
16     statement-expression ;
17
18 statement-expression:
19     invocation-expression
20     object-creation-expression
21     assignment
22     post-increment-expression
23     post-decrement-expression
24     pre-increment-expression
25     pre-decrement-expression
26
27 selection-statement:
28     if-statement
29     switch-statement
30
31 if-statement:
32     if ( boolean-expression ) embedded-statement
33     if ( boolean-expression ) embedded-statement else embedded-statement
34
35 boolean-expression:
36     expression
37
38 switch-statement:
39     switch ( expression ) switch-block
40
41 switch-block:
42     { switch-sectionsopt }
43
44 switch-sections:
45     switch-section
46     switch-sections switch-section
47
48 switch-section:
49     switch-labels statement-list
50
51 switch-labels:
52     switch-label
53     switch-labels switch-label
54
55 switch-label:
56     case constant-expression :
57     default :
58
59 iteration-statement:
60     while-statement
61     do-statement
62     for-statement
63     foreach-statement

```

```

1      while-statement:
2          while ( boolean-expression ) embedded-statement
3
4      do-statement:
5          do embedded-statement while ( boolean-expression ) ;
6
7      for-statement:
8          for ( for-initializeropt ; for-conditionopt ; for-iteratoropt ) embedded-statement
9
10     for-initializer:
11         local-variable-declaration
12         statement-expression-list
13
14     for-condition:
15         boolean-expression
16
17     for-iterator:
18         statement-expression-list
19
20     statement-expression-list:
21         statement-expression
22         statement-expression-list , statement-expression
23
24     foreach-statement:
25         foreach ( type identifier in expression ) embedded-statement
26
27     jump-statement:
28         break-statement
29         continue-statement
30         goto-statement
31         return-statement
32         throw-statement
33
34     break-statement:
35         break ;
36
37     continue-statement:
38         continue ;
39
40     goto-statement:
41         goto identifier ;
42         goto case constant-expression ;
43         goto default ;
44
45     return-statement:
46         return expressionopt ;
47
48     throw-statement:
49         throw expressionopt ;
50
51     try-statement:
52         try block catch-clauses
53         try block finally-clause
54         try block catch-clauses finally-clause
55
56     catch-clauses:
57         specific-catch-clauses general-catch-clauseopt
58         specific-catch-clausesopt general-catch-clause
59
60     specific-catch-clauses:
61         specific-catch-clause
62         specific-catch-clauses specific-catch-clause

```

```

1  specific-catch-clause:
2      catch ( class-type identifieropt ) block
3
4  general-catch-clause:
5      catch block
6
7  finally-clause:
8      finally block
9
10 checked-statement:
11     checked block
12
13 unchecked-statement:
14     unchecked block
15
16 lock-statement:
17     lock ( expression ) embedded-statement
18
19 using-statement:
20     using ( resource-acquisition ) embedded-statement
21
22 resource-acquisition:
23     local-variable-declaration
24     expression
25
26 compilation-unit:
27     using-directivesopt global-attributesopt namespace-member-declarationsopt
28
29 namespace-declaration:
30     namespace qualified-identifier namespace-body ;opt
31
32 qualified-identifier:
33     identifier
34     qualified-identifier . identifier
35
36 namespace-body:
37     { using-directivesopt namespace-member-declarationsopt }
38
39 using-directives:
40     using-directive
41     using-directives using-directive
42
43 using-directive:
44     using-alias-directive
45     using-namespace-directive
46
47 using-alias-directive:
48     using identifier = namespace-or-type-name ;
49
50 using-namespace-directive:
51     using namespace-name ;
52
53 namespace-member-declarations:
54     namespace-member-declaration
55     namespace-member-declarations namespace-member-declaration
56
57 namespace-member-declaration:
58     namespace-declaration
59     type-declaration

```

```

1      type-declaration:
2          class-declaration
3          struct-declaration
4          interface-declaration
5          enum-declaration
6          delegate-declaration

```

7 A.2.6 Classes

```

8      class-declaration:
9          attributesopt class-modifiersopt class identifier class-baseopt class-body ;opt
10     class-modifiers:
11         class-modifier
12         class-modifiers class-modifier
13     class-modifier:
14         new
15         public
16         protected
17         internal
18         private
19         abstract
20         sealed
21     class-base:
22         : class-type
23         : interface-type-list
24         : class-type , interface-type-list
25     interface-type-list:
26         interface-type
27         interface-type-list , interface-type
28     class-body:
29         { class-member-declarationsopt }
30     class-member-declarations:
31         class-member-declaration
32         class-member-declarations class-member-declaration
33     class-member-declaration:
34         constant-declaration
35         field-declaration
36         method-declaration
37         property-declaration
38         event-declaration
39         indexer-declaration
40         operator-declaration
41         constructor-declaration
42         destructor-declaration
43         static-constructor-declaration
44         type-declaration
45     constant-declaration:
46         attributesopt constant-modifiersopt const type constant-declarators ;
47     constant-modifiers:
48         constant-modifier
49         constant-modifiers constant-modifier

```

```

1      constant-modifier:
2          new
3          public
4          protected
5          internal
6          private
7
8      constant-declarators:
9          constant-declarator
10         constant-declarators , constant-declarator
11
12     constant-declarator:
13         identifier = constant-expression
14
15     field-declaration:
16         attributesopt field-modifiersopt type variable-declarators ;
17
18     field-modifiers:
19         field-modifier
20         field-modifiers field-modifier
21
22     field-modifier:
23         new
24         public
25         protected
26         internal
27         private
28         static
29         readonly
30         volatile
31
32     variable-declarators:
33         variable-declarator
34         variable-declarators , variable-declarator
35
36     variable-declarator:
37         identifier
38         identifier = variable-initializer
39
40     variable-initializer:
41         expression
42         array-initializer
43
44     method-declaration:
45         method-header method-body
46
47     method-header:
48         attributesopt method-modifiersopt return-type member-name ( formal-parameter-listopt )
49
50     method-modifiers:
51         method-modifier
52         method-modifiers method-modifier

```

```

1      method-modifier:
2          new
3          public
4          protected
5          internal
6          private
7          static
8          virtual
9          sealed
10         override
11         abstract
12         extern
13
14     return-type:
15         type
16         void
17
18     member-name:
19         identifier
20         interface-type . identifier
21
22     method-body:
23         block
24         ;
25
26     formal-parameter-list:
27         fixed-parameters
28         fixed-parameters , parameter-array
29         parameter-array
30
31     fixed-parameters:
32         fixed-parameter
33         fixed-parameters , fixed-parameter
34
35     fixed-parameter:
36         attributesopt parameter-modifieropt type identifier
37
38     parameter-modifier:
39         ref
40         out
41
42     parameter-array:
43         attributesopt params array-type identifier
44
45     property-declaration:
46         attributesopt property-modifiersopt type member-name { accessor-declarations }
47
48     property-modifiers:
49         property-modifier
50         property-modifiers property-modifier

```



```

1  property-modifier:
2      new
3      public
4      protected
5      internal
6      private
7      static
8      virtual
9      sealed
10     override
11     abstract
12     extern
13
14 member-name:
15     identifier
16     interface-type . identifier
17
18 accessor-declarations:
19     get-accessor-declaration set-accessor-declarationopt
20     set-accessor-declaration get-accessor-declarationopt
21
22 get-accessor-declaration:
23     attributesopt get accessor-body
24
25 set-accessor-declaration:
26     attributesopt set accessor-body
27
28 accessor-body:
29     block
30     ;
31
32 event-declaration:
33     attributesopt event-modifiersopt event type variable-declarators ;
34     attributesopt event-modifiersopt event type member-name { event-accessor-declarations
35     }
36
37 event-modifiers:
38     event-modifier
39     event-modifiers event-modifier
40
41 event-modifier:
42     new
43     public
44     protected
45     internal
46     private
47     static
48     virtual
49     sealed
50     override
51     abstract
52     extern
53
54 event-accessor-declarations:
55     add-accessor-declaration remove-accessor-declaration
56     remove-accessor-declaration add-accessor-declaration
57
58 add-accessor-declaration:
59     attributesopt add block

```

```

1      remove-accessor-declaration:
2          attributesopt remove block
3
4      indexer-declaration:
5          attributesopt indexer-modifiersopt indexer-declarator { accessor-declarations }
6
7      indexer-modifiers:
8          indexer-modifier
9          indexer-modifiers indexer-modifier
10
11      indexer-modifier:
12          new
13          public
14          protected
15          internal
16          private
17          virtual
18          sealed
19          override
20          abstract
21          extern
22
23      indexer-declarator:
24          type this [ formal-parameter-list ]
25          type interface-type . this [ formal-parameter-list ]
26
27      operator-declaration:
28          attributesopt operator-modifiers operator-declarator operator-body
29
30      operator-modifiers:
31          operator-modifier
32          operator-modifiers operator-modifier
33
34      operator-modifier:
35          public
36          static
37          extern
38
39      operator-declarator:
40          unary-operator-declarator
41          binary-operator-declarator
42          conversion-operator-declarator
43
44      unary-operator-declarator:
45          type operator overloadable-unary-operator ( type identifier )
46
47      overloadable-unary-operator: one of
48          + - ! ~ ++ -- true false
49
50      binary-operator-declarator:
51          type operator overloadable-binary-operator ( type identifier , type identifier )
52
53      overloadable-binary-operator: one of
54          + - * / % & | ^ << >> == != > < >= <=
55
56      conversion-operator-declarator:
57          implicit operator type ( type identifier )
58          explicit operator type ( type identifier )
59
60      operator-body:
61          block
62          ;

```

```

1  constructor-declaration:
2      attributesopt constructor-modifiersopt constructor-declarator constructor-body
3
4  constructor-modifiers:
5      constructor-modifier
6      constructor-modifiers constructor-modifier
7
8  constructor-modifier:
9      public
10     protected
11     internal
12     private
13     extern
14
15  constructor-declarator:
16     identifier ( formal-parameter-listopt ) constructor-initializeropt
17
18  constructor-initializer:
19     : base ( argument-listopt )
20     : this ( argument-listopt )
21
22  constructor-body:
23     block
24     ;
25
26  static-constructor-declaration:
27     attributesopt static-constructor-modifiers identifier ( ) static-constructor-body
28
29  static-constructor-modifiers:
30     externopt static
31     static externopt
32
33  static-constructor-body:
34     block
35     ;
36
37  destructor-declaration:
38     attributesopt externopt ~ identifier ( ) destructor-body
39
40  destructor-body:
41     block
42     ;

```

A.2.7 Structs

```

34  struct-declaration:
35     attributesopt struct-modifiersopt struct identifier struct-interfacesopt struct-body ;opt
36
37  struct-modifiers:
38     struct-modifier
39     struct-modifiers struct-modifier
40
41  struct-modifier:
42     new
43     public
44     protected
45     internal
46     private
47
48  struct-interfaces:
49     : interface-type-list

```

```

1      struct-body:
2          { struct-member-declarationsopt }
3
4      struct-member-declarations:
5          struct-member-declaration
6          struct-member-declarations struct-member-declaration
7
8      struct-member-declaration:
9          constant-declaration
10         field-declaration
11         method-declaration
12         property-declaration
13         event-declaration
14         indexer-declaration
15         operator-declaration
16         constructor-declaration
17         static-constructor-declaration
18         type-declaration

```

17 A.2.8 Arrays

```

18     array-type:
19         non-array-type rank-specifiers
20
21     non-array-type:
22         type
23
24     rank-specifiers:
25         rank-specifier
26         rank-specifiers rank-specifier
27
28     rank-specifier:
29         [ dim-separatorsopt ]
30
31     dim-separators:
32         ,
33         dim-separators ,
34
35     array-initializer:
36         { variable-initializer-listopt }
37         { variable-initializer-list , }
38
39     variable-initializer-list:
40         variable-initializer
41         variable-initializer-list , variable-initializer
42
43     variable-initializer:
44         expression
45         array-initializer

```

39 A.2.9 Interfaces

```

40     interface-declaration:
41         attributesopt interface-modifiersopt interface identifier interface-baseopt interface-body
42         ;opt
43
44     interface-modifiers:
45         interface-modifier
46         interface-modifiers interface-modifier

```

```

1  interface-modifier:
2      new
3      public
4      protected
5      internal
6      private
7
8  interface-base:
9      : interface-type-list
10
11 interface-body:
12     { interface-member-declarationsopt }
13
14 interface-member-declarations:
15     interface-member-declaration
16     interface-member-declarations interface-member-declaration
17
18 interface-member-declaration:
19     interface-method-declaration
20     interface-property-declaration
21     interface-event-declaration
22     interface-indexer-declaration
23
24 interface-method-declaration:
25     attributesopt newopt return-type identifier ( formal-parameter-listopt ) ;
26
27 interface-property-declaration:
28     attributesopt newopt type identifier { interface-accessors }
29
30 interface-accessors:
31     attributesopt get ;
32     attributesopt set ;
33     attributesopt get ; attributesopt set ;
34     attributesopt set ; attributesopt get ;
35
36 interface-event-declaration:
37     attributesopt newopt event type identifier ;
38
39 interface-indexer-declaration:
40     attributesopt newopt type this [ formal-parameter-list ] { interface-accessors }

```

A.2.10 Enums

```

33 enum-declaration:
34     attributesopt enum-modifiersopt enum identifier enum-baseopt enum-body ;opt
35
36 enum-base:
37     : integral-type
38
39 enum-body:
40     { enum-member-declarationsopt }
41     { enum-member-declarations , }
42
43 enum-modifiers:
44     enum-modifier
45     enum-modifiers enum-modifier
46
47 enum-modifier:
48     new
49     public
50     protected
51     internal
52     private

```

```

1      enum-member-declarations:
2          enum-member-declaration
3          enum-member-declarations , enum-member-declaration
4
5      enum-member-declaration:
6          attributesopt identifier
7          attributesopt identifier = constant-expression

```

A.2.11 Delegates

```

8      delegate-declaration:
9          attributesopt delegate-modifiersopt delegate type identifier ( formal-parameter-listopt )
10         ;
11
12      delegate-modifiers:
13          delegate-modifier
14          delegate-modifiers delegate-modifier
15
16      delegate-modifier:
17          new
18          public
19          protected
20          internal
21          private

```

A.2.12 Attributes

```

22      global-attributes:
23          global-attribute-sections
24
25      global-attribute-sections:
26          global-attribute-section
27          global-attribute-sections global-attribute-section
28
29      global-attribute-section:
30          [ global-attribute-target-specifier attribute-list ]
31          [ global-attribute-target-specifier attribute-list ,]
32
33      global-attribute-target-specifier:
34          global-attribute-target :
35
36      global-attribute-target:
37          assembly
38
39      attributes:
40          attribute-sections
41
42      attribute-sections:
43          attribute-section
44          attribute-sections attribute-section
45
46      attribute-section:
47          [ attribute-target-specifieropt attribute-list ]
48          [ attribute-target-specifieropt attribute-list ,]
49
50      attribute-target-specifier:
51          attribute-target :

```

```

1      attribute-target:
2          field
3          event
4          method
5          module
6          param
7          property
8          return
9          type
10     attribute-list:
11         attribute
12         attribute-list , attribute
13     attribute:
14         attribute-name attribute-argumentsopt
15     attribute-name:
16         type-name
17     attribute-arguments:
18         ( positional-argument-listopt )
19         ( positional-argument-list , named-argument-list )
20         ( named-argument-list )
21     positional-argument-list:
22         positional-argument
23         positional-argument-list , positional-argument
24     positional-argument:
25         attribute-argument-expression
26     named-argument-list:
27         named-argument
28         named-argument-list , named-argument
29     named-argument:
30         identifier = attribute-argument-expression
31     attribute-argument-expression:
32         expression

```

A.3 Grammar extensions for unsafe code

```

33     embedded-statement:
34         ...
35         unsafe-statement
36     unsafe-statement:
37         unsafe block
38     type:
39         value-type
40         reference-type
41         pointer-type
42     pointer-type:
43         unmanaged-type *
44         void *
45     unmanaged-type:
46         type
47

```

```

1      primary-no-array-creation-expression:
2          ...
3          pointer-member-access
4          pointer-element-access
5          sizeof-expression
6
7      unary-expression:
8          ...
9          pointer-indirection-expression
10         addressof-expression
11
12     pointer-indirection-expression:
13         * unary-expression
14
15     pointer-member-access:
16         primary-expression -> identifier
17
18     pointer-element-access:
19         primary-no-array-creation-expression [ expression ]
20
21     addressof-expression:
22         & unary-expression
23
24     sizeof-expression:
25         sizeof ( unmanaged-type )
26
27     embedded-statement:
28         ...
29         fixed-statement
30
31     fixed-statement:
32         fixed ( pointer-type fixed-pointer-declarators ) embedded-statement
33
34     fixed-pointer-declarators:
35         fixed-pointer-declarator
36         fixed-pointer-declarators , fixed-pointer-declarator
37
38     fixed-pointer-declarator:
39         identifier = fixed-pointer-initializer
40
41     fixed-pointer-initializer:
42         & variable-reference
43         expression
44
45     variable-initializer:
46         expression
47         array-initializer
48         stackalloc-initializer
49
50     stackalloc-initializer:
51         stackalloc unmanaged-type [ expression ]
52
53     End of informative text.

```


B. Portability issues

This clause is informative.

This annex collects some information about portability that appears in this ECMA Standard.

B.1 Undefined behavior

A program that does not contain any occurrences of the `unsafe` modifier cannot exhibit any undefined behavior.

The behavior is undefined in the following circumstances:

1. When dereferencing the result of converting one pointer type to another, and the resulting pointer is not correctly aligned for the pointed-to type. (§25.4)
2. When the unary `*` operator is applied to a pointer containing an invalid value (§25.5.1).
3. When a pointer is subscripted to access an out-of-bounds element (§25.5.3).
4. Modifying objects of managed type through fixed pointers (§25.6)
5. The initial content of memory allocated by `stackalloc` (§25.7).

B.2 Implementation-defined behavior

A conforming implementation is required to document its choice of behavior in each of the areas listed in this clause. The following are implementation-defined:

- 1 • The behavior when an identifier not in Normalization Form C is encountered (§9.4.2).
- 2 • The values of any application parameters passed to `Main` by the host environment prior to application
- 3 startup (§10.1).
- 4 • The mechanism by which linkage to an external function is achieved (§17.5.7).
- 5 • The impact of thread termination when no matching catch clause is found for an exception and the code
- 6 that initially started that thread is reached. (§23.3)
- 7 • The purpose of attribute target specifiers other than those identified by this standard (§24.2).
- 8 • The mappings between pointers and integers (§25.4).
- 9 • The effect of applying the unary `*` operator to a `null` pointer (§25.5.1).
- 10 • The behavior when pointer arithmetic overflows the domain of the pointer type (§25.5.5).
- 11 • The result of the `sizeof` operator for other than the pre-defined value types (§25.5.8).
- 12 • The behavior of the `fixed` statement if the array expression is null or if the array has zero
- 13 elements (§25.6).
- 14 • The behavior of the `fixed` statement if the string expression is null (§25.6).
- 15 • The value returned when a stack allocation of size zero is made (§25.7).

16 B.3 Unspecified behavior

- 17
- 18 • The time at which the destructor (if any) for an object is run, once that object has become eligible for
- 19 destruction.
- 20 • The value of the result when converting out-of-range values from `float` or `double` values to an integral
- 21 type in an unchecked context (§13.2.1).
- 22 • The layout of arrays, except in an unsafe context (§14.5.10.2).
- 23 • The exact timing of static field initialization (§17.4.5.1).
- 24 • The order in which members are packed into a struct (§25.5.8).

25 B.4 Other Issues

- 26
- 27 1. The exact results of floating-point expression evaluation may vary from one implementation to another,
- 28 because an implementation is permitted to evaluate such expressions using a greater range and/or
- 29 precision than is required. (§11.1.5)
- 30 2. The CLI reserves certain signatures for compatibility with other programming languages. (§17.2.7)

31 **End of informative text.**

C. Naming guidelines

This annex is informative.

One of the most important elements of predictability and discoverability is the use of a consistent naming pattern. Many of the common user questions don't even arise once these conventions are understood and widely used. There are three elements to the naming guidelines:

1. Casing – use of the correct capitalization style
2. Mechanical – use nouns for classes, verbs for methods, etc.
3. Word choice – use consistent terms across class libraries.

The following section lays out rules for the first two elements, and some philosophy for the third.

C.1 Capitalization styles

The following section describes different ways of capitalizing identifiers.

C.1.1 Pascal casing

This convention capitalizes the first character of each word. For example:

```
color    BitConverter
```

C.1.2 Camel casing

This convention capitalizes the first character of each word except the first word. For example:

```
backgroundColor    totalValueCount
```

C.1.3 All uppercase

Only use all uppercase letters for an identifier if it contains an abbreviation. For example:

```
System.IO  
System.WinForms.UI
```

C.1.4 Capitalization summary

The following table summarizes the capitalization style for the different kinds of identifiers:

Type	Case	Notes
Class	PascalCase	
Attribute Class	PascalCase	Has a suffix of <code>Attribute</code>
Exception Class	PascalCase	Has a suffix of <code>Exception</code>
Constant	PascalCase	
Enum type	PascalCase	
Enum values	PascalCase	
Event	PascalCase	
Interface	PascalCase	Has a prefix of <code>I</code>
Local variable	camelCase	
Method	PascalCase	
Namespace	PascalCase	
Property	PascalCase	
Public Instance Field	PascalCase	Rarely used (use a property instead)
Protected Instance Field	camelCase	Rarely used (use a property instead)
Parameter	camelCase	

1

2 **C.2 Word choice**

- 3 • Do avoid using class names duplicated in heavily used namespaces. For example, don't use the
4 following for a class name.

5 `System` `Collections` `Forms` `UI`

- 6 • Do not use abbreviations in identifiers.

- 7 • If you must use abbreviations, do use camelCase for any abbreviation containing more than two
8 characters, even if this is not the usual abbreviation.

9 **C.3 Namespaces**

10 The general rule for namespace naming is: `CompanyName.TechnologyName`.

- Do avoid the possibility of two published namespaces having the same name, by prefixing namespace names with a company name or other well-established brand. For example, `Microsoft.Office` for the Office Automation classes provided by Microsoft.
- Do use PascalCase, and separate logical components with periods (as in `Microsoft.Office.PowerPoint`). If your brand employs non-traditional casing, do follow the casing defined by your brand, even if it deviates from normal namespace casing (for example, `NeXT.WebObjects`, and `ee.cummings`).
- Do use plural namespace names where appropriate. For example, use `System.Collections` rather than `System.Collection`. Exceptions to this rule are brand names and abbreviations. For example, use `System.IO` not `System.IOs`.
- Do not have namespaces and classes with the same name.

C.4 Classes

- Do name classes with nouns or noun phrases.
 - Do use PascalCase.
 - Do use sparingly, abbreviations in class names.
 - Do not use any prefix (such as “C”, for example). Where possible, avoid starting with the letter “I”, since that is the recommended prefix for interface names. If you must start with that letter, make sure the second character is lowercase, as in `IdentityStore`.
 - Do not use any underscores.
- ```

public class FileStream { ... }
public class Button { ... }
public class String { ... }

```

## C.5 Interfaces

- Do name interfaces with nouns or noun phrases, or adjectives describing behavior. For example, `IComponent` (descriptive noun), `ICustomAttributeProvider` (noun phrase), and `IPersistable` (adjective).
- Do use PascalCase.
- Do use sparingly, abbreviations in interface names.
- Do not use any underscores.
- Do prefix interface names with the letter “I”, to indicate that the type is an interface.
- Do use similar names when defining a class/interface pair where the class is a standard implementation of the interface. The names should differ only by the “I” prefix in the interface name. This approach is used for the interface `IComponent` and its standard implementation, `Component`.

```

public interface IComponent { ... }
public class Component : IComponent { ... }
public interface IServiceProvider { ... }
public interface IFormatable { ... }

```

**C.6 Enums**

- Do use PascalCase for enums.
- Do use PascalCase for enum value names.
- Do use sparingly, abbreviations in enum names.
- Do not use a family-name prefix on enum.
- Do not use any “Enum” suffix on enum types.
- Do use a singular name for enums
- Do use a plural name for bit fields
- Do define enumerated values using an enum if they are used in a parameter or property. This gives development tools a chance at knowing the possible values for a property or parameter.

```

11 public enum FileMode{
12 Create,
13 CreateNew,
14 Open,
15 OpenOrCreate,
16 Truncate
17 }

```

- Do use the Flags custom attribute if the numeric values are meant to be bitwise ORed together

```

19 [Flags]
20 public enum Bindings {
21 CreateInstance,
22 DefaultBinding,
23 ExcatBinding,
24 GetField,
25 GetProperty,
26 IgnoreCase,
27 InvokeMethod,
28 NonPublic,
29 OABinding,
30 SetField
31 SetProperty,
32 Static
33 }

```

- Do use `int` as the underlying type of an enum. (An exception to this rule is if the enum represents flags and there are more than 32 flags, or the enum may grow to that many flags in the future, or the type needs to be different from `int` for backward compatibility.)
- Do use enums only if the value can be completely expressed as a set of bit flags. Do not use enums for open sets (such as operating system version).

## C.7 Static fields

- Do name static members with nouns, noun phrases, or abbreviations for nouns.
- Do name static members using PascalCase.
- Do not use Hungarian-type prefixes on static member names.

## C.8 Parameters

- Do use descriptive names such that a parameter's name and type clearly imply its meaning.
- Do name parameters using camelCase.
- Do prefer names based on a parameter's meaning, to names based on the parameter's type. It is likely that development tools will provide the information about type in a convenient way, so the parameter name can be put to better use describing semantics rather than type.
- Do not reserve parameters for future use. If more data is need in the next version, a new overload can be added.
- Do not use Hungarian-type prefixes.

```
Type GetType (string typeName)
string Format (string format, object [] args)
```

## C.9 Methods

- Do name methods with verbs or verb phrases.
  - Do name methods with PascalCase
- ```
RemoveAll(), GetCharArray(), Invoke()
```

C.10 Properties

- Do name properties using noun or noun phrases
- Do name properties with PascalCase
- Consider having a property with the same as a type. When declaring a property with the same name as a type, also make the type of the property be that type. In other words, the following is okay

```
public enum Color {...}
public class Control {
    public Color Color { get {...} set {...} }
}
```

but this is not

```
public enum Color {...}
public class Control {
    public int Color { get {...} set {...} }
}
```

In the latter case, it will not be possible to refer to the members of the `Color` enum because `Color.xxx` will be interpreted as being a member access that first gets the value of the `Color` property (of type `int`) and then accesses a member of that value (which would have to be an instance member of `System.Int32`).

C.11 Events

- Do name event handlers with the “EventHandler” suffix.

```
public delegate void MouseEventHandler(object sender, MouseEventArgs e);
```

- Do use two parameters named `sender` and `e`. The `sender` parameter represents the object that raised the event, and this parameter is always of type `object`, even if it is possible to employ a more specific type. The state associated with the event is encapsulated in an instance `e` of an event class. Use an appropriate and specific event class for its type.

```
public delegate void MouseEventHandler(object sender, MouseEventArgs e);
```

- Do name event argument classes with the “EventArgs” suffix.

```
public class MouseEventArgs : EventArgs {
    int x;
    int y;
    public MouseEventArgs(int x, int y)
        { this.x = x; this.y = y; }
    public int x { get { return x; } }
    public int y { get { return y; } }
}
```

- Do name event names that have a concept of pre- and post-operation using the present and past tense (do not use `BeforeXxx/AfterXxx` pattern). For example, a close event that could be canceled would have a `Closing` and `Closed` event.

```
public event ControlEventHandler ControlAdded {
    //..
}
```

- Consider naming events with a verb.

C.12 Case sensitivity

- Don't use names that require case sensitivity. Components might need to be usable from both case-sensitive and case-insensitive languages. Since case-insensitive languages cannot distinguish between two names within the same context that differ only by case, components must avoid this situation.

Examples of what not to do:

- Don't have two namespaces whose names differ only by case.

```
namespace ee.cummings;
namespace Ee.Cummings;
```

- Don't have a method with two parameters whose names differ only by case.

```
void F(string a, string A)
```

- Don't have a namespace with two types whose names differ only by case.

```
System.Windows.Point p;
System.Windows.POINT pp;
```

- Don't have a type with two properties whose names differ only by case.

```
int F {get, set};
int F {get, set}
```

- Don't have a type with two methods whose names differ only by case.

```
void f();
void F();
```


1 C.13 Avoiding type name confusion

2 Different languages use different names to identify the fundamental managed types, so in a multi-language
 3 environment, designers must take care to avoid language-specific terminology. This section describes a set
 4 of rules that help avoid type name confusion.

- 5 • Do use semantically interesting names rather than type names.
- 6 • In the rare case that a parameter has no semantic meaning beyond its type, use a generic name. For
 7 example, a class that supports writing a variety of data types into a stream might have:

```
8     void write(double value);
9     void write(float value);
10    void write(long value);
11    void write(int value);
12    void write(short value);
```

13 rather than a language-specific alternative such as:

```
14    void write(double doublevalue);
15    void write(float floatValue);
16    void write(long longvalue);
17    void write(int intValue);
18    void write(short shortValue);
```

- 19 • In the extremely rare case that it is necessary to have a uniquely named method for each fundamental
 20 data type, do use the following universal type names: Sbyte, Byte, Int16, UInt16, Int32, UInt32,
 21 Int64, UInt64, Single, Double, Boolean, Char, String, and Object. For example, a class that
 22 supports reading a variety of data types from a stream might have:

```
23    double ReadDouble();
24    float ReadSingle();
25    long ReadInt64();
26    int ReadInt32();
27    short ReadInt16();
```

28 rather than a language-specific alternative such as:

```
29    double ReadDouble();
30    float ReadFloat();
31    long ReadLong();
32    int ReadInt();
33    short ReadShort();
```

34 **End of informative text.**

D. Standard Library

A conforming C# implementation shall provide a set of types having specific semantics. For convenience, these types and their members are listed here, in alphabetical order. For a formal definition of these types and their members, refer to ECMA-335, 1st Edition, December 2001, *Common Language Infrastructure (CLI), Partition IV; Base Class Library (BCL), Extended Numerics Library, and Extended Array Library*, which are included by reference in this ECMA Standard.

This rest of this clause is informative.

```
// Namespace: System, Library: BCL
public class ApplicationException: Exception
{
    public ApplicationException();
    public ApplicationException(string message);
    public ApplicationException(string message, Exception innerException);
}

// Namespace: System, Library: BCL
public class ArgumentException: SystemException
{
    public ArgumentException();
    public ArgumentException(string message);
    public ArgumentException(string message, Exception innerException);
    public ArgumentException(string message, string paramName, Exception
        innerException);
    public ArgumentException(string message, string paramName);
    public virtual string ParamName { get; }
}

// Namespace: System, Library: BCL
public class ArgumentNullException: ArgumentException
{
    public ArgumentNullException();
    public ArgumentNullException(string paramName);
    public ArgumentNullException(string paramName, string message);
}

// Namespace: System, Library: BCL
public class ArgumentOutOfRangeException: ArgumentException
{
    public ArgumentOutOfRangeException();
```

```

1      public ArgumentOutOfRangeException(string paramName);
2      public ArgumentOutOfRangeException(string paramName, string message);
3      public ArgumentOutOfRangeException(string paramName, object actualValue,
4          string message);
5      public virtual object ActualValue { get; }
6  }
7
8  // Namespace: System, Library: BCL
9  public class ArithmeticException: SystemException
10 {
11     public ArithmeticException();
12     public ArithmeticException(string message);
13     public ArithmeticException(string message, Exception innerException);
14 }
15
16 // Namespace: System, Library: BCL
17 public abstract class Array: ICloneable, ICollection, IEnumerable, IList
18 {
19     protected Array();
20     public static int BinarySearch(Array array, object value);
21     public static int BinarySearch(Array array, int index, int length, object
22         value);
23     public static int BinarySearch(Array array, object value, IComparer
24         comparer);
25     public static int BinarySearch(Array array, int index, int length, object
26         value, IComparer comparer);
27     public static void Clear(Array array, int index, int length);
28     public virtual object Clone();
29     public static void Copy(Array sourceArray, Array destinationArray, int
30         length);
31     public static void Copy(Array sourceArray, int sourceIndex, Array
32         destinationArray, int destinationIndex, int length);
33     public virtual void CopyTo(Array array, int index);
34     public static Array CreateInstance(Type elementType, int length);
35     public static Array CreateInstance(Type elementType, int length1, int
36         length2);
37     public static Array CreateInstance(Type elementType, int length1, int
38         length2, int length3);
39     public static Array CreateInstance(Type elementType, int[] lengths);
40     public static Array CreateInstance(Type elementType, int[] lengths, int[]
41         lowerBounds);
42     public virtual IEnumerator GetEnumerator();
43     public object GetValue(int[] indices);
44     public object GetValue(int index);

```

```

1      public object GetValue(int index1, int index2);
2      public object GetValue(int index1, int index2, int index3);
3      public static int IndexOf(Array array, object value);
4      public static int IndexOf(Array array, object value, int startIndex);
5      public static int IndexOf(Array array, object value, int startIndex, int
6          count);
7      public static int LastIndexOf(Array array, object value);
8      public static int LastIndexOf(Array array, object value, int startIndex);
9      public static int LastIndexOf(Array array, object value, int startIndex,
10         int count);
11     public static void Reverse(Array array);
12     public static void Reverse(Array array, int index, int length);
13     public void SetValue(object value, int index);
14     public void SetValue(object value, int index1, int index2);
15     public void SetValue(object value, int index1, int index2, int index3);
16     public void SetValue(object value, int[] indices);
17     public static void Sort(Array array);
18     public static void Sort(Array keys, Array items);
19     public static void Sort(Array array, int index, int length);
20     public static void Sort(Array keys, Array items, int index, int length);
21     public static void Sort(Array array, IComparer comparer);
22     public static void Sort(Array keys, Array items, IComparer comparer);
23     public static void Sort(Array array, int index, int length, IComparer
24         comparer);
25     public static void Sort(Array keys, Array items, int index, int length,
26         IComparer comparer);
27     int IList.Add(object value);
28     void IList.Clear();
29     bool IList.Contains(object value);
30     int IList.IndexOf(object value);
31     void IList.Insert(int index, object value);
32     void IList.Remove(object value);
33     void IList.RemoveAt(int index);
34     bool IList.IsFixedSize { get; }
35     bool IList.IsReadOnly { get; }
36     bool ICollection.IsSynchronized { get; }
37     public int Length { get; }
38     public long LongLength {get;}
39     public int Rank { get; }
40     object ICollection.SyncRoot { get; }
41     int ICollection.Count { get; }
42     public virtual object this[int index] { get; set; }
43 }
44

```

```

1  // Namespace: System.Collections, Library: BCL
2  public class ArrayList: ICloneable, ICollection, IEnumerable, IList
3  {
4      public ArrayList();
5      public ArrayList(int capacity);
6      public ArrayList(ICollection c);
7      public static ArrayList Adapter(IList list);
8      public virtual int Add(object value);
9      public virtual void AddRange(ICollection c);
10     public virtual int BinarySearch(object value, IComparer comparer);
11     public virtual int BinarySearch(object value);
12     public virtual int BinarySearch(int index, int count, object value,
13         IComparer comparer);
14     public virtual void Clear();
15     public virtual object Clone();
16     public virtual bool Contains(object item);
17     public virtual void CopyTo(Array array, int arrayIndex);
18     public virtual void CopyTo(int index, Array array, int arrayIndex, int
19         count);
20     public virtual void CopyTo(Array array);
21     public static ArrayList FixedSize(ArrayList list);
22     public virtual IEnumerator GetEnumerator();
23     public virtual IEnumerator GetEnumerator(int index, int count);
24     public virtual ArrayList GetRange(int index, int count);
25     public virtual int IndexOf(object value);
26     public virtual int IndexOf(object value, int startIndex, int count);
27     public virtual int IndexOf(object value, int startIndex);
28     public virtual void Insert(int index, object value);
29     public virtual void InsertRange(int index, ICollection c);
30     public virtual int LastIndexOf(object value, int startIndex, int count);
31     public virtual int LastIndexOf(object value, int startIndex);
32     public virtual int LastIndexOf(object value);
33     public static ArrayList ReadOnly(ArrayList list);
34     public virtual void Remove(object obj);
35     public virtual void RemoveAt(int index);
36     public virtual void RemoveRange(int index, int count);
37     public static ArrayList Repeat(object value, int count);
38     public virtual void Reverse(int index, int count);
39     public virtual void Reverse();
40     public virtual void SetRange(int index, ICollection c);
41     public virtual void Sort(int index, int count, IComparer comparer);
42     public virtual void Sort(IComparer comparer);
43     public virtual void Sort();
44     public static ArrayList Synchronized(ArrayList list);

```

```

1      public virtual Array ToArray(Type type);
2      public virtual object[] ToArray();
3      public virtual void TrimToSize();
4      public virtual int Capacity { get; set; }
5      int ICollection.Count { get; }
6      public virtual int Count { get; }
7      bool IList.IsFixedSize { get; }
8      public virtual bool IsFixedSize { get; }
9      bool IList.IsReadOnly { get; }
10     public virtual bool IsReadOnly { get; }
11     bool ICollection.IsSynchronized { get; }
12     public virtual bool IsSynchronized { get; }
13     public virtual object this[int index] { get; set; }
14     object ICollection.SyncRoot { get; }
15     public virtual object SyncRoot { get; }
16 }
17
18 // Namespace: System, Library: BCL
19 public class ArrayTypeMismatchException: SystemException
20 {
21     public ArrayTypeMismatchException();
22     public ArrayTypeMismatchException(string message);
23     public ArrayTypeMismatchException(string message, Exception innerException);
24 }
25
26 // Namespace: System.Text, Library: BCL
27 public class ASCIIEncoding: Encoding
28 {
29     public ASCIIEncoding();
30     public override int GetByteCount(char[] chars, int index, int count);
31     public override int GetByteCount(string chars);
32     public override int GetBytes(string chars, int charIndex, int charCount,
33         byte[] bytes, int byteIndex);
34     public override int GetBytes(char[] chars, int charIndex, int charCount,
35         byte[] bytes, int byteIndex);
36     public override int GetCharCount(byte[] bytes, int index, int count);
37     public override int GetChars(byte[] bytes, int byteIndex, int byteCount,
38         char[] chars, int charIndex);
39     public override int GetMaxByteCount(int charCount);
40     public override int GetMaxCharCount(int byteCount);
41     public override string GetString(byte[] bytes, int byteIndex, int
42         byteCount);
43     public override string GetString(byte[] bytes);
44 }

```

```

1
2 // Namespace: System, Library: BCL
3 public delegate void AsyncCallback(IAsyncResult ar);
4
5 // Namespace: System, Library: BCL
6 public abstract class Attribute
7 {
8     protected Attribute();
9     public override bool Equals(object obj);
10    public override int GetHashCode();
11 }
12
13 // Namespace: System, Library: BCL
14 public enum AttributeTargets
15 {
16     All = Assembly | 0x2 | Class | Struct | Enum | Constructor | Method |
17         Property | Field | Event | Interface | Parameter | Delegate |
18         ReturnValue,
19     Assembly = 0x1,
20     Class = 0x4,
21     Constructor = 0x20,
22     Delegate = 0x1000,
23     Enum = 0x10,
24     Event = 0x200,
25     Field = 0x100,
26     Interface = 0x400,
27     Method = 0x40,
28     Parameter = 0x800,
29     Property = 0x80,
30     ReturnValue = 0x2000,
31     Struct = 0x8,
32 }
33
34 // Namespace: System, Library: BCL
35 public sealed class AttributeUsageAttribute: Attribute
36 {
37     public AttributeUsageAttribute(AttributeTargets validOn);
38     public bool AllowMultiple { get; set; }
39     public bool Inherited { get; set; }
40     public AttributeTargets ValidOn { get; }
41 }
42
43 // Namespace: System, Library: BCL
44 public struct Boolean: IComparable

```



```

1  {
2      public static readonly string FalseString;
3      public static readonly string TrueString;
4      public int CompareTo(object obj);
5      public override bool Equals(object obj);
6      public override int GetHashCode();
7      public static bool Parse(string value);
8      public string ToString(IFormatProvider provider);
9      public override string ToString();
10 }
11
12 // Namespace: System, Library: BCL
13 public struct Byte: IComparable, IFormattable
14 {
15     public const byte MaxValue = 255;
16     public const byte MinValue = 0;
17     public int CompareTo(object value);
18     public override bool Equals(object obj);
19     public override int GetHashCode();
20     public static byte Parse(string s);
21     public static byte Parse(string s, NumberStyles style);
22     public static byte Parse(string s, IFormatProvider provider);
23     public static byte Parse(string s, NumberStyles style, IFormatProvider
24         provider);
25     public string ToString(IFormatProvider provider);
26     public string ToString(string format, IFormatProvider provider);
27     public override string ToString();
28     public string ToString(string format);
29 }
30
31 // Namespace: System, Library: BCL
32 public struct Char: IComparable
33 {
34     public const char MaxValue = (char)0xFFFF;
35     public const char MinValue = (char)0x0;
36     public int CompareTo(object value);
37     public override bool Equals(object obj);
38     public override int GetHashCode();
39     public static double GetNumericValue(char c);
40     public static double GetNumericValue(string s, int index);
41     public static UnicodeCategory GetUnicodeCategory(char c);
42     public static UnicodeCategory GetUnicodeCategory(string s, int index);
43     public static bool IsControl(char c);
44     public static bool IsControl(string s, int index);

```

```

1      public static bool IsDigit(char c);
2      public static bool IsDigit(string s, int index);
3      public static bool IsLetter(char c);
4      public static bool IsLetter(string s, int index);
5      public static bool IsLetterOrDigit(char c);
6      public static bool IsLetterOrDigit(string s, int index);
7      public static bool IsLower(char c);
8      public static bool IsLower(string s, int index);
9      public static bool IsNumber(char c);
10     public static bool IsNumber(string s, int index);
11     public static bool IsPunctuation(char c);
12     public static bool IsPunctuation(string s, int index);
13     public static bool IsSeparator(char c);
14     public static bool IsSeparator(string s, int index);
15     public static bool IsSurrogate(char c);
16     public static bool IsSurrogate(string s, int index);
17     public static bool Issymbol(char c);
18     public static bool Issymbol(string s, int index);
19     public static bool IsUpper(char c);
20     public static bool IsUpper(string s, int index);
21     public static bool IsWhiteSpace(char c);
22     public static bool IsWhiteSpace(string s, int index);
23     public static char Parse(string s);
24     public static char ToLower(char c);
25     public string ToString(IFormatProvider provider);
26     public override string ToString();
27     public static char ToUpper(char c);
28 }
29
30 // Namespace: System, Library: BCL
31 public sealed class CharEnumerator: ICloneable, IEnumerator
32 {
33     public object Clone();
34     public bool MoveNext();
35     public void Reset();
36     public char Current { get; }
37     object IEnumerator.Current { get; }
38 }
39
40 // Namespace: System, Library: BCL
41 public sealed class CLSCompliantAttribute: Attribute
42 {
43     public CLSCompliantAttribute(bool isCompliant);
44     public bool IsCompliant { get; }

```

```

1  }
2
3  // Namespace: System.Security, Library: BCL
4  public abstract class CodeAccessPermission: IPermission
5  {
6      protected CodeAccessPermission();
7      public void Assert();
8      public abstract IPermission Copy();
9      public void Demand();
10     public void Deny();
11     public abstract void FromXml(SecurityElement elem);
12     public abstract IPermission Intersect(IPermission target);
13     public abstract bool IsSubsetOf(IPermission target);
14     public override string ToString();
15     public abstract SecurityElement ToXml();
16     public virtual IPermission Union(IPermission other);
17 }
18
19 // Namespace: System.Security.Permissions, Library: BCL
20 public abstract class CodeAccessSecurityAttribute: SecurityAttribute
21 {
22     protected CodeAccessSecurityAttribute();
23     public CodeAccessSecurityAttribute(SecurityAction action);
24 }
25
26 // Namespace: System.Collections, Library: BCL
27 public sealed class Comparer: IComparer
28 {
29     public static readonly Comparer Default;
30     public int Compare(object a, object b);
31 }
32
33 // Namespace: System.Diagnostics, Library: BCL
34 public sealed class ConditionalAttribute: Attribute
35 {
36     public ConditionalAttribute(string conditionString);
37     public string ConditionString { get; }
38 }
39
40 // Namespace: System, Library: BCL
41 public sealed class Console
42 {
43     public static Stream OpenStandardError();
44     public static Stream OpenStandardError(int bufferSize);

```

```

1      public static Stream OpenStandardInput();
2      public static Stream OpenStandardInput(int bufferSize);
3      public static Stream OpenStandardOutput();
4      public static Stream OpenStandardOutput(int bufferSize);
5      public static int Read();
6      public static string ReadLine();
7      public static void SetError(TextWriter newError);
8      public static void SetIn(TextReader newIn);
9      public static void SetOut(TextWriter newOut);
10     public static void Write(string format, object arg0);
11     public static void Write(string format, object arg0, object arg1);
12     public static void Write(string format, object arg0, object arg1, object
13         arg2);
14     public static void Write(string format, params object[] arg);
15     public static void Write(bool value);
16     public static void Write(char value);
17     public static void Write(char[] buffer);
18     public static void Write(char[] buffer, int index, int count);
19     public static void Write(double value);
20     public static void Write(decimal value);
21     public static void Write(float value);
22     public static void Write(int value);
23     public static void Write(uint value);
24     public static void Write(long value);
25     public static void Write(ulong value);
26     public static void Write(object value);
27     public static void Write(string value);
28     public static void WriteLine();
29     public static void WriteLine(bool value);
30     public static void WriteLine(char value);
31     public static void WriteLine(char[] buffer);
32     public static void WriteLine(char[] buffer, int index, int count);
33     public static void WriteLine(decimal value);
34     public static void WriteLine(double value);
35     public static void WriteLine(float value);
36     public static void WriteLine(int value);
37     public static void WriteLine(uint value);
38     public static void WriteLine(long value);
39     public static void WriteLine(ulong value);
40     public static void WriteLine(object value);
41     public static void WriteLine(string value);
42     public static void WriteLine(string format, object arg0);
43     public static void WriteLine(string format, object arg0, object arg1);
44     public static void WriteLine(string format, object arg0, object arg1,

```

```

1      object arg2);
2      public static void WriteLine(string format, params object[] arg);
3      public static TextWriter Error { get; }
4      public static TextReader In { get; }
5      public static TextWriter Out { get; }
6  }
7
8  // Namespace: System, Library: BCL
9  public sealed class Convert
10 {
11     public static bool ToBoolean(bool value);
12     public static bool ToBoolean(sbyte value);
13     public static bool ToBoolean(byte value);
14     public static bool ToBoolean(short value);
15     public static bool ToBoolean(ushort value);
16     public static bool ToBoolean(int value);
17     public static bool ToBoolean(uint value);
18     public static bool ToBoolean(long value);
19     public static bool ToBoolean(ulong value);
20     public static bool ToBoolean(string value);
21     public static bool ToBoolean(float value);
22     public static bool ToBoolean(double value);
23     public static bool ToBoolean(decimal value);
24     public static byte ToByte(bool value);
25     public static byte ToByte(byte value);
26     public static byte ToByte(char value);
27     public static byte ToByte(sbyte value);
28     public static byte ToByte(short value);
29     public static byte ToByte(ushort value);
30     public static byte ToByte(int value);
31     public static byte ToByte(uint value);
32     public static byte ToByte(long value);
33     public static byte ToByte(ulong value);
34     public static byte ToByte(float value);
35     public static byte ToByte(double value);
36     public static byte ToByte(decimal value);
37     public static byte ToByte(string value);
38     public static byte ToByte(string value, IFormatProvider provider);
39     public static char ToChar(char value);
40     public static char ToChar(sbyte value);
41     public static char ToChar(byte value);
42     public static char ToChar(short value);
43     public static char ToChar(ushort value);
44     public static char ToChar(int value);

```

```
1    public static char ToChar(uint value);
2    public static char ToChar(long value);
3    public static char ToChar(ulong value);
4    public static char ToChar(string value);
5    public static DateTime ToDateTime(DateTime value);
6    public static DateTime ToDateTime(string value);
7    public static DateTime ToDateTime(string value, IFormatProvider provider);
8    public static decimal ToDecimal(sbyte value);
9    public static decimal ToDecimal(byte value);
10   public static decimal ToDecimal(short value);
11   public static decimal ToDecimal(ushort value);
12   public static decimal ToDecimal(int value);
13   public static decimal ToDecimal(uint value);
14   public static decimal ToDecimal(long value);
15   public static decimal ToDecimal(ulong value);
16   public static decimal ToDecimal(float value);
17   public static decimal ToDecimal(double value);
18   public static decimal ToDecimal(string value);
19   public static decimal ToDecimal(string value, IFormatProvider provider);
20   public static decimal ToDecimal(decimal value);
21   public static decimal ToDecimal(bool value);
22   public static double ToDouble(sbyte value);
23   public static double ToDouble(byte value);
24   public static double ToDouble(short value);
25   public static double ToDouble(ushort value);
26   public static double ToDouble(int value);
27   public static double ToDouble(uint value);
28   public static double ToDouble(long value);
29   public static double ToDouble(ulong value);
30   public static double ToDouble(float value);
31   public static double ToDouble(double value);
32   public static double ToDouble(decimal value);
33   public static double ToDouble(string value);
34   public static double ToDouble(string value, IFormatProvider provider);
35   public static double ToDouble(bool value);
36   public static short.ToInt16(bool value);
37   public static short.ToInt16(char value);
38   public static short.ToInt16(sbyte value);
39   public static short.ToInt16(byte value);
40   public static short.ToInt16(ushort value);
41   public static short.ToInt16(int value);
42   public static short.ToInt16(uint value);
43   public static short.ToInt16(short value);
44   public static short.ToInt16(long value);
```

```
1 public static short ToInt16(ulong value);
2 public static short ToInt16(float value);
3 public static short ToInt16(double value);
4 public static short ToInt16(decimal value);
5 public static short ToInt16(string value);
6 public static short ToInt16(string value, IFormatProvider provider);
7 public static int ToInt32(bool value);
8 public static int ToInt32(char value);
9 public static int ToInt32(sbyte value);
10 public static int ToInt32(byte value);
11 public static int ToInt32(short value);
12 public static int ToInt32(ushort value);
13 public static int ToInt32(uint value);
14 public static int ToInt32(int value);
15 public static int ToInt32(long value);
16 public static int ToInt32(ulong value);
17 public static int ToInt32(float value);
18 public static int ToInt32(double value);
19 public static int ToInt32(decimal value);
20 public static int ToInt32(string value);
21 public static int ToInt32(string value, IFormatProvider provider);
22 public static long ToInt64(bool value);
23 public static long ToInt64(char value);
24 public static long ToInt64(sbyte value);
25 public static long ToInt64(byte value);
26 public static long ToInt64(short value);
27 public static long ToInt64(ushort value);
28 public static long ToInt64(int value);
29 public static long ToInt64(uint value);
30 public static long ToInt64(ulong value);
31 public static long ToInt64(long value);
32 public static long ToInt64(float value);
33 public static long ToInt64(double value);
34 public static long ToInt64(decimal value);
35 public static long ToInt64(string value);
36 public static long ToInt64(string value, IFormatProvider provider);
37 public static sbyte ToSByte(bool value);
38 public static sbyte ToSByte(sbyte value);
39 public static sbyte ToSByte(char value);
40 public static sbyte ToSByte(byte value);
41 public static sbyte ToSByte(short value);
42 public static sbyte ToSByte(ushort value);
43 public static sbyte ToSByte(int value);
44 public static sbyte ToSByte(uint value);
```

```
1    public static sbyte ToSByte(long value);
2    public static sbyte ToSByte(ulong value);
3    public static sbyte ToSByte(float value);
4    public static sbyte ToSByte(double value);
5    public static sbyte ToSByte(decimal value);
6    public static sbyte ToSByte(string value);
7    public static sbyte ToSByte(string value, IFormatProvider provider);
8    public static float ToSingle(sbyte value);
9    public static float ToSingle(byte value);
10   public static float ToSingle(short value);
11   public static float ToSingle(ushort value);
12   public static float ToSingle(int value);
13   public static float ToSingle(uint value);
14   public static float ToSingle(long value);
15   public static float ToSingle(ulong value);
16   public static float ToSingle(float value);
17   public static float ToSingle(double value);
18   public static float ToSingle(decimal value);
19   public static float ToSingle(string value);
20   public static float ToSingle(string value, IFormatProvider provider);
21   public static float ToSingle(bool value);
22   public static string ToString(bool value);
23   public static string ToString(char value);
24   public static string ToString(sbyte value);
25   public static string ToString(sbyte value, IFormatProvider provider);
26   public static string ToString(byte value);
27   public static string ToString(byte value, IFormatProvider provider);
28   public static string ToString(short value);
29   public static string ToString(short value, IFormatProvider provider);
30   public static string ToString(ushort value);
31   public static string ToString(ushort value, IFormatProvider provider);
32   public static string ToString(int value);
33   public static string ToString(int value, IFormatProvider provider);
34   public static string ToString(uint value);
35   public static string ToString(uint value, IFormatProvider provider);
36   public static string ToString(long value);
37   public static string ToString(long value, IFormatProvider provider);
38   public static string ToString(ulong value);
39   public static string ToString(ulong value, IFormatProvider provider);
40   public static string ToString(float value);
41   public static string ToString(float value, IFormatProvider provider);
42   public static string ToString(double value);
43   public static string ToString(double value, IFormatProvider provider);
44   public static string ToString(decimal value);
```



```
1 public static string ToString(decimal value, IFormatProvider provider);
2 public static string ToString(DateTime value);
3 public static string ToString(DateTime value, IFormatProvider provider);
4 public static string ToString(string value);
5 public static ushort ToUInt16(bool value);
6 public static ushort ToUInt16(char value);
7 public static ushort ToUInt16(sbyte value);
8 public static ushort ToUInt16(byte value);
9 public static ushort ToUInt16(short value);
10 public static ushort ToUInt16(int value);
11 public static ushort ToUInt16(ushort value);
12 public static ushort ToUInt16(uint value);
13 public static ushort ToUInt16(long value);
14 public static ushort ToUInt16(ulong value);
15 public static ushort ToUInt16(float value);
16 public static ushort ToUInt16(double value);
17 public static ushort ToUInt16(decimal value);
18 public static ushort ToUInt16(string value);
19 public static ushort ToUInt16(string value, IFormatProvider provider);
20 public static uint ToUInt32(bool value);
21 public static uint ToUInt32(char value);
22 public static uint ToUInt32(sbyte value);
23 public static uint ToUInt32(byte value);
24 public static uint ToUInt32(short value);
25 public static uint ToUInt32(ushort value);
26 public static uint ToUInt32(int value);
27 public static uint ToUInt32(uint value);
28 public static uint ToUInt32(long value);
29 public static uint ToUInt32(ulong value);
30 public static uint ToUInt32(float value);
31 public static uint ToUInt32(double value);
32 public static uint ToUInt32(decimal value);
33 public static uint ToUInt32(string value);
34 public static uint ToUInt32(string value, IFormatProvider provider);
35 public static ulong ToUInt64(bool value);
36 public static ulong ToUInt64(char value);
37 public static ulong ToUInt64(sbyte value);
38 public static ulong ToUInt64(byte value);
39 public static ulong ToUInt64(short value);
40 public static ulong ToUInt64(ushort value);
41 public static ulong ToUInt64(int value);
42 public static ulong ToUInt64(uint value);
43 public static ulong ToUInt64(long value);
44 public static ulong ToUInt64(ulong value);
```

```

1      public static ulong ToUInt64(float value);
2      public static ulong ToUInt64(double value);
3      public static ulong ToUInt64(decimal value);
4      public static ulong ToUInt64(string value);
5      public static ulong ToUInt64(string value, IFormatProvider provider);
6  }
7
8  // Namespace: System, Library: BCL
9  public struct DateTime: IComparable, IFormattable
10 {
11     public DateTime(long ticks);
12     public DateTime(int year, int month, int day);
13     public DateTime(int year, int month, int day, int hour, int minute, int
14         second);
15     public DateTime(int year, int month, int day, int hour, int minute, int
16         second, int millisecond);
17     public static readonly DateTime MaxValue;
18     public static readonly DateTime MinValue;
19     public DateTime Add(TimeSpan value);
20     public DateTime AddDays(double value);
21     public DateTime AddHours(double value);
22     public DateTime AddMilliseconds(double value);
23     public DateTime AddMinutes(double value);
24     public DateTime AddMonths(int months);
25     public DateTime AddSeconds(double value);
26     public DateTime AddTicks(long value);
27     public DateTime AddYears(int value);
28     public static int Compare(DateTime t1, DateTime t2);
29     public int CompareTo(object value);
30     public static int DaysInMonth(int year, int month);
31     public override bool Equals(object value);
32     public static bool Equals(DateTime t1, DateTime t2);
33     public override int GetHashCode();
34     public static bool IsLeapYear(int year);
35     public static DateTime operator +(DateTime d, TimeSpan t);
36     public static bool operator ==(DateTime d1, DateTime d2);
37     public static bool operator >(DateTime t1, DateTime t2);
38     public static bool operator >=(DateTime t1, DateTime t2);
39     public static bool operator !=(DateTime d1, DateTime d2);
40     public static bool operator <(DateTime t1, DateTime t2);
41     public static bool operator <=(DateTime t1, DateTime t2);
42     public static DateTime operator -(DateTime d, TimeSpan t);
43     public static TimeSpan operator -(DateTime d1, DateTime d2);
44     public static DateTime Parse(string s);

```

```

1      public static DateTime Parse(string s, IFormatProvider provider);
2      public static DateTime Parse(string s, IFormatProvider provider,
3          DateTimeStyles styles);
4      public static DateTime ParseExact(string s, string format, IFormatProvider
5          provider);
6      public static DateTime ParseExact(string s, string format, IFormatProvider
7          provider, DateTimeStyles style);
8      public static DateTime ParseExact(string s, string[] formats,
9          IFormatProvider provider, DateTimeStyles style);
10     public TimeSpan Subtract(DateTime value);
11     public DateTime Subtract(TimeSpan value);
12     public DateTime ToLocalTime();
13     public string ToLongDateString();
14     public string ToLongTimeString();
15     public string ToShortDateString();
16     public string ToShortTimeString();
17     public string ToString(IFormatProvider provider);
18     public string ToString(string format, IFormatProvider provider);
19     public override string ToString();
20     public string ToString(string format);
21     public DateTime ToUniversalTime();
22     public DateTime Date { get; }
23     public int Day { get; }
24     public int DayOfYear { get; }
25     public int Hour { get; }
26     public int Millisecond { get; }
27     public int Minute { get; }
28     public int Month { get; }
29     public static DateTime Now { get; }
30     public int Second { get; }
31     public long Ticks { get; }
32     public TimeSpan TimeOfDay { get; }
33     public static DateTime Today { get; }
34     public static DateTime.UtcNow { get; }
35     public int Year { get; }
36 }
37
38 // Namespace: System.Globalization, Library: BCL
39 public sealed class DateTimeFormatInfo: ICloneable, IFormatProvider
40 {
41     public DateTimeFormatInfo();
42     public object Clone();
43     public string GetAbbreviatedMonthName(int month);
44     public int GetEra(string eraName);

```

```

1      public string GetEraName(int era);
2      public object GetFormat(Type formatType);
3      public string GetMonthName(int month);
4      public static DateTimeFormatInfo ReadOnly(DateTimeFormatInfo dtfi);
5      public string[] AbbreviatedDayNames { get; set; }
6      public string[] AbbreviatedMonthNames { get; set; }
7      public string AMDesignator { get; set; }
8      public static DateTimeFormatInfo CurrentInfo { get; }
9      public string DateSeparator { get; set; }
10     public string[] DayNames { get; set; }
11     public string FullDateTimePattern { get; set; }
12     public static DateTimeFormatInfo InvariantInfo { get; }
13     public bool IsReadOnly { get; }
14     public string LongDatePattern { get; set; }
15     public string LongTimePattern { get; set; }
16     public string MonthDayPattern { get; set; }
17     public string[] MonthNames { get; set; }
18     public string PMDesignator { get; set; }
19     public string ShortDatePattern { get; set; }
20     public string ShortTimePattern { get; set; }
21     public string TimeSeparator { get; set; }
22     public string YearMonthPattern { get; set; }
23 }
24
25 // Namespace: System.Globalization, Library: BCL
26 public enum DateTimeStyles
27 {
28     AdjustToUniversal = 0x10,
29     AllowInnerWhite = 0x4,
30     AllowLeadingWhite = 0x1,
31     AllowTrailingWhite = 0x2,
32     AllowWhiteSpaces = AllowLeadingWhite | AllowTrailingWhite | AllowInnerWhite,
33     NoCurrentDateDefault = 0x8,
34     None = 0x0,
35 }
36
37 // Namespace: System, Library: ExtendedNumerics
38 public struct Decimal: IComparable, IFormattable
39 {
40     public Decimal(int value);
41     public Decimal(uint value);
42     public Decimal(long value);
43     public Decimal(ulong value);
44     public Decimal(float value);

```

```

1    public Decimal(double value);
2    public Decimal(int[] bits);
3    public static readonly decimal MaxValue;
4    public static readonly decimal MinusOne;
5    public static readonly decimal MinValue;
6    public static readonly decimal One;
7    public static readonly decimal Zero;
8    public static decimal Add(decimal d1, decimal d2);
9    public static int Compare(decimal d1, decimal d2);
10   public int CompareTo(object value);
11   public static decimal Divide(decimal d1, decimal d2);
12   public override bool Equals(object value);
13   public static bool Equals(decimal d1, decimal d2);
14   public static decimal Floor(decimal d);
15   public static int[] GetBits(decimal d);
16   public override int GetHashCode();
17   public static decimal Multiply(decimal d1, decimal d2);
18   public static decimal Negate(decimal d);
19   public static Decimal operator +(Decimal d1, Decimal d2);
20   public static Decimal operator --(Decimal d);
21   public static Decimal operator /(Decimal d1, Decimal d2);
22   public static bool operator ==(Decimal d1, Decimal d2);
23   public static explicit operator Decimal(float value);
24   public static explicit operator Decimal(double value);
25   public static explicit operator byte(Decimal value);
26   public static explicit operator sbyte(Decimal value);
27   public static explicit operator char(Decimal value);
28   public static explicit operator short(Decimal value);
29   public static explicit operator ushort(Decimal value);
30   public static explicit operator int(Decimal value);
31   public static explicit operator uint(Decimal value);
32   public static explicit operator long(Decimal value);
33   public static explicit operator ulong(Decimal value);
34   public static explicit operator float(Decimal value);
35   public static explicit operator double(Decimal value);
36   public static bool operator >(Decimal d1, Decimal d2);
37   public static bool operator >=(Decimal d1, Decimal d2);
38   public static implicit operator Decimal(byte value);
39   public static implicit operator Decimal(sbyte value);
40   public static implicit operator Decimal(short value);
41   public static implicit operator Decimal(ushort value);
42   public static implicit operator Decimal(char value);
43   public static implicit operator Decimal(int value);
44   public static implicit operator Decimal(uint value);

```

```

1      public static implicit operator Decimal(long value);
2      public static implicit operator Decimal(ulong value);
3      public static Decimal operator ++(Decimal d);
4      public static bool operator !=(Decimal d1, Decimal d2);
5      public static bool operator <(Decimal d1, Decimal d2);
6      public static bool operator <=(Decimal d1, Decimal d2);
7      public static Decimal operator %(Decimal d1, Decimal d2);
8      public static Decimal operator *(Decimal d1, Decimal d2);
9      public static Decimal operator -(Decimal d1, Decimal d2);
10     public static Decimal operator -(Decimal d);
11     public static Decimal operator +(Decimal d);
12     public static decimal Parse(string s);
13     public static decimal Parse(string s, NumberStyles style);
14     public static decimal Parse(string s, IFormatProvider provider);
15     public static decimal Parse(string s, NumberStyles style, IFormatProvider
16         provider);
17     public static decimal Remainder(decimal d1, decimal d2);
18     public static decimal Round(decimal d, int decimals);
19     public static decimal Subtract(decimal d1, decimal d2);
20     public string ToString(IFormatProvider provider);
21     public string ToString(string format, IFormatProvider provider);
22     public override string ToString();
23     public string ToString(string format);
24     public static decimal Truncate(decimal d);
25 }
26
27 // Namespace: System.Text, Library: BCL
28 public abstract class Decoder
29 {
30     protected Decoder();
31     public abstract int GetCharCount(byte[] bytes, int index, int count);
32     public abstract int GetChars(byte[] bytes, int byteIndex, int byteCount,
33         char[] chars, int charIndex);
34 }
35
36 // Namespace: System, Library: BCL
37 public abstract class Delegate: ICloneable
38 {
39     public virtual object Clone();
40     public static Delegate Combine(Delegate a, Delegate b);
41     public static Delegate Combine(Delegate[] delegates);
42     public override bool Equals(object obj);
43     public override int GetHashCode();
44     public virtual Delegate[] GetInvocationList();

```

```

1     public static bool operator ==(Delegate d1, Delegate d2);
2     public static bool operator !=(Delegate d1, Delegate d2);
3     public static Delegate Remove(Delegate source, Delegate value);
4     public static Delegate RemoveAll(Delegate source, Delegate value);
5     public object Target { get; }
6 }
7
8 // Namespace: System.Collections, Library: BCL
9 public struct DictionaryEntry
10 {
11     public DictionaryEntry(object key, object value);
12     public object Key { get; set; }
13     public object Value { get; set; }
14 }
15
16 // Namespace: System.IO, Library: BCL
17 public sealed class Directory
18 {
19     public static void Delete(string path);
20     public static void Delete(string path, bool recursive);
21     public static bool Exists(string path);
22     public static DateTime GetCreationTime(string path);
23     public static string GetCurrentDirectory();
24     public static string[] GetDirectories(string path);
25     public static string[] GetDirectories(string path, string searchPattern);
26     public static string GetDirectoryRoot(string path);
27     public static string[] GetFiles(string path);
28     public static string[] GetFiles(string path, string searchPattern);
29     public static string[] GetFileSystemEntries(string path);
30     public static string[] GetFileSystemEntries(string path, string
31         searchPattern);
32     public static DateTime GetLastAccessTime(string path);
33     public static DateTime GetLastWriteTime(string path);
34     public static void Move(string sourceDirName, string destDirName);
35     public static void SetCreationTime(string path, DateTime creationTime);
36     public static void SetCurrentDirectory(string path);
37     public static void SetLastAccessTime(string path, DateTime lastAccessTime);
38     public static void SetLastWriteTime(string path, DateTime lastWriteTime);
39 }
40
41 // Namespace: System.IO, Library: BCL
42 public class DirectoryNotFoundException: IOException
43 {
44     public DirectoryNotFoundException();

```

```

1      public DirectoryNotFoundException(string message);
2      public DirectoryNotFoundException(string message, Exception innerException);
3  }
4
5  // Namespace: System, Library: BCL
6  public class DivideByZeroException: ArithmeticException
7  {
8      public DivideByZeroException();
9      public DivideByZeroException(string message);
10     public DivideByZeroException(string message, Exception innerException);
11 }
12
13 // Namespace: System, Library: ExtendedNumerics
14 public struct Double: IComparable, IFormattable
15 {
16     public const double Epsilon = 4.9406564584124654e-324;
17     public const double MaxValue = 1.7976931348623157e+308;
18     public const double MinValue = -1.7976931348623157e+308;
19     public const double NaN = (double)0.0 / (double)0.0;
20     public const double NegativeInfinity = (double)-1.0 / (double)(0.0);
21     public const double PositiveInfinity = (double)1.0 / (double)(0.0);
22     public int CompareTo(object value);
23     public override bool Equals(object obj);
24     public override int GetHashCode();
25     public static bool IsInfinity(double d);
26     public static bool IsNaN(double d);
27     public static bool IsNegativeInfinity(double d);
28     public static bool IsPositiveInfinity(double d);
29     public static double Parse(string s);
30     public static double Parse(string s, NumberStyles style);
31     public static double Parse(string s, IFormatProvider provider);
32     public static double Parse(string s, NumberStyles style, IFormatProvider
33         provider);
34     public string ToString(IFormatProvider provider);
35     public string ToString(string format, IFormatProvider provider);
36     public override string ToString();
37     public string ToString(string format);
38 }
39
40 // Namespace: System, Library: BCL
41 public class DuplicateWaitObjectException: ArgumentException
42 {
43     public DuplicateWaitObjectException();
44     public DuplicateWaitObjectException(string parameterName);

```



```

1      public DuplicateWaitObjectException(string parameterName, string message);
2  }
3
4  // Namespace: System.Text, Library: BCL
5  public abstract class Encoder
6  {
7      protected Encoder();
8      public abstract int GetByteCount(char[] chars, int index, int count, bool
9          flush);
10     public abstract int GetBytes(char[] chars, int charIndex, int charCount,
11         byte[] bytes, int byteIndex, bool flush);
12 }
13
14 // Namespace: System.Text, Library: BCL
15 public abstract class Encoding
16 {
17     protected Encoding();
18     public static byte[] Convert(Encoding srcEncoding, Encoding dstEncoding,
19         byte[] bytes);
20     public static byte[] Convert(Encoding srcEncoding, Encoding dstEncoding,
21         byte[] bytes, int index, int count);
22     public override bool Equals(object value);
23     public abstract int GetByteCount(char[] chars, int index, int count);
24     public virtual int GetByteCount(string s);
25     public virtual int GetByteCount(char[] chars);
26     public virtual int GetBytes(string s, int charIndex, int charCount, byte[]
27         bytes, int byteIndex);
28     public virtual byte[] GetBytes(string s);
29     public abstract int GetBytes(char[] chars, int charIndex, int charCount,
30         byte[] bytes, int byteIndex);
31     public virtual byte[] GetBytes(char[] chars, int index, int count);
32     public virtual byte[] GetBytes(char[] chars);
33     public abstract int GetCharCount(byte[] bytes, int index, int count);
34     public virtual int GetCharCount(byte[] bytes);
35     public abstract int GetChars(byte[] bytes, int byteIndex, int byteCount,
36         char[] chars, int charIndex);
37     public virtual char[] GetChars(byte[] bytes, int index, int count);
38     public virtual char[] GetChars(byte[] bytes);
39     public virtual Decoder GetDecoder();
40     public virtual Encoder GetEncoder();
41     public override int GetHashCode();
42     public abstract int GetMaxByteCount(int charCount);
43     public abstract int GetMaxCharCount(int byteCount);
44     public virtual byte[] GetPreamble();

```

```

1      public virtual string GetString(byte[] bytes, int index, int count);
2      public virtual string GetString(byte[] bytes);
3      public static Encoding ASCII { get; }
4      public static Encoding BigEndianUnicode { get; }
5      public static Encoding Default { get; }
6      public static Encoding Unicode { get; }
7      public static Encoding UTF8 { get; }
8  }
9
10     // Namespace: System.IO, Library: BCL
11     public class EndOfStreamException: IOException
12     {
13         public EndOfStreamException();
14         public EndOfStreamException(string message);
15         public EndOfStreamException(string message, Exception innerException);
16     }
17
18     // Namespace: System, Library: BCL
19     public abstract class Enum: ValueType, IComparable, IFormattable
20     {
21         public int CompareTo(object target);
22         public override bool Equals(object obj);
23         public static string Format(Type enumType, object value, string format);
24         public override int GetHashCode();
25         public static string GetName(Type enumType, object value);
26         public static string[] GetNames(Type enumType);
27         public static Type GetUnderlyingType(Type enumType);
28         public static Array GetValues(Type enumType);
29         public static bool IsDefined(Type enumType, object value);
30         public static object Parse(Type enumType, string value);
31         public static object Parse(Type enumType, string value, bool ignoreCase);
32         public static object ToObject(Type enumType, object value);
33         public static object ToObject(Type enumType, sbyte value);
34         public static object ToObject(Type enumType, short value);
35         public static object ToObject(Type enumType, int value);
36         public static object ToObject(Type enumType, byte value);
37         public static object ToObject(Type enumType, ushort value);
38         public static object ToObject(Type enumType, uint value);
39         public static object ToObject(Type enumType, long value);
40         public static object ToObject(Type enumType, ulong value);
41         public string ToString(IFormatProvider provider);
42         public string ToString(string format, IFormatProvider provider);
43         public override string ToString();
44         public string ToString(string format);

```

```

1  }
2
3  // Namespace: System, Library: BCL
4  public sealed class Environment
5  {
6      public static void Exit(int exitCode);
7      public static string[] GetCommandLineArgs();
8      public static string GetEnvironmentVariable(string variable);
9      public static IDictionary GetEnvironmentVariables();
10     public static string CommandLine { get; }
11     public static int ExitCode { get; set; }
12     public bool HasShutdownStarted { get; }
13     public static string NewLine { get; }
14     public static string StackTrace { get; }
15     public static int TickCount { get; }
16     public static Version Version { get; }
17 }
18
19 // Namespace: System.Security.Permissions, Library: BCL
20 public sealed class EnvironmentPermission: CodeAccessPermission
21 {
22     public EnvironmentPermission(PermissionState state);
23     public EnvironmentPermission(EnvironmentPermissionAccess flag, string
24         pathList);
25     public override IPermission Copy();
26     public override void FromXml(SecurityElement esd);
27     public override IPermission Intersect(IPermission target);
28     public override bool IsSubsetOf(IPermission target);
29     public override SecurityElement ToXml();
30     public override IPermission Union(IPermission other);
31 }
32
33 // Namespace: System.Security.Permissions, Library: BCL
34 public enum EnvironmentPermissionAccess
35 {
36     AllAccess = Read | Write,
37     NoAccess = 0x0,
38     Read = 0x1,
39     Write = 0x2,
40 }
41
42 // Namespace: System.Security.Permissions, Library: BCL
43 public sealed class EnvironmentPermissionAttribute: CodeAccessSecurityAttribute
44 {

```

C# LANGUAGE SPECIFICATION

```
1      public EnvironmentPermissionAttribute(SecurityAction action);
2      public override IPermission CreatePermission();
3      public string All { get; }
4      public string Read { get; set; }
5      public string Write { get; set; }
6  }
7
8  // Namespace: System, Library: BCL
9  public class EventArgs
10 {
11     public EventArgs();
12     public static readonly EventArgs Empty;
13 }
14
15 // Namespace: System, Library: BCL
16 public delegate void EventHandler(object sender, EventArgs e);
17
18 // Namespace: System, Library: BCL
19 public class Exception
20 {
21     public Exception();
22     public Exception(string message);
23     public Exception(string message, Exception innerException);
24     public virtual Exception GetBaseException();
25     public override string ToString();
26     public Exception InnerException { get; }
27     public virtual string Message { get; }
28     public virtual string StackTrace { get; }
29 }
30
31 // Namespace: System, Library: BCL
32 public sealed class ExecutionEngineException : SystemException
33 {
34     public ExecutionEngineException();
35     public ExecutionEngineException(string message);
36     public ExecutionEngineException(string message, Exception innerException);
37 }
38
39 // Namespace: System.IO, Library: BCL
40 public sealed class File
41 {
42     public static StreamWriter AppendText(string path);
43     public static void Copy(string sourceFileName, string destFileName);
44     public static void Copy(string sourceFileName, string destFileName, bool
```

```

1      overwrite);
2      public static FileStream Create(string path);
3      public static FileStream Create(string path, int bufferSize);
4      public static StreamWriter CreateText(string path);
5      public static void Delete(string path);
6      public static bool Exists(string path);
7      public static DateTime GetCreationTime(string path);
8      public static DateTime GetLastAccessTime(string path);
9      public static DateTime GetLastWriteTime(string path);
10     public static void Move(string sourceFileName, string destFileName);
11     public static FileStream Open(string path, FileMode mode);
12     public static FileStream Open(string path, FileMode mode, FileAccess
13         access);
14     public static FileStream Open(string path, FileMode mode, FileAccess
15         access, FileShare share);
16     public static FileStream OpenRead(string path);
17     public static StreamReader OpenText(string path);
18     public static FileStream OpenWrite(string path);
19     public static void SetCreationTime(string path, DateTime creationTime);
20     public static void SetLastAccessTime(string path, DateTime lastAccessTime);
21     public static void SetLastWriteTime(string path, DateTime lastWriteTime);
22 }
23
24 // Namespace: System.IO, Library: BCL
25 public enum FileAccess
26 {
27     Read = 0x1,
28     Readwrite = Read | Write,
29     Write = 0x2,
30 }
31
32 // Namespace: System.Security.Permissions, Library: BCL
33 public sealed class FileIOPermission: CodeAccessPermission
34 {
35     public FileIOPermission(PermissionState state);
36     public FileIOPermission(FileIOPermissionAccess access, string path);
37     public override IPermission Copy();
38     public override void FromXml(SecurityElement esd);
39     public override IPermission Intersect(IPermission target);
40     public override bool IsSubsetOf(IPermission target);
41     public override SecurityElement ToXml();
42     public override IPermission Union(IPermission other);
43 }
44

```

C# LANGUAGE SPECIFICATION

```
1  // Namespace: System.Security.Permissions, Library: BCL
2  public enum FileIOPermissionAccess
3  {
4      AllAccess = Read | Write | Append | PathDiscovery,
5      Append = 0x4,
6      NoAccess = 0x0,
7      PathDiscovery = 0x8,
8      Read = 0x1,
9      Write = 0x2,
10 }
11
12 // Namespace: System.Security.Permissions, Library: BCL
13 public sealed class FileIOPermissionAttribute: CodeAccessSecurityAttribute
14 {
15     public FileIOPermissionAttribute(SecurityAction action);
16     public override IPermission CreatePermission();
17     public string All { get; set; }
18     public string Append { get; set; }
19     public string PathDiscovery { get; set; }
20     public string Read { get; set; }
21     public string Write { get; set; }
22 }
23
24 // Namespace: System.IO, Library: BCL
25 public class FileLoadException: IOException
26 {
27     public FileLoadException();
28     public FileLoadException(string message);
29     public FileLoadException(string message, Exception inner);
30     public FileLoadException(string message, string fileName);
31     public FileLoadException(string message, string fileName, Exception inner);
32     public override string ToString();
33     public string FileName { get; }
34     public override string Message { get; }
35 }
36
37 // Namespace: System.IO, Library: BCL
38 public enum FileMode
39 {
40     Append = 6,
41     Create = 2,
42     CreateNew = 1,
43     Open = 3,
44     OpenOrCreate = 4,
```

```

1      Truncate = 5,
2  }
3
4  // Namespace: System.IO, Library: BCL
5  public class FileNotFoundException: IOException
6  {
7      public FileNotFoundException();
8      public FileNotFoundException(string message);
9      public FileNotFoundException(string message, Exception innerException);
10     public FileNotFoundException(string message, string fileName);
11     public FileNotFoundException(string message, string fileName, Exception
12         innerException);
13     public override string ToString();
14     public string FileName { get; }
15     public override string Message { get; }
16 }
17
18 // Namespace: System.IO, Library: BCL
19 public enum FileShare
20 {
21     None = 0x0,
22     Read = 0x1,
23     ReadWrite = Read | Write,
24     Write = 0x2,
25 }
26
27 // Namespace: System.IO, Library: BCL
28 public class FileStream: Stream
29 {
30     public FileStream(string path, FileMode mode);
31     public FileStream(string path, FileMode mode, FileAccess access);
32     public FileStream(string path, FileMode mode, FileAccess access, FileShare
33         share);
34     public FileStream(string path, FileMode mode, FileAccess access, FileShare
35         share, int bufferSize);
36     public FileStream(string path, FileMode mode, FileAccess access, FileShare
37         share, int bufferSize, bool useAsync);
38     public override IAsyncResult BeginRead(byte[] array, int offset, int
39         numBytes, AsyncCallback userCallback, object stateObject);
40     public override IAsyncResult BeginWrite(byte[] array, int offset, int
41         numBytes, AsyncCallback userCallback, object stateObject);
42     public override void Close();
43     protected virtual void Dispose(bool disposing);
44     public override int EndRead(IAsyncResult asyncResult);

```

```

1      public override void EndWrite(IAsyncResult asyncResult);
2      ~FileStream();
3      public override void Flush();
4      public override int Read(byte[] array, int offset, int count);
5      public override int ReadByte();
6      public override long Seek(long offset, SeekOrigin origin);
7      public override void SetLength(long value);
8      public override void Write(byte[] array, int offset, int count);
9      public override void WriteByte(byte value);
10     public override bool CanRead { get; }
11     public override bool CanSeek { get; }
12     public override bool CanWrite { get; }
13     public virtual bool IsAsync { get; }
14     public override long Length { get; }
15     public override long Position { get; set; }
16 }
17
18 // Namespace: System, Library: BCL
19 public class FlagsAttribute: Attribute
20 {
21     public FlagsAttribute();
22 }
23
24 // Namespace: System, Library: BCL
25 public class FormatException: SystemException
26 {
27     public FormatException();
28     public FormatException(string message);
29     public FormatException(string message, Exception innerException);
30 }
31
32 // Namespace: System, Library: BCL
33 public sealed class GC
34 {
35     public static void KeepAlive(object obj);
36     public static void ReRegisterForFinalize(object obj);
37     public static void SuppressFinalize(object obj);
38     public static void WaitForPendingFinalizers();
39 }
40
41 // Namespace: System.Collections, Library: BCL
42 public class Hashtable: ICloneable, ICollection, IDictionary, IEnumerable
43 {
44     public Hashtable();

```



```

1      public Hashtable(int capacity);
2      public Hashtable(IHashCodeProvider hcp, IComparer comparer);
3      public Hashtable(int capacity, IHashCodeProvider hcp, IComparer comparer);
4      public Hashtable(IDictionary d);
5      public Hashtable(IDictionary d, IHashCodeProvider hcp, IComparer comparer);
6      public virtual void Add(object key, object value);
7      public virtual void Clear();
8      public virtual object Clone();
9      public virtual bool Contains(object key);
10     public virtual bool ContainsKey(object key);
11     public virtual bool ContainsValue(object value);
12     public virtual void CopyTo(Array array, int arrayIndex);
13     public virtual IDictionaryEnumerator GetEnumerator();
14     protected virtual int GetHash(object key);
15     protected virtual bool KeyEquals(object item, object key);
16     public virtual void Remove(object key);
17     public static Hashtable Synchronized(Hashtable table);
18     IEnumerator IEnumerable.GetEnumerator();
19     int ICollection.Count { get; }
20     public virtual int Count { get; }
21     bool IDictionary.IsFixedSize { get; }
22     public virtual bool IsFixedSize { get; }
23     bool IDictionary.IsReadOnly { get; }
24     public virtual bool IsReadOnly { get; }
25     bool ICollection.IsSynchronized { get; }
26     public virtual bool IsSynchronized { get; }
27     public virtual object this[object key] { get; set; }
28     ICollection IDictionary.Keys { get; }
29     public virtual ICollection Keys { get; }
30     object ICollection.SyncRoot { get; }
31     public virtual object SyncRoot { get; }
32     ICollection IDictionary.Values { get; }
33     public virtual ICollection Values { get; }
34 }
35
36 // Namespace: System, Library: BCL
37 public interface IAsyncResult
38 {
39     object AsyncState { get; }
40     WaitHandle AsyncWaitHandle { get; }
41     bool CompletedSynchronously { get; }
42     bool IsCompleted { get; }
43 }
44

```

```
1  // Namespace: System, Library: BCL
2  public interface ICloneable
3  {
4      object Clone();
5  }
6
7  // Namespace: System.Collections, Library: BCL
8  public interface ICollection: IEnumerable
9  {
10     void CopyTo(Array array, int index);
11     int Count { get; }
12     bool IsSynchronized { get; }
13     object SyncRoot { get; }
14 }
15
16 // Namespace: System, Library: BCL
17 public interface IComparable
18 {
19     int CompareTo(object obj);
20 }
21
22 // Namespace: System.Collections, Library: BCL
23 public interface IComparer
24 {
25     int Compare(object x, object y);
26 }
27
28 // Namespace: System.Collections, Library: BCL
29 public interface IDictionary: ICollection, IEnumerable
30 {
31     void Add(object key, object value);
32     void Clear();
33     bool Contains(object key);
34     IDictionaryEnumerator GetEnumerator();
35     void Remove(object key);
36     bool IsFixedSize { get; }
37     bool IsReadOnly { get; }
38     object this[object key] { get; set; }
39     ICollection Keys { get; }
40     ICollection Values { get; }
41 }
42
43 // Namespace: System.Collections, Library: BCL
44 public interface IDictionaryEnumerator: IEnumerator
```

```

1  {
2      DictionaryEntry Entry { get; }
3      object Key { get; }
4      object Value { get; }
5  }
6
7  // Namespace: System, Library: BCL
8  public interface IDisposable
9  {
10     void Dispose();
11 }
12
13 // Namespace: System.Collections, Library: BCL
14 public interface IEnumerable
15 {
16     IEnumerator GetEnumerator();
17 }
18
19 // Namespace: System.Collections, Library: BCL
20 public interface IEnumerator
21 {
22     bool MoveNext();
23     void Reset();
24     object Current { get; }
25 }
26
27 // Namespace: System, Library: BCL
28 public interface IFormatProvider
29 {
30     object GetFormat(Type formatType);
31 }
32
33 // Namespace: System, Library: BCL
34 public interface IFormattable
35 {
36     string ToString(string format, IFormatProvider formatProvider);
37 }
38
39 // Namespace: System.Collections, Library: BCL
40 public interface IHashCodeProvider
41 {
42     int GetHashCode(object obj);
43 }
44

```

```
1  // Namespace: System.Collections, Library: BCL
2  public interface IList: ICollection, IEnumerable
3  {
4      int Add(object value);
5      void Clear();
6      bool Contains(object value);
7      int IndexOf(object value);
8      void Insert(int index, object value);
9      void Remove(object value);
10     void RemoveAt(int index);
11     bool IsFixedSize { get; }
12     bool IsReadOnly { get; }
13     object this[int index] { get; set; }
14 }
15
16 // Namespace: System, Library: BCL
17 public sealed class IndexOutOfRangeException: SystemException
18 {
19     public IndexOutOfRangeException();
20     public IndexOutOfRangeException(string message);
21     public IndexOutOfRangeException(string message, Exception innerException);
22 }
23
24 // Namespace: System, Library: BCL
25 public struct Int16: IComparable, IFormattable
26 {
27     public const short MaxValue = 32767;
28     public const short MinValue = -32768;
29     public int CompareTo(object value);
30     public override bool Equals(object obj);
31     public override int GetHashCode();
32     public static short Parse(string s);
33     public static short Parse(string s, NumberStyles style);
34     public static short Parse(string s, IFormatProvider provider);
35     public static short Parse(string s, NumberStyles style, IFormatProvider
36         provider);
37     public string ToString(IFormatProvider provider);
38     public string ToString(string format, IFormatProvider provider);
39     public override string ToString();
40     public string ToString(string format);
41 }
42
43 // Namespace: System, Library: BCL
44 public struct Int32: IComparable, IFormattable
```

```

1  {
2      public const int MaxValue = 2147483647;
3      public const int MinValue = -2147483648;
4      public int CompareTo(object value);
5      public override bool Equals(object obj);
6      public override int GetHashCode();
7      public static int Parse(string s);
8      public static int Parse(string s, NumberStyles style);
9      public static int Parse(string s, IFormatProvider provider);
10     public static int Parse(string s, NumberStyles style, IFormatProvider
11         provider);
12     public string ToString(IFormatProvider provider);
13     public string ToString(string format, IFormatProvider provider);
14     public override string ToString();
15     public string ToString(string format);
16 }
17
18 // Namespace: System, Library: BCL
19 public struct Int64: IComparable, IFormattable
20 {
21     public const long MaxValue = 9223372036854775807;
22     public const long MinValue = -9223372036854775808;
23     public int CompareTo(object value);
24     public override bool Equals(object obj);
25     public override int GetHashCode();
26     public static long Parse(string s);
27     public static long Parse(string s, NumberStyles style);
28     public static long Parse(string s, IFormatProvider provider);
29     public static long Parse(string s, NumberStyles style, IFormatProvider
30         provider);
31     public string ToString(IFormatProvider provider);
32     public string ToString(string format, IFormatProvider provider);
33     public override string ToString();
34     public string ToString(string format);
35 }
36
37 // Namespace: System.Threading, Library: BCL
38 public sealed class Interlocked
39 {
40     public static int CompareExchange(ref int location1, int value, int
41         comparand);
42     public static float CompareExchange(ref float location1, float value, float
43         comparand);
44     public static object CompareExchange(ref object location1, object value,

```

```
1      object comparand);
2      public static int Decrement(ref int location);
3      public static long Decrement(ref long location);
4      public static int Exchange(ref int location1, int value);
5      public static float Exchange(ref float location1, float value);
6      public static object Exchange(ref object location1, object value);
7      public static int Increment(ref int location);
8      public static long Increment(ref long location);
9  }
10
11  // Namespace: System, Library: BCL
12  public class InvalidCastException: SystemException
13  {
14      public InvalidCastException();
15      public InvalidCastException(string message);
16      public InvalidCastException(string message, Exception innerException);
17  }
18
19  // Namespace: System, Library: BCL
20  public class InvalidOperationException: SystemException
21  {
22      public InvalidOperationException();
23      public InvalidOperationException(string message);
24      public InvalidOperationException(string message, Exception innerException);
25  }
26
27  // Namespace: System, Library: BCL
28  public sealed class InvalidProgramException: SystemException
29  {
30      public InvalidProgramException();
31      public InvalidProgramException(string message);
32      public InvalidProgramException(string message, Exception inner);
33  }
34
35  // Namespace: System.IO, Library: BCL
36  public class IOException: SystemException
37  {
38      public IOException();
39      public IOException(string message);
40      public IOException(string message, Exception innerException);
41  }
42
43  // Namespace: System.Security, Library: BCL
44  public interface IPermission
```

```

1  {
2      IPermission Copy();
3      void Demand();
4      IPermission Intersect(IPermission target);
5      bool IsSubsetOf(IPermission target);
6      IPermission Union(IPermission target);
7  }
8
9  // Namespace: System, Library: BCL
10 public abstract class MarshalByRefObject
11 {
12 }
13
14 // Namespace: System, Library: ExtendedNumerics
15 public sealed class Math
16 {
17     public const double E = 2.71828182845905;
18     public const double PI = 3.14159265358979;
19     public static sbyte Abs(sbyte value);
20     public static short Abs(short value);
21     public static int Abs(int value);
22     public static long Abs(long value);
23     public static float Abs(float value);
24     public static double Abs(double value);
25     public static decimal Abs(decimal value);
26     public static double Acos(double d);
27     public static double Asin(double d);
28     public static double Atan(double d);
29     public static double Atan2(double y, double x);
30     public static long BigMul(int a, int b);
31     public static double Ceiling(double a);
32     public static double Cos(double d);
33     public static double Cosh(double value);
34     public static int DivRem(int a, int b, out int result);
35     public static long DivRem(long a, long b, out long result);
36     public static double Exp(double d);
37     public static double Floor(double d);
38     public static double IEEERemainder(double x, double y);
39     public static double Log(double d);
40     public static double Log(double a, double newBase);
41     public static double Log10(double d);
42     public static sbyte Max(sbyte val1, sbyte val2);
43     public static byte Max(byte val1, byte val2);
44     public static short Max(short val1, short val2);

```

```

1      public static ushort Max(ushort val1, ushort val2);
2      public static int Max(int val1, int val2);
3      public static uint Max(uint val1, uint val2);
4      public static long Max(long val1, long val2);
5      public static ulong Max(ulong val1, ulong val2);
6      public static float Max(float val1, float val2);
7      public static double Max(double val1, double val2);
8      public static decimal Max(decimal val1, decimal val2);
9      public static sbyte Min(sbyte val1, sbyte val2);
10     public static byte Min(byte val1, byte val2);
11     public static short Min(short val1, short val2);
12     public static ushort Min(ushort val1, ushort val2);
13     public static int Min(int val1, int val2);
14     public static uint Min(uint val1, uint val2);
15     public static long Min(long val1, long val2);
16     public static ulong Min(ulong val1, ulong val2);
17     public static float Min(float val1, float val2);
18     public static double Min(double val1, double val2);
19     public static decimal Min(decimal val1, decimal val2);
20     public static double Pow(double x, double y);
21     public static double Round(double a);
22     public static double Round(double value, int digits);
23     public static decimal Round(decimal d);
24     public static int Sign(sbyte value);
25     public static int Sign(short value);
26     public static int Sign(int value);
27     public static int Sign(long value);
28     public static int Sign(float value);
29     public static int Sign(double value);
30     public static int Sign(decimal value);
31     public static double Sin(double a);
32     public static double Sinh(double value);
33     public static double Sqrt(double d);
34     public static double Tan(double a);
35     public static double Tanh(double value);
36 }
37
38 // Namespace: System.IO, Library: BCL
39 public class MemoryStream: Stream
40 {
41     public MemoryStream();
42     public MemoryStream(int capacity);
43     public MemoryStream(byte[] buffer);
44     public MemoryStream(byte[] buffer, bool writable);

```



```

1     public MemoryStream(byte[] buffer, int index, int count);
2     public MemoryStream(byte[] buffer, int index, int count, bool writable);
3     public MemoryStream(byte[] buffer, int index, int count, bool writable,
4         bool publiclyVisible);
5     public override void Close();
6     public override void Flush();
7     public virtual byte[] GetBuffer();
8     public override int Read(byte[] buffer, int offset, int count);
9     public override int ReadByte();
10    public override long Seek(long offset, SeekOrigin loc);
11    public override void SetLength(long value);
12    public virtual byte[] ToArray();
13    public override void Write(byte[] buffer, int offset, int count);
14    public override void WriteByte(byte value);
15    public virtual void WriteTo(Stream stream);
16    public override bool CanRead { get; }
17    public override bool CanSeek { get; }
18    public override bool CanWrite { get; }
19    public virtual int Capacity { get; set; }
20    public override long Length { get; }
21    public override long Position { get; set; }
22 }
23
24 // Namespace: System.Threading, Library: BCL
25 public sealed class Monitor
26 {
27     public static void Enter(object obj);
28     public static void Exit(object obj);
29     public static void Pulse(object obj);
30     public static void PulseAll(object obj);
31     public static bool TryEnter(object obj);
32     public static bool TryEnter(object obj, int millisecondsTimeout);
33     public static bool TryEnter(object obj, TimeSpan timeout);
34     public static bool wait(object obj, int millisecondsTimeout);
35     public static bool wait(object obj, TimeSpan timeout);
36     public static bool wait(object obj);
37 }
38
39 // Namespace: System, Library: ExtendedNumerics
40 public class NotFiniteNumberException: ArithmeticException
41 {
42     public NotFiniteNumberException();
43     public NotFiniteNumberException(double offendingNumber);
44     public NotFiniteNumberException(string message);

```

```

1      public NotFiniteNumberException(string message, double offendingNumber);
2      public NotFiniteNumberException(string message, double offendingNumber,
3          Exception innerException);
4      public double OffendingNumber { get; }
5  }
6
7  // Namespace: System, Library: BCL
8  public class NotSupportedException: SystemException
9  {
10     public NotSupportedException();
11     public NotSupportedException(string message);
12     public NotSupportedException(string message, Exception innerException);
13 }
14
15 // Namespace: System, Library: BCL
16 public class NullReferenceException: SystemException
17 {
18     public NullReferenceException();
19     public NullReferenceException(string message);
20     public NullReferenceException(string message, Exception innerException);
21 }
22
23 // Namespace: System.Globalization, Library: BCL
24 public sealed class NumberFormatInfo: ICloneable, IFormatProvider
25 {
26     public NumberFormatInfo();
27     public object Clone();
28     public object GetFormat(Type formatType);
29     public static NumberFormatInfo ReadOnly(NumberFormatInfo nfi);
30     public int CurrencyDecimalDigits { get; set; }
31     public string CurrencyDecimalSeparator { get; set; }
32     public string CurrencyGroupSeparator { get; set; }
33     public int[] CurrencyGroupSizes { get; set; }
34     public int CurrencyNegativePattern { get; set; }
35     public int CurrencyPositivePattern { get; set; }
36     public string CurrencySymbol { get; set; }
37     public static NumberFormatInfo CurrentInfo { get; }
38     public static NumberFormatInfo InvariantInfo { get; }
39     public bool IsReadOnly { get; }
40     public string NaNSymbol { get; set; }
41     public string NegativeInfinitySymbol { get; set; }
42     public string NegativeSign { get; set; }
43     public int NumberDecimalDigits { get; set; }
44     public string NumberDecimalSeparator { get; set; }

```

```

1      public string NumberGroupSeparator { get; set; }
2      public int[] NumberGroupSizes { get; set; }
3      public int NumberNegativePattern { get; set; }
4      public int PercentDecimalDigits { get; set; }
5      public string PercentDecimalSeparator { get; set; }
6      public string PercentGroupSeparator { get; set; }
7      public int[] PercentGroupSizes { get; set; }
8      public int PercentNegativePattern { get; set; }
9      public int PercentPositivePattern { get; set; }
10     public string PercentSymbol { get; set; }
11     public string PerMilleSymbol { get; set; }
12     public string PositiveInfinitySymbol { get; set; }
13     public string PositiveSign { get; set; }
14 }
15
16 // Namespace: System.Globalization, Library: BCL
17 public enum NumberStyles
18 {
19     AllowCurrencySymbol = 0x100,
20     AllowDecimalPoint = 0x20,
21     AllowExponent = 0x80,
22     AllowHexSpecifier = 0x200,
23     AllowLeadingSign = 0x4,
24     AllowLeadingWhite = 0x1,
25     AllowParentheses = 0x10,
26     AllowThousands = 0x40,
27     AllowTrailingSign = 0x8,
28     AllowTrailingWhite = 0x2,
29     Any = AllowLeadingWhite | AllowTrailingWhite | AllowLeadingSign |
30         AllowTrailingSign | AllowParentheses | AllowDecimalPoint |
31         AllowThousands | AllowExponent | AllowCurrencySymbol,
32     Currency = AllowLeadingWhite | AllowTrailingWhite | AllowLeadingSign |
33         AllowTrailingSign | AllowParentheses | AllowDecimalPoint |
34         AllowThousands | AllowCurrencySymbol,
35     Float = AllowLeadingWhite | AllowTrailingWhite | AllowLeadingSign |
36         AllowDecimalPoint | AllowExponent,
37     HexNumber = AllowLeadingWhite | AllowTrailingWhite | AllowHexSpecifier,
38     Integer = AllowLeadingWhite | AllowTrailingWhite | AllowLeadingSign,
39     None = 0x0,
40     Number = AllowLeadingWhite | AllowTrailingWhite | AllowLeadingSign |
41         AllowTrailingSign | AllowDecimalPoint | AllowThousands,
42 }
43
44 // Namespace: System, Library: BCL

```

```

1  public class Object
2  {
3      public Object();
4      public virtual bool Equals(object obj);
5      public static bool Equals(object objA, object objB);
6      ~Object();
7      public virtual int GetHashCode();
8      public Type GetType();
9      protected object MemberwiseClone();
10     public static bool ReferenceEquals(object objA, object objB);
11     public virtual string ToString();
12 }
13
14 // Namespace: System, Library: BCL
15 public class ObjectDisposedException: InvalidOperationException
16 {
17     public ObjectDisposedException(string objectName);
18     public ObjectDisposedException(string objectName, string message);
19     public override string Message { get; }
20     public string ObjectName { get; }
21 }
22
23 // Namespace: System, Library: BCL
24 public sealed class ObsoleteAttribute: Attribute
25 {
26     public ObsoleteAttribute();
27     public ObsoleteAttribute(string message);
28     public ObsoleteAttribute(string message, bool error);
29     public bool IsError { get; }
30     public string Message { get; }
31 }
32
33 // Namespace: System, Library: BCL
34 public class OutOfMemoryException: SystemException
35 {
36     public OutOfMemoryException();
37     public OutOfMemoryException(string message);
38     public OutOfMemoryException(string message, Exception innerException);
39 }
40
41 // Namespace: System, Library: BCL
42 public class OverflowException: ArithmeticException
43 {
44     public OverflowException();

```

```

1      public OverflowException(string message);
2      public OverflowException(string message, Exception innerException);
3  }
4
5  // Namespace: System.IO, Library: BCL
6  public sealed class Path
7  {
8      public static readonly char AltDirectorySeparatorChar;
9      public static readonly char DirectorySeparatorChar;
10     public static readonly char PathSeparator;
11     public static string ChangeExtension(string path, string extension);
12     public static string Combine(string path1, string path2);
13     public static string GetDirectoryName(string path);
14     public static string GetExtension(string path);
15     public static string GetFileName(string path);
16     public static string GetFileNameWithoutExtension(string path);
17     public static string GetFullPath(string path);
18     public static string GetPathRoot(string path);
19     public static string GetTempFileName();
20     public static string GetTempPath();
21     public static bool HasExtension(string path);
22     public static bool IsPathRooted(string path);
23 }
24
25 // Namespace: System.IO, Library: BCL
26 public class PathTooLongException: IOException
27 {
28     public PathTooLongException();
29     public PathTooLongException(string message);
30     public PathTooLongException(string message, Exception innerException);
31 }
32
33 // Namespace: System.Security, Library: BCL
34 public class PermissionSet: ICollection, IEnumerable
35 {
36     public PermissionSet(PermissionState state);
37     public PermissionSet(PermissionSet permSet);
38     public virtual IPermission AddPermission(IPermission perm);
39     public virtual void Assert();
40     public virtual PermissionSet Copy();
41     public virtual void CopyTo(Array array, int index);
42     public virtual void Demand();
43     public virtual void Deny();
44     public virtual void FromXml(SecurityElement et);

```

```

1      public virtual IEnumerator GetEnumerator();
2      public virtual bool IsSubsetOf(PermissionSet target);
3      public virtual void PermitOnly();
4      public override string ToString();
5      public virtual SecurityElement ToXml();
6      public virtual PermissionSet Union(PermissionSet other);
7      int ICollection.Count { get; }
8      bool ICollection.IsSynchronized { get; }
9      object ICollection.SyncRoot { get; }
10     }
11
12     // Namespace: System.Security.Permissions, Library: BCL
13     public enum PermissionState
14     {
15         None = 0,
16         Unrestricted = 1,
17     }
18
19     // Namespace: System, Library: BCL
20     public class Random
21     {
22         public Random();
23         public Random(int Seed);
24         public virtual int Next(int maxValue);
25         public virtual int Next(int minValue, int maxValue);
26         public virtual int Next();
27         public virtual void NextBytes(byte[] buffer);
28         public virtual double NextDouble();
29     }
30
31     // Namespace: System, Library: BCL
32     public class RankException: SystemException
33     {
34         public RankException();
35         public RankException(string message);
36         public RankException(string message, Exception innerException);
37     }
38
39     // Namespace: System, Library: BCL
40     public struct SByte: IComparable, IFormattable
41     {
42         public const sbyte MaxValue = 127;
43         public const sbyte MinValue = -128;
44         public int CompareTo(object obj);

```

```

1      public override bool Equals(object obj);
2      public override int GetHashCode();
3      public static sbyte Parse(string s);
4      public static sbyte Parse(string s, NumberStyles style);
5      public static sbyte Parse(string s, IFormatProvider provider);
6      public static sbyte Parse(string s, NumberStyles style, IFormatProvider
7          provider);
8      public string ToString(IFormatProvider provider);
9      public string ToString(string format, IFormatProvider provider);
10     public override string ToString();
11     public string ToString(string format);
12 }
13
14 // Namespace: System.Security.Permissions, Library: BCL
15 public enum SecurityAction
16 {
17     Assert = 3,
18     Demand = 2,
19     Deny = 4,
20     InheritanceDemand = 7,
21     LinkDemand = 6,
22     PermitOnly = 5,
23     RequestMinimum = 8,
24     RequestOptional = 9,
25     RequestRefuse = 10,
26 }
27
28 // Namespace: System.Security.Permissions, Library: BCL
29 public abstract class SecurityAttribute: Attribute
30 {
31     protected SecurityAttribute();
32     public SecurityAttribute(SecurityAction action);
33     public abstract IPermission CreatePermission();
34     public bool Unrestricted { get; set; }
35 }
36
37 // Namespace: System.Security, Library: BCL
38 public sealed class SecurityElement
39 {
40     public override string ToString();
41 }
42
43 // Namespace: System.Security, Library: BCL
44 public class SecurityException: SystemException

```

```

1  {
2      public SecurityException();
3      public SecurityException(string message);
4      public SecurityException(string message, Exception inner);
5  }
6
7  // Namespace: System.Security.Permissions, Library: BCL
8  public sealed class SecurityPermission: CodeAccessPermission
9  {
10     public SecurityPermission(PermissionState state);
11     public SecurityPermission(SecurityPermissionFlag flag);
12     public override IPermission Copy();
13     public override void FromXml(SecurityElement esd);
14     public override IPermission Intersect(IPermission target);
15     public override bool IsSubsetOf(IPermission target);
16     public override SecurityElement ToXml();
17     public override IPermission Union(IPermission target);
18 }
19
20 // Namespace: System.Security.Permissions, Library: BCL
21 public sealed class SecurityPermissionAttribute: CodeAccessSecurityAttribute
22 {
23     public SecurityPermissionAttribute(SecurityAction action);
24     public override IPermission CreatePermission();
25     public SecurityPermissionFlag Flags { get; set; }
26 }
27
28 // Namespace: System.Security.Permissions, Library: BCL
29 public enum SecurityPermissionFlag
30 {
31     Assertion = 0x1,
32     ControlThread = 0x10,
33     Execution = 0x8,
34     NoFlags = 0x0,
35     SkipVerification = 0x4,
36     UnmanagedCode = 0x2,
37 }
38
39 // Namespace: System.IO, Library: BCL
40 public enum SeekOrigin
41 {
42     Begin = 0,
43     Current = 1,
44     End = 2,

```



```

1  }
2
3  // Namespace: System, Library: ExtendedNumerics
4  public struct Single: IComparable, IFormattable
5  {
6      public const float Epsilon = (float)1.401298E-45;
7      public const float MaxValue = (float)3.402823E+38;
8      public const float MinValue = (float)-3.402823E+38;
9      public const float NaN = (float)0.0 / (float)0.0;
10     public const float NegativeInfinity = (float)-1.0 / (float)0.0;
11     public const float PositiveInfinity = (float)1.0 / (float)0.0;
12     public int CompareTo(object value);
13     public override bool Equals(object obj);
14     public override int GetHashCode();
15     public static bool IsInfinity(float f);
16     public static bool IsNaN(float f);
17     public static bool IsNegativeInfinity(float f);
18     public static bool IsPositiveInfinity(float f);
19     public static float Parse(string s);
20     public static float Parse(string s, NumberStyles style);
21     public static float Parse(string s, IFormatProvider provider);
22     public static float Parse(string s, NumberStyles style, IFormatProvider
23         provider);
24     public string ToString(IFormatProvider provider);
25     public string ToString(string format, IFormatProvider provider);
26     public override string ToString();
27     public string ToString(string format);
28 }
29
30 // Namespace: System, Library: BCL
31 public sealed class StackOverflowException: SystemException
32 {
33     public StackOverflowException();
34     public StackOverflowException(string message);
35     public StackOverflowException(string message, Exception innerException);
36 }
37
38 // Namespace: System.IO, Library: BCL
39 public abstract class Stream: MarshalByRefObject, IDisposable
40 {
41     protected Stream();
42     public static readonly Stream Null;
43     public virtual IAsyncResult BeginRead(byte[] buffer, int offset, int count,
44         AsyncCallback callback, object state);

```

```

1      public virtual IAsyncResult BeginWrite(byte[] buffer, int offset, int
2          count, AsyncCallback callback, object state);
3      public virtual void Close();
4      protected virtual WaitHandle CreateWaitHandle();
5      public virtual int EndRead(IAsyncResult asyncResult);
6      public virtual void EndWrite(IAsyncResult asyncResult);
7      public abstract void Flush();
8      public abstract int Read(byte[] buffer, int offset, int count);
9      public virtual int ReadByte();
10     public abstract long Seek(long offset, SeekOrigin origin);
11     public abstract void SetLength(long value);
12     void IDisposable.Dispose();
13     public abstract void Write(byte[] buffer, int offset, int count);
14     public virtual void WriteByte(byte value);
15     public abstract bool CanRead { get; }
16     public abstract bool CanSeek { get; }
17     public abstract bool CanWrite { get; }
18     public abstract long Length { get; }
19     public abstract long Position { get; set; }
20 }
21
22 // Namespace: System.IO, Library: BCL
23 public class StreamReader: TextReader
24 {
25     public StreamReader(Stream stream);
26     public StreamReader(Stream stream, bool detectEncodingFromByteOrderMarks);
27     public StreamReader(Stream stream, Encoding encoding);
28     public StreamReader(Stream stream, Encoding encoding, bool
29         detectEncodingFromByteOrderMarks);
30     public StreamReader(Stream stream, Encoding encoding, bool
31         detectEncodingFromByteOrderMarks, int bufferSize);
32     public StreamReader(string path);
33     public StreamReader(string path, bool detectEncodingFromByteOrderMarks);
34     public StreamReader(string path, Encoding encoding);
35     public StreamReader(string path, Encoding encoding, bool
36         detectEncodingFromByteOrderMarks);
37     public StreamReader(string path, Encoding encoding, bool
38         detectEncodingFromByteOrderMarks, int bufferSize);
39     public override void Close();
40     public void DiscardBufferedData();
41     protected override void Dispose(bool disposing);
42     public override int Peek();
43     public override int Read(char[] buffer, int index, int count);
44     public override int Read();

```

```

1      public override string ReadLine();
2      public override string ReadToEnd();
3      public virtual Stream BaseStream { get; }
4      public virtual Encoding CurrentEncoding { get; }
5  }
6
7  // Namespace: System.IO, Library: BCL
8  public class StreamWriter: TextWriter
9  {
10     public StreamWriter(Stream stream);
11     public StreamWriter(Stream stream, Encoding encoding);
12     public StreamWriter(Stream stream, Encoding encoding, int bufferSize);
13     public StreamWriter(string path);
14     public StreamWriter(string path, bool append);
15     public StreamWriter(string path, bool append, Encoding encoding);
16     public StreamWriter(string path, bool append, Encoding encoding, int
17         bufferSize);
18     public override void Close();
19     protected override void Dispose(bool disposing);
20     ~StreamWriter();
21     public override void Flush();
22     public override void Write(string value);
23     public override void Write(char[] buffer, int index, int count);
24     public override void Write(char[] buffer);
25     public override void Write(char value);
26     public virtual bool AutoFlush { get; set; }
27     public virtual Stream BaseStream { get; }
28     public override Encoding Encoding { get; }
29 }
30
31 // Namespace: System, Library: BCL
32 public sealed class String: ICloneable, IComparable, IEnumerable
33 {
34     unsafe public String(char* value);
35     unsafe public String(char* value, int startIndex, int length);
36     public String(char[] value, int startIndex, int length);
37     public String(char[] value);
38     public String(char c, int count);
39     public static readonly string Empty;
40     public object Clone();
41     public static int Compare(string strA, string strB);
42     public static int Compare(string strA, string strB, bool ignoreCase);
43     public static int Compare(string strA, int indexA, string strB, int indexB,
44         int length);

```

```

1      public static int Compare(string strA, int indexA, string strB, int indexB,
2          int length, bool ignoreCase);
3      public static int CompareOrdinal(string strA, string strB);
4      public static int CompareOrdinal(string strA, int indexA, string strB, int
5          indexB, int length);
6      public int CompareTo(object value);
7      public static string Concat(object arg0, object arg1);
8      public static string Concat(object arg0, object arg1, object arg2);
9      public static string Concat(params object[] args);
10     public static string Concat(string str0, string str1);
11     public static string Concat(string str0, string str1, string str2);
12     public static string Concat(params string[] values);
13     public static string Copy(string str);
14     public void CopyTo(int sourceIndex, char[] destination, int
15         destinationIndex, int count);
16     public bool Endswith(string value);
17     public override bool Equals(object obj);
18     public static bool Equals(string a, string b);
19     public static string Format(string format, object arg0);
20     public static string Format(string format, object arg0, object arg1);
21     public static string Format(string format, object arg0, object arg1, object
22         arg2);
23     public static string Format(string format, params object[] args);
24     public static string Format(IFormatProvider provider, string format, params
25         object[] args);
26     public CharEnumerator GetEnumerator();
27     public override int GetHashCode();
28     public int IndexOf(char value);
29     public int IndexOf(char value, int startIndex);
30     public int IndexOf(char value, int startIndex, int count);
31     public int IndexOf(string value);
32     public int IndexOf(string value, int startIndex);
33     public int IndexOf(string value, int startIndex, int count);
34     public int IndexOfAny(char[] anyOf);
35     public int IndexOfAny(char[] anyOf, int startIndex);
36     public int IndexOfAny(char[] anyOf, int startIndex, int count);
37     public string Insert(int startIndex, string value);
38     public static string Intern(string str);
39     public static string IsInterned(string str);
40     public static string Join(string separator, string[] value);
41     public static string Join(string separator, string[] value, int startIndex,
42         int count);
43     public int LastIndexOf(char value);
44     public int LastIndexOf(char value, int startIndex);

```

```

1      public int LastIndexOf(char value, int startIndex, int count);
2      public int LastIndexOf(string value);
3      public int LastIndexOf(string value, int startIndex);
4      public int LastIndexOf(string value, int startIndex, int count);
5      public int LastIndexOfAny(char[] anyOf);
6      public int LastIndexOfAny(char[] anyOf, int startIndex);
7      public int LastIndexOfAny(char[] anyOf, int startIndex, int count);
8      public static bool operator ==(String a, String b);
9      public static bool operator !=(String a, String b);
10     public string PadLeft(int totalWidth);
11     public string PadLeft(int totalWidth, char paddingChar);
12     public string PadRight(int totalWidth);
13     public string PadRight(int totalWidth, char paddingChar);
14     public string Remove(int startIndex, int count);
15     public string Replace(char oldChar, char newChar);
16     public string Replace(string oldValue, string newValue);
17     public string[] Split(params char[] separator);
18     public string[] Split(char[] separator, int count);
19     public bool StartsWith(string value);
20     public string Substring(int startIndex);
21     public string Substring(int startIndex, int length);
22     IEnumerator IEnumerable.GetEnumerator();
23     public char[] ToCharArray();
24     public char[] ToCharArray(int startIndex, int length);
25     public string ToLower();
26     public string ToString(IFormatProvider provider);
27     public override string ToString();
28     public string ToUpper();
29     public string Trim(params char[] trimChars);
30     public string Trim();
31     public string TrimEnd(params char[] trimChars);
32     public string TrimStart(params char[] trimChars);
33     public char this[int index] { get; }
34     public int Length { get; }
35 }
36
37 // Namespace: System.Text, Library: BCL
38 public sealed class StringBuilder
39 {
40     public StringBuilder();
41     public StringBuilder(int capacity);
42     public StringBuilder(string value);
43     public StringBuilder Append(char value, int repeatCount);
44     public StringBuilder Append(char[] value, int startIndex, int charCount);

```

```
1    public StringBuilder Append(string value);
2    public StringBuilder Append(string value, int startIndex, int count);
3    public StringBuilder Append(bool value);
4    public StringBuilder Append(sbyte value);
5    public StringBuilder Append(byte value);
6    public StringBuilder Append(char value);
7    public StringBuilder Append(short value);
8    public StringBuilder Append(int value);
9    public StringBuilder Append(long value);
10   public StringBuilder Append(float value);
11   public StringBuilder Append(double value);
12   public StringBuilder Append(decimal value);
13   public StringBuilder Append(ushort value);
14   public StringBuilder Append(uint value);
15   public StringBuilder Append(ulong value);
16   public StringBuilder Append(object value);
17   public StringBuilder Append(char[] value);
18   public StringBuilder AppendFormat(string format, object arg0);
19   public StringBuilder AppendFormat(string format, object arg0, object arg1);
20   public StringBuilder AppendFormat(string format, object arg0, object arg1,
21       object arg2);
22   public StringBuilder AppendFormat(string format, params object[] args);
23   public StringBuilder AppendFormat(IFormatProvider provider, string format,
24       params object[] args);
25   public int EnsureCapacity(int capacity);
26   public bool Equals(StringBuilder sb);
27   public StringBuilder Insert(int index, string value, int count);
28   public StringBuilder Insert(int index, string value);
29   public StringBuilder Insert(int index, bool value);
30   public StringBuilder Insert(int index, sbyte value);
31   public StringBuilder Insert(int index, byte value);
32   public StringBuilder Insert(int index, short value);
33   public StringBuilder Insert(int index, char value);
34   public StringBuilder Insert(int index, char[] value);
35   public StringBuilder Insert(int index, char[] value, int startIndex, int
36       charCount);
37   public StringBuilder Insert(int index, int value);
38   public StringBuilder Insert(int index, long value);
39   public StringBuilder Insert(int index, float value);
40   public StringBuilder Insert(int index, double value);
41   public StringBuilder Insert(int index, decimal value);
42   public StringBuilder Insert(int index, ushort value);
43   public StringBuilder Insert(int index, uint value);
44   public StringBuilder Insert(int index, ulong value);
```

```

1      public StringBuilder Insert(int index, object value);
2      public StringBuilder Remove(int startIndex, int length);
3      public StringBuilder Replace(string oldValue, string newValue);
4      public StringBuilder Replace(string oldValue, string newValue, int
5          startIndex, int count);
6      public StringBuilder Replace(char oldChar, char newChar);
7      public StringBuilder Replace(char oldChar, char newChar, int startIndex,
8          int count);
9      public override string ToString();
10     public string ToString(int startIndex, int length);
11     public int Capacity { get; set; }
12     public char this[int index] { get; set; }
13     public int Length { get; set; }
14 }
15
16 // Namespace: System.IO, Library: BCL
17 public class StringReader: TextReader
18 {
19     public StringReader(string s);
20     public override void Close();
21     protected override void Dispose(bool disposing);
22     public override int Peek();
23     public override int Read(char[] buffer, int index, int count);
24     public override int Read();
25     public override string ReadLine();
26     public override string ReadToEnd();
27 }
28
29 // Namespace: System.IO, Library: BCL
30 public class StringWriter: TextWriter
31 {
32     public StringWriter();
33     public StringWriter(IFormatProvider formatProvider);
34     public StringWriter(StringBuilder sb);
35     public StringWriter(StringBuilder sb, IFormatProvider formatProvider);
36     public override void Close();
37     protected override void Dispose(bool disposing);
38     public virtual StringBuilder GetStringBuilder();
39     public override string ToString();
40     public override void Write(string value);
41     public override void Write(char[] buffer, int index, int count);
42     public override void Write(char value);
43     public override Encoding Encoding { get; }
44 }

```

```

1
2 // Namespace: System.Threading, Library: BCL
3 public class SynchronizationLockException: SystemException
4 {
5     public SynchronizationLockException();
6     public SynchronizationLockException(string message);
7     public SynchronizationLockException(string message, Exception
8         innerException);
9 }
10
11 // Namespace: System, Library: BCL
12 public class SystemException: Exception
13 {
14     public SystemException();
15     public SystemException(string message);
16     public SystemException(string message, Exception innerException);
17 }
18
19 // Namespace: System.IO, Library: BCL
20 public abstract class TextReader: MarshalByRefObject, IDisposable
21 {
22     protected TextReader();
23     public static readonly TextReader Null;
24     public virtual void Close();
25     protected virtual void Dispose(bool disposing);
26     public virtual int Peek();
27     public virtual int Read(char[] buffer, int index, int count);
28     public virtual int Read();
29     public virtual int ReadBlock(char[] buffer, int index, int count);
30     public virtual string ReadLine();
31     public virtual string ReadToEnd();
32     public static TextReader Synchronized(TextReader reader);
33     void IDisposable.Dispose();
34 }
35
36 // Namespace: System.IO, Library: BCL
37 public abstract class TextWriter: MarshalByRefObject, IDisposable
38 {
39     protected TextWriter();
40     protected TextWriter(IFormatProvider formatProvider);
41     public static readonly TextWriter Null;
42     public virtual void Close();
43     protected virtual void Dispose(bool disposing);
44     public virtual void Flush();

```



```

1      public static TextWriter Synchronized(TextWriter writer);
2      void IDisposable.Dispose();
3      public virtual void Write(string format, params object[] arg);
4      public virtual void Write(string format, object arg0, object arg1, object
5          arg2);
6      public virtual void Write(string format, object arg0, object arg1);
7      public virtual void Write(string format, object arg0);
8      public virtual void Write(object value);
9      public virtual void Write(string value);
10     public virtual void Write(decimal value);
11     public virtual void Write(double value);
12     public virtual void Write(float value);
13     public virtual void Write(ulong value);
14     public virtual void Write(long value);
15     public virtual void Write(uint value);
16     public virtual void Write(int value);
17     public virtual void Write(bool value);
18     public virtual void Write(char[] buffer, int index, int count);
19     public virtual void Write(char[] buffer);
20     public virtual void Write(char value);
21     public virtual void WriteLine(string format, params object[] arg);
22     public virtual void WriteLine(string format, object arg0, object arg1,
23         object arg2);
24     public virtual void WriteLine(string format, object arg0, object arg1);
25     public virtual void WriteLine(string format, object arg0);
26     public virtual void WriteLine(object value);
27     public virtual void WriteLine(string value);
28     public virtual void WriteLine(decimal value);
29     public virtual void WriteLine(double value);
30     public virtual void WriteLine(float value);
31     public virtual void WriteLine(ulong value);
32     public virtual void WriteLine(long value);
33     public virtual void WriteLine(uint value);
34     public virtual void WriteLine(int value);
35     public virtual void WriteLine(bool value);
36     public virtual void WriteLine(char[] buffer, int index, int count);
37     public virtual void WriteLine(char[] buffer);
38     public virtual void WriteLine(char value);
39     public virtual void WriteLine();
40     public abstract Encoding Encoding { get; }
41     public virtual IFormatProvider FormatProvider { get; }
42     public virtual string NewLine { get; set; }
43 }
44

```

```

1  // Namespace: System, Library: BCL
2  public sealed class Thread
3  {
4      public Thread(ThreadStart start);
5      public void Abort(object stateInfo);
6      public void Abort();
7      ~Thread();
8      public void Join();
9      public bool Join(int millisecondsTimeout);
10     public bool Join(TimeSpan timeout);
11     public static void MemoryBarrier ();
12     public static void ResetAbort();
13     public static void Sleep(int millisecondsTimeout);
14     public static void Sleep(TimeSpan timeout);
15     public void Start();
16     public static byte VolatileRead (ref byte address);
17     public static short VolatileRead (ref short address);
18     public static int VolatileRead (ref int address);
19     public static long VolatileRead (ref long address);
20     public static sbyte VolatileRead (ref sbyte address);
21     public static ushort VolatileRead (ref ushort address);
22     public static uint VolatileRead (ref uint address);
23     public static ulong VolatileRead (ref ulong address);
24     public static float VolatileRead (ref float address);
25     public static double VolatileRead (ref double address);
26     public static object VolatileRead (ref object address);
27     public static void VolatileWrite (ref byte address, byte value);
28     public static void VolatileWrite (ref short address, short value);
29     public static void VolatileWrite (ref int address, int value);
30     public static void VolatileWrite (ref long address, long value);
31     public static void VolatileWrite (ref sbyte address, sbyte value);
32     public static void VolatileWrite (ref ushort address, ushort value);
33     public static void VolatileWrite (ref uint address, uint value);
34     public static void VolatileWrite (ref ulong address, ulong value);
35     public static void VolatileWrite (ref float address, float value);
36     public static void VolatileWrite (ref double address, double value);
37     public static void VolatileWrite (ref object address, object value);
38     public static Thread CurrentThread { get; }
39     public bool IsAlive { get; }
40     public bool IsBackground { get; set; }
41     public string Name { get; set; }
42     public ThreadPriority Priority { get; set; }
43     public ThreadState ThreadState { get; }
44 }

```

```

1
2 // Namespace: System.Threading, Library: BCL
3 public sealed class ThreadAbortException: SystemException
4 {
5     public object ExceptionState { get; }
6 }
7
8 // Namespace: System.Threading, Library: BCL
9 public enum ThreadPriority
10 {
11     AboveNormal = 3,
12     BelowNormal = 1,
13     Highest = 4,
14     Lowest = 0,
15     Normal = 2,
16 }
17
18 // Namespace: System.Threading, Library: BCL
19 public delegate void ThreadStart();
20
21 // Namespace: System.Threading, Library: BCL
22 public enum ThreadState
23 {
24     Aborted = 0x100,
25     AbortRequested = 0x80,
26     Background = 0x4,
27     Running = 0x0,
28     Stopped = 0x10,
29     Unstarted = 0x8,
30     waitsleepJoin = 0x20,
31 }
32
33 // Namespace: System.Threading, Library: BCL
34 public class ThreadStateException: SystemException
35 {
36     public ThreadStateException();
37     public ThreadStateException(string message);
38     public ThreadStateException(string message, Exception innerException);
39 }
40
41 // Namespace: System.Threading, Library: BCL
42 public sealed class Timeout
43 {
44     public const int Infinite = -1;

```

```

1  }
2
3  // Namespace: System.Threading, Library: BCL
4  public sealed class Timer: MarshalByRefObject, IDisposable
5  {
6      public Timer(TimerCallback callback, object state, int dueTime, int period);
7      public Timer(TimerCallback callback, object state, TimeSpan dueTime,
8          TimeSpan period);
9      public bool Change(int dueTime, int period);
10     public bool Change(TimeSpan dueTime, TimeSpan period);
11     public void Dispose();
12     public bool Dispose(WaitHandle notifyObject);
13     ~Timer();
14 }
15
16 // Namespace: System.Threading, Library: BCL
17 public delegate void TimerCallback(object state);
18
19 // Namespace: System, Library: BCL
20 public struct TimeSpan: IComparable
21 {
22     public TimeSpan(long ticks);
23     public TimeSpan(int hours, int minutes, int seconds);
24     public TimeSpan(int days, int hours, int minutes, int seconds);
25     public TimeSpan(int days, int hours, int minutes, int seconds, int
26         milliseconds);
27     public static readonly TimeSpan MaxValue;
28     public static readonly TimeSpan MinValue;
29     public const long TicksPerDay = 864000000000;
30     public const long TicksPerHour = 36000000000;
31     public const long TicksPerMillisecond = 10000;
32     public const long TicksPerMinute = 600000000;
33     public const long TicksPerSecond = 10000000;
34     public static readonly TimeSpan Zero;
35     public TimeSpan Add(TimeSpan ts);
36     public static int Compare(TimeSpan t1, TimeSpan t2);
37     public int CompareTo(object value);
38     public TimeSpan Duration();
39     public override bool Equals(object value);
40     public static bool Equals(TimeSpan t1, TimeSpan t2);
41     public static TimeSpan FromDays(double value);
42     public static TimeSpan FromHours(double value);
43     public static TimeSpan FromMilliseconds(double value);
44     public static TimeSpan FromMinutes(double value);

```

```

1      public static TimeSpan FromSeconds(double value);
2      public static TimeSpan FromTicks(long value);
3      public override int GetHashCode();
4      public TimeSpan Negate();
5      public static TimeSpan operator +(TimeSpan t1, TimeSpan t2);
6      public static bool operator ==(TimeSpan t1, TimeSpan t2);
7      public static bool operator >(TimeSpan t1, TimeSpan t2);
8      public static bool operator >=(TimeSpan t1, TimeSpan t2);
9      public static bool operator !=(TimeSpan t1, TimeSpan t2);
10     public static bool operator <(TimeSpan t1, TimeSpan t2);
11     public static bool operator <=(TimeSpan t1, TimeSpan t2);
12     public static TimeSpan operator -(TimeSpan t1, TimeSpan t2);
13     public static TimeSpan operator -(TimeSpan t);
14     public static TimeSpan operator +(TimeSpan t);
15     public static TimeSpan Parse(string s);
16     public TimeSpan Subtract(TimeSpan ts);
17     public override string ToString();
18     public int Days { get; }
19     public int Hours { get; }
20     public int Milliseconds { get; }
21     public int Minutes { get; }
22     public int Seconds { get; }
23     public long Ticks { get; }
24     public double TotalDays { get; }
25     public double TotalHours { get; }
26     public double TotalMilliseconds { get; }
27     public double TotalMinutes { get; }
28     public double TotalSeconds { get; }
29 }
30
31 // Namespace: System, Library: BCL
32 public abstract class Type: Object
33 {
34     public virtual int GetArrayRank();
35     public abstract Type GetElementType();
36     public override int GetHashCode();
37     public virtual bool IsAssignableFrom(Type c);
38     public virtual bool IsInstanceOfType(object o);
39     public virtual bool IsSubclassOf(Type c);
40     public override string ToString();
41     public abstract Type BaseType { get; }
42     public abstract string FullName { get; }
43     public bool IsArray { get; }
44     public bool IsClass { get; }

```

C# LANGUAGE SPECIFICATION

```
1     public bool IsEnum { get; }
2     public bool IsInterface { get; }
3     public bool IsPointer { get; }
4     public bool IsValueType { get; }
5 }
6
7 // Namespace: System, Library: BCL
8 public sealed class TypeInitializationException: SystemException
9 {
10     public string TypeName { get; }
11 }
12
13 // Namespace: System, Library: BCL
14 public struct UInt16: IComparable, IFormattable
15 {
16     public const ushort MaxValue = 65535;
17     public const ushort MinValue = 0;
18     public int CompareTo(object value);
19     public override bool Equals(object obj);
20     public override int GetHashCode();
21     public static ushort Parse(string s);
22     public static ushort Parse(string s, NumberStyles style);
23     public static ushort Parse(string s, IFormatProvider provider);
24     public static ushort Parse(string s, NumberStyles style, IFormatProvider
25         provider);
26     public string ToString(IFormatProvider provider);
27     public string ToString(string format, IFormatProvider provider);
28     public override string ToString();
29     public string ToString(string format);
30 }
31
32 // Namespace: System, Library: BCL
33 public struct UInt32: IComparable, IFormattable
34 {
35     public const uint MaxValue = 4294967295;
36     public const uint MinValue = 0;
37     public int CompareTo(object value);
38     public override bool Equals(object obj);
39     public override int GetHashCode();
40     public static uint Parse(string s);
41     public static uint Parse(string s, NumberStyles style);
42     public static uint Parse(string s, IFormatProvider provider);
43     public static uint Parse(string s, NumberStyles style, IFormatProvider
44         provider);
```

```

1     public string ToString(IFormatProvider provider);
2     public string ToString(string format, IFormatProvider provider);
3     public override string ToString();
4     public string ToString(string format);
5 }
6
7 // Namespace: System, Library: BCL
8 public struct UInt64: IComparable, IFormattable
9 {
10     public const ulong MaxValue = 18446744073709551615;
11     public const ulong MinValue = 0;
12     public int CompareTo(object value);
13     public override bool Equals(object obj);
14     public override int GetHashCode();
15     public static ulong Parse(string s);
16     public static ulong Parse(string s, NumberStyles style);
17     public static ulong Parse(string s, IFormatProvider provider);
18     public static ulong Parse(string s, NumberStyles style, IFormatProvider
19         provider);
20     public string ToString(IFormatProvider provider);
21     public string ToString(string format, IFormatProvider provider);
22     public override string ToString();
23     public string ToString(string format);
24 }
25
26 // Namespace: System, Library: BCL
27 public class UnauthorizedAccessException: SystemException
28 {
29     public UnauthorizedAccessException();
30     public UnauthorizedAccessException(string message);
31     public UnauthorizedAccessException(string message, Exception inner);
32 }
33
34 // Namespace: System.Globalization, Library: BCL
35 public enum UnicodeCategory
36 {
37     ClosePunctuation = 21,
38     ConnectorPunctuation = 18,
39     Control = 14,
40     CurrencySymbol = 26,
41     DashPunctuation = 19,
42     DecimalDigitNumber = 8,
43     EnclosingMark = 7,
44     FinalQuotePunctuation = 23,

```

```

1      Format = 15,
2      InitialQuotePunctuation = 22,
3      LetterNumber = 9,
4      LineSeparator = 12,
5      LowercaseLetter = 1,
6      MathSymbol = 25,
7      ModifierLetter = 3,
8      ModifiersSymbol = 27,
9      NonSpacingMark = 5,
10     OpenPunctuation = 20,
11     OtherLetter = 4,
12     OtherNotAssigned = 29,
13     OtherNumber = 10,
14     OtherPunctuation = 24,
15     Othersymbol = 28,
16     ParagraphSeparator = 13,
17     PrivateUse = 17,
18     SpaceSeparator = 11,
19     SpacingCombiningMark = 6,
20     Surrogate = 16,
21     TitlecaseLetter = 2,
22     UppercaseLetter = 0,
23 }
24
25 // Namespace: System.Text, Library: BCL
26 public class UnicodeEncoding: Encoding
27 {
28     public UnicodeEncoding();
29     public UnicodeEncoding(bool bigEndian, bool byteOrderMark);
30     public override bool Equals(object value);
31     public override int GetByteCount(char[] chars, int index, int count);
32     public override int GetByteCount(string s);
33     public override int GetBytes(string s, int charIndex, int charCount, byte[]
34         bytes, int byteIndex);
35     public override byte[] GetBytes(string s);
36     public override int GetBytes(char[] chars, int charIndex, int charCount,
37         byte[] bytes, int byteIndex);
38     public override int GetCharCount(byte[] bytes, int index, int count);
39     public override int GetChars(byte[] bytes, int byteIndex, int byteCount,
40         char[] chars, int charIndex);
41     public override Decoder GetDecoder();
42     public override int GetHashCode();
43     public override int GetMaxByteCount(int charCount);
44     public override int GetMaxCharCount(int byteCount);

```



```

1      public override byte[] GetPreamble();
2  }
3
4  // Namespace: System.Text, Library: BCL
5  public class UTF8Encoding: Encoding
6  {
7      public UTF8Encoding();
8      public UTF8Encoding(bool encodersShouldEmitUTF8Identifier);
9      public UTF8Encoding(bool encodersShouldEmitUTF8Identifier, bool
10         throwOnInvalidBytes);
11     public override bool Equals(object value);
12     public override int GetByteCount(char[] chars, int index, int count);
13     public override int GetByteCount(string chars);
14     public override int GetBytes(string s, int charIndex, int charCount, byte[]
15         bytes, int byteIndex);
16     public override byte[] GetBytes(string s);
17     public override int GetBytes(char[] chars, int charIndex, int charCount,
18         byte[] bytes, int byteIndex);
19     public override int GetCharCount(byte[] bytes, int index, int count);
20     public override int GetChars(byte[] bytes, int byteIndex, int byteCount,
21         char[] chars, int charIndex);
22     public override Decoder GetDecoder();
23     public override Encoder GetEncoder();
24     public override int GetHashCode();
25     public override int GetMaxByteCount(int charCount);
26     public override int GetMaxCharCount(int byteCount);
27     public override byte[] GetPreamble();
28 }
29
30 // Namespace: System, Library: BCL
31 public abstract class ValueType
32 {
33     protected ValueType();
34     public override bool Equals(object obj);
35     public override int GetHashCode();
36     public override string ToString();
37 }
38
39 // Namespace: System.Security, Library: BCL
40 public class VerificationException: SystemException
41 {
42     public VerificationException();
43     public VerificationException(string message);
44     public VerificationException(string message, Exception innerException);

```

```

1  }
2
3  // Namespace: System, Library: BCL
4  public sealed class Version: ICloneable, IComparable
5  {
6      public Version(int major, int minor, int build, int revision);
7      public Version(int major, int minor, int build);
8      public Version(int major, int minor);
9      public Version(string version);
10     public Version();
11     public object Clone();
12     public int CompareTo(object version);
13     public override bool Equals(object obj);
14     public override int GetHashCode();
15     public static bool operator ==(Version v1, Version v2);
16     public static bool operator >(Version v1, Version v2);
17     public static bool operator >=(Version v1, Version v2);
18     public static bool operator !=(Version v1, Version v2);
19     public static bool operator <(Version v1, Version v2);
20     public static bool operator <=(Version v1, Version v2);
21     public int Build { get; }
22     public int Major { get; }
23     public int Minor { get; }
24     public int Revision { get; }
25 }
26
27 // Namespace: System.Threading, Library: BCL
28 public abstract class WaitHandle: MarshalByRefObject, IDisposable
29 {
30     public WaitHandle();
31     public virtual void Close();
32     protected virtual void Dispose(bool explicitDisposing);
33     ~WaitHandle();
34     void IDisposable.Dispose();
35     public static bool WaitAll(WaitHandle[] waitHandles);
36     public static int WaitAny(WaitHandle[] waitHandles);
37     public virtual bool WaitOne();
38 }
39
40 End of informative text.

```

E. Documentation Comments

This clause is informative.

C# provides a mechanism for programmers to document their code using a special comment syntax that contains XML text. Comments using such syntax are called **documentation comments**. The XML generation tool is called the **documentation generator**. (This generator could be, but need not be, the C# compiler itself.) The output produced by the documentation generator is called the **documentation file**. A documentation file is used as input to a **documentation viewer**; a tool intended to produce some sort of visual display of type information and its associated documentation.

A conforming C# compiler is not required to check the syntax of documentation comments; such comments are simply ordinary comments. A conforming compiler is permitted to do such checking, however.

This specification suggests a set of standard tags to be used in documentation comments. For C# implementations targeting the CLI, it also provides information about the documentation generator and the format of the documentation file. No information is provided about the documentation viewer.

E.1 Introduction

Comments having a special form can be used to direct a tool to produce XML from those comments and the source code elements, which they precede. Such comments are *single-line comments* of the form `///...` or *delimited comments* of the form `/** ... */`. They must immediately precede a user-defined type (such as a class, delegate, or interface) or a member (such as a field, event, property, or method) that they annotate.

Syntax:

```
single-line-doc-comment::
    /// input-charactersopt

delimited-doc-comment::
    /** delimited-comment-charactersopt */
```

In a *single-line-doc-comment*, if there is a *whitespace* character following the `///` characters on each of the *single-line-doc-comments* adjacent to the current *single-line-doc-comment*, then that *whitespace* character is not included in the XML output.

In a *delimited-doc-comment*, if the first non-*whitespace* character on the second line is an *asterisk* and the same pattern of optional *whitespace* characters and an *asterisk* character is repeated at the beginning of each of the lines within the *delimited-doc-comment*, then the characters of the repeated pattern are not included in the XML output. The pattern may include *whitespace* characters after, as well as before, the *asterisk* character.

Example:

```
/**
 * <remarks>Class <c>Point</c> models a point in a two-dimensional
 * plane.</remarks>
 */
public class Point
{
    /// <remarks>method <c>draw</c> renders the point.</remarks>
    void draw() {...}
}
```

The text within documentation comments must be well formed according to the rules of XML (<http://www.w3.org/TR/REC-xml>). If the XML is ill formed, a warning is generated and the documentation file will contain a comment saying that an error was encountered.

Although developers are free to create their own set of tags, a recommended set is defined in §E.2. Some of the recommended tags have special meanings:

- The `<param>` tag is used to describe parameters. If such a tag is used, the documentation generator must verify that the specified parameter exists and that all parameters are described in documentation comments. If such verification fails, the documentation generator issues a warning.
- The `cref` attribute can be attached to any tag to provide a reference to a code element. The documentation generator must verify that this code element exists. If the verification fails, the documentation generator issues a warning. When looking for a name described in a `cref` attribute, the documentation generator must respect namespace visibility according to `using` statements appearing within the source code.
- The `<summary>` tag is intended to be used by a documentation viewer to display additional information about a type or member.

Note carefully that the documentation file does not provide full information about the type and members (for example, it does not contain any type information). To get such information about a type or member, the documentation file must be used in conjunction with reflection on the actual type or member.

E.2 Recommended tags

The documentation generator must accept and process any tag that is valid according to the rules of XML. The following tags provide commonly used functionality in user documentation. (Of course, other tags are possible.)

Tag	Reference	Purpose
<code><c></code>	§E.2.1	Set text in a code-like font
<code><code></code>	§E.2.2	Set one or more lines of source code or program output
<code><example></code>	§E.2.3	Indicate an example
<code><exception></code>	§E.2.4	Identifies the exceptions a method can throw
<code><list></code>	§E.2.5	Create a list or table
<code><para></code>	§E.2.6	Permit structure to be added to text
<code><param></code>	§E.2.7	Describe a parameter for a method or constructor
<code><paramref></code>	§E.2.8	Identify that a word is a parameter name
<code><permission></code>	§E.2.9	Document the security accessibility of a member
<code><remarks></code>	§E.2.10	Describe a type
<code><returns></code>	§E.2.11	Describe the return value of a method
<code><see></code>	§E.2.12	Specify a link
<code><seealso></code>	§E.2.13	Generate a <i>See Also</i> entry
<code><summary></code>	§E.2.14	Describe a member of a type
<code><value></code>	§E.2.15	Describe a property

E.2.1 `<c>`

This tag provides a mechanism to indicate that a fragment of text within a description should be set a special font such as that used for a block of code. (For lines of actual code, use `<code>` (§E.2.2).)

Syntax:

```
<c>text to be set like code</c>
```

Example:

```
///  
/// <remarks>Class <c>Point</c> models a point in a two-dimensional  
/// plane.</remarks>  
public class Point  
{  
    // ...  
}
```

E.2.2 <code>

This tag is used to set one or more lines of source code or program output in some special font. (For small code fragments in narrative, use <c> (§E.2.1).)

Syntax:

```
<code>source code or program output</code>
```

Example:

```
///  
/// <summary>This method changes the point's location by  
/// the given x- and y-offsets.  
/// <example>For example:  
/// <code>  
///     Point p = new Point(3,5);  
///     p.Translate(-1,3);  
/// </code>  
/// results in <c>p</c>'s having the value (2,8).  
/// </example>  
/// </summary>  
public void Translate(int xor, int yor) {  
    X += xor;  
    Y += yor;  
}
```

E.2.3 <example>

This tag allows example code within a comment, to specify how a method or other library member may be used. Ordinarily, this would also involve use of the tag <code> (§E.2.2) as well.

Syntax:

```
<example>description</example>
```

Example:

See <code> (§E.2.2) for an example.

E.2.4 <exception>

This tag provides a way to document the exceptions a method can throw.

Syntax:

```
<exception cref="member">description</exception>
```

where

```
cref="member"
```

The name of a member. The documentation generator checks that the given member exists and translates *member* to the canonical element name in the documentation file.

description

A description of the circumstances in which the exception is thrown.

Example:

```

public class DataBaseOperations
{
    /// <exception cref="MasterFileFormatException"></exception>
    /// <exception cref="MasterFileLockedOpenException"></exception>
    public static void ReadRecord(int flag) {
        if (flag == 1)
            throw new MasterFileFormatException();
        else if (flag == 2)
            throw new MasterFileLockedOpenException();
        // ...
    }
}

```

E.2.5 <list>

This tag is used to create a list or table of items. It may contain a <listheader> block to define the heading row of either a table or definition list. (When defining a table, only an entry for *term* in the heading need be supplied.)

Each item in the list is specified with an <item> block. When creating a definition list, both *term* and *description* must be specified. However, for a table, bulleted list, or numbered list, only *description* need be specified.

Syntax:

```

<list type="bullet" | "number" | "table">
  <listheader>
    <term>term</term>
    <description>description</description>
  </listheader>
  <item>
    <term>term</term>
    <description>description</description>
  </item>
  ...
  <item>
    <term>term</term>
    <description>description</description>
  </item>
</list>

```

where

term

The term to define, whose definition is in *description*.

description

Either an item in a bullet or numbered list, or the definition of a *term*.

Example:

```

1  public class MyClass
2  {
3      /// <remarks>Here is an example of a bulleted list:
4      /// <list type="bullet">
5      /// <item>
6      /// <description>Item 1.</description>
7      /// </item>
8      /// <item>
9      /// <description>Item 2.</description>
10     /// </item>
11     /// </list>
12     /// </remarks>
13     public static void Main () {
14         // ...
15     }
16 }
17

```

E.2.6 <para>

This tag is for use inside other tags, such as <remarks> (§E.2.10) or <returns> (§E.2.11), and permits structure to be added to text.

Syntax:

```
<para>content</para>
```

where

content

The text of the paragraph.

Example:

```

27     /// <summary>This is the entry point of the Point class testing
28     program.
29     /// <para>This program tests each method and operator, and
30     /// is intended to be run after any non-trivial maintenance has
31     /// been performed on the Point class.</para></summary>
32     public static void Main() {
33         // ...
34     }
35

```

E.2.7 <param>

This tag is used to describe a parameter for a method, constructor, or indexer.

Syntax:

```
<param name="name">description</param>
```

where

name

The name of the parameter.

description

A description of the parameter.

Example:

```

45     /// <summary>This method changes the point's location to
46     /// the given coordinates.</summary>
47     /// <param><c>xor</c> is the new x-coordinate.</param>
48     /// <param><c>yor</c> is the new y-coordinate.</param>
49     public void Move(int xor, int yor) {
50         X = xor;
51         Y = yor;
52     }
53

```

E.2.8 <paramref>

This tag is used to indicate that a word is a parameter. The documentation file can be processed to format this parameter in some distinct way.

Syntax:

```
<paramref name="name" />
```

where

name

The name of the parameter.

Example:

```
/// <summary>This constructor initializes the new Point to
/// (<paramref name="xor"/>,<paramref name="yor"/>).</summary>
/// <param><c>xor</c> is the new Point's x-coordinate.</param>
/// <param><c>yor</c> is the new Point's y-coordinate.</param>
public Point(int xor, int yor) {
    X = xor;
    Y = yor;
}
```

E.2.9 <permission>

This tag allows the security accessibility of a member to be documented.

Syntax:

```
<permission cref="member">description</permission>
```

where

cref="*member*"

The name of a member. The documentation generator checks that the given code element exists and translates *member* to the canonical element name in the documentation file.

description

A description of the access to the member.

Example:

```
/// <permission cref="System.Security.PermissionSet">Everyone can
/// access this method.</permission>
public static void Test() {
    // ...
}
```

E.2.10 <remarks>

This tag is used to specify overview information about a type. (Use <summary> (§E.2.14) to describe the members of a type.)

Syntax:

```
<remarks>description</remarks>
```

where

description

The text of the remarks.

Example:

```

1  /// <remarks>Class <c>Point</c> models a point in a two-dimensional
2  plane.</remarks>
3  public class Point
4  {
5      // ...
6  }
7

```

E.2.11 <returns>

This tag is used to describe the return value of a method.

Syntax:

```
<returns>description</returns>
```

where

description

A description of the return value.

Example:

```

16  /// <summary>Report a point's location as a string.</summary>
17  /// <returns>A string representing a point's location, in the form
18  (x,y),
19  /// without any leading, training, or embedded whitespace.</returns>
20  public override string ToString() {
21      return "(" + X + "," + Y + ")";
22  }
23

```

E.2.12 <see>

This tag allows a link to be specified within text. (Use <seealso> (§E.2.13) to indicate text that is to appear in a *See Also* section.)

Syntax:

```
<see cref="member" />
```

where

cref="*member*"

The name of a member. The documentation generator checks that the given code element exists and passes *member* to the element name in the documentation file.

Example:

```

33  /// <summary>This method changes the point's location to
34  /// the given coordinates.</summary>
35  /// <see cref="Translate"/>
36  public void Move(int xor, int yor) {
37      X = xor;
38      Y = yor;
39  }
40
41  /// <summary>This method changes the point's location by
42  /// the given x- and y-offsets.
43  /// </summary>
44  /// <see cref="Move"/>
45  public void Translate(int xor, int yor) {
46      X += xor;
47      Y += yor;
48  }
49

```

E.2.13 <seealso>

This tag allows an entry to be generated for the *See Also* section. (Use <see> (§E.2.12) to specify a link from within text.)

Syntax:

```
<seealso cref="member"/>
```

where

```
cref="member"
```

The name of a member. The documentation generator checks that the given code element exists and passes *member* to the element name in the documentation file.

Example:

```
/// <summary>This method determines whether two Points have the same
/// location.</summary>
/// <seealso cref="operator==" />
/// <seealso cref="operator!=" />
public override bool Equals(object o) {
    // ...
}
```

E.2.14 <summary>

This tag can be used to describe a member for a type. (Use <remarks> (§E.2.10) to describe the type itself.)

Syntax:

```
<summary>description</summary>
```

where

```
description
```

A summary of the member.

Example:

```
/// <summary>This constructor initializes the new Point to
(0,0).</summary>
public Point() : this(0,0) {
}
```

E.2.15 <value>

This tag allows a property to be described.

Syntax:

```
<value>property description</value>
```

where

```
property description
```

A description for the property.

Example:

```
/// <value>Property <c>X</c> represents the point's x-coordinate.</value>
public int X
{
    get { return x; }
    set { x = value; }
}
```

E.3 Processing the documentation file

The following information is intended for C# implementations targeting the CLI.

The documentation generator generates an ID string for each element in the source code that is tagged with a documentation comment. This ID string uniquely identifies a source element. A documentation viewer can use an ID string to identify the corresponding metadata/reflection item to which the documentation applies.

1 The documentation file is not a hierarchical representation of the source code; rather, it is a flat list with a
 2 generated ID string for each element.

3 E.3.1 ID string format

4 The documentation generator observes the following rules when it generates the ID strings:

- 5 • No white space is placed in the string.
- 6 • The first part of the string identifies the kind of member being documented, via a single character
 7 followed by a colon. The following kinds of members are defined:

Character	Description
E	Event
F	Field
M	Method (including constructors, destructors, and operators)
N	Namespace
P	Property (including indexers)
T	Type (such as class, delegate, enum, interface, and struct)
!	Error string; the rest of the string provides information about the error. For example, the documentation generator generates error information for links that cannot be resolved.

- 8 • The second part of the string is the fully qualified name of the element, starting at the root of the
 9 namespace. The name of the element, its enclosing type(s), and namespace are separated by periods. If the
 10 name of the item itself has periods, they are replaced by the NUMBER SIGN # (U+000D). (It is assumed that
 11 no element has this character in its name.)
- 12 • For methods and properties with arguments, the argument list follows, enclosed in parentheses. For
 13 those without arguments, the parentheses are omitted. The arguments are separated by commas. The
 14 encoding of each argument is the same as a CLI signature, as follows: Arguments are represented by their
 15 fully qualified name. For example, `int` becomes `System.Int32`, `string` becomes `System.String`,
 16 `object` becomes `System.Object`, and so on. Arguments having the `out` or `ref` modifier have a '@'
 17 following their type name. Arguments passed by value or via `params` have no special notation. Arguments
 18 that are arrays are represented as `[lowerbound:size, ..., lowerbound:size]` where the number of commas is the
 19 rank – 1, and the lower bounds and size of each dimension, if known, are represented in decimal. If a lower
 20 bound or size is not specified, it is omitted. If the lower bound and size for a particular dimension are
 21 omitted, the ':' is omitted as well. Jagged arrays are represented by one "[]" per level. Arguments that have
 22 pointer types other than void are represented using a '*' following the type name. A void pointer is
 23 represented using a type name of `"System.Void"`.

24 E.3.2 ID string examples

25 The following examples each show a fragment of C# code, along with the ID string produced from each
 26 source element capable of having a documentation comment:

- 27 • Types are represented using their fully qualified name.

```

28     enum Color {Red, Blue, Green};
29     namespace Acme
30     {
31     interface IProcess { /* ... */ }
32     struct ValueType { /* ... */ }
```

```

1      class widget: Iprocess
2      {
3          public class NestedClass { /* ... */ }
4          public interface IMenuItem { /* ... */ }
5          public delegate void Del(int i);
6          public enum Direction {North, South, East, West};
7      }
8  }
9
10     "T:Color"
11     "T:Acme.IProcess"
12     "T:Acme.ValueType"
13     "T:Acme.Widget"
14     "T:Acme.Widget.NestedClass"
15     "T:Acme.Widget.IMenuItem"
16     "T:Acme.Widget.Del"
17     "T:Acme.Widget.Direction"

```

- Fields are represented by their fully qualified name.

```

18     namespace Acme
19     {
20         struct ValueType
21         {
22             private int total;
23         }
24
25         class widget: Iprocess
26         {
27             public class NestedClass
28             {
29                 private int value;
30
31                 private string message;
32                 private static Color defaultColor;
33                 private const double PI = 3.14159;
34                 protected readonly double monthlyAverage;
35                 private long[] array1;
36                 private widget[,] array2;
37                 private unsafe int *pCount;
38                 private unsafe float **ppValues;
39             }
40
41             "F:Acme.ValueType.total"
42             "F:Acme.Widget.NestedClass.value"
43             "F:Acme.Widget.message"
44             "F:Acme.Widget.defaultColor"
45             "F:Acme.Widget.PI"
46             "F:Acme.Widget.monthlyAverage"
47             "F:Acme.Widget.array1"
48             "F:Acme.Widget.array2"
49             "F:Acme.Widget.pCount"
50             "F:Acme.Widget.ppValues"

```

- Constructors.

```

51     namespace Acme
52     {
53         class widget: Iprocess
54         {
55             static widget() { /* ... */ }
56             public widget() { /* ... */ }
57             public widget(string s) { /* ... */ }
58         }
59     }
60
61     "M:Acme.Widget.#cctor"
62     "M:Acme.Widget.#ctor"
63     "M:Acme.Widget.#ctor(System.String)"

```

1 • Destructors.

```

2     namespace Acme
3     {
4     class widget: Iprocess
5     {
6         ~widget() { /* ... */ }
7     }
8     }
9     "M:Acme.Widget.Finalize"
```

10 • Methods.

```

11     namespace Acme
12     {
13     struct valueType
14     {
15         public void M(int i) { /* ... */ }
16     }
17     class widget: Iprocess
18     {
19         public class NestedClass
20         {
21             public void M(int i) { /* ... */ }
22         }
23         public static void M0() { /* ... */ }
24         public void M1(char c, out float f, ref valueType v) { /* ... */ }
25         public void M2(short[] x1, int[,] x2, long[][] x3) { /* ... */ }
26         public void M3(long[][] x3, widget[,] x4) { /* ... */ }
27         public unsafe void M4(char *pc, Color **pf) { /* ... */ }
28         public unsafe void M5(void *pv, double *[,] pd) { /* ... */ }
29         public void M6(int i, params object[] args) { /* ... */ }
30     }
31     }
32     "M:Acme.ValueType.M(System.Int32)"
33     "M:Acme.Widget.NestedClass.M(System.Int32)"
34     "M:Acme.Widget.M0"
35     "M:Acme.Widget.M1(System.Char,System.Single@,Acme.ValueType@)"
36     "M:Acme.Widget.M2(System.Int16[],System.Int32[0:,0:],System.Int64[][])"
37     "M:Acme.Widget.M3(System.Int64[],Acme.Widget[0:,0:,0:][])"
38     "M:Acme.Widget.M4(System.Char*,Color*)"
39     "M:Acme.Widget.M5(=VOID*,System.Double*[0:,0:][])"
40     "M:Acme.Widget.M6(System.Int32,System.Object[])"
41
```

42 • Properties and indexers.

```

43     namespace Acme
44     {
45     class widget: Iprocess
46     {
47         public int width {get { /* ... */ } set { /* ... */ }}
48         public int this[int i] {get { /* ... */ } set { /* ... */ }}
49         public int this[string s, int i] {get { /* ... */ } set { /* ... */ }}
50     }
51     }
52     "P:Acme.Widget.Width"
53     "P:Acme.Widget.Item(System.Int32)"
54     "P:Acme.Widget.Item(System.String,System.Int32)"
```

- Events

```

namespace Acme
{
    class widget: Iprocess
    {
        public event Del AnEvent;
    }
}

"E:Acme.Widget.AnEvent"
```

- Unary operators.

```

namespace Acme
{
    class widget: Iprocess
    {
        public static widget operator+(widget x) { /* ... */ }
    }
}

"M:Acme.Widget.op_UnaryPlus(Acme.Widget)"
```

The complete set of unary operator function names used is as follows: `op_UnaryPlus`, `op_UnaryNegation`, `op_Negation`, `op_OnesComplement`, `op_Increment`, `op_Decrement`, `op_True`, and `op_False`.

- Binary operators.

```

namespace Acme
{
    class widget: Iprocess
    {
        public static widget operator+(widget x1, widget x2) { return x1; }
    }
}

"M:Acme.Widget.op_Addition(Acme.Widget,Acme.Widget)"
```

The complete set of binary operator function names used is as follows: `op_Addition`, `op_Subtraction`, `op_Multiply`, `op_Division`, `op_Modulus`, `op_BitwiseAnd`, `op_BitwiseOr`, `op_ExclusiveOr`, `op_LeftShift`, `op_RightShift`, `op_Equality`, `op_Inequality`, `op_LessThan`, `op_LessThanOrEqual`, `op_GreaterThan`, and `op_GreaterThanOrEqual`.

- Conversion operators have a trailing '~' followed by the return type.

```

namespace Acme
{
    class widget: Iprocess
    {
        public static explicit operator int(widget x) { /* ... */ }
        public static implicit operator long(widget x) { /* ... */ }
    }
}

"M:Acme.Widget.op_Explicit(Acme.Widget)~System.Int32"
"M:Acme.Widget.op_Implicit(Acme.Widget)~System.Int64"
```

E.4 An example

E.4.1 C# source code

The following example shows the source code of a Point class:

```

1 namespace Graphics
2 {
3
4     /// <remarks>Class <c>Point</c> models a point in a two-dimensional
5     plane.
6     /// </remarks>
7     public class Point
8     {
9         /// <summary>Instance variable <c>x</c> represents the point's
10        /// x-coordinate.</summary>
11        private int x;
12
13        /// <summary>Instance variable <c>y</c> represents the point's
14        /// y-coordinate.</summary>
15        private int y;
16
17        /// <value>Property <c>X</c> represents the point's x-
18        coordinate.</value>
19        public int X
20        {
21            get { return x; }
22            set { x = value; }
23        }
24
25        /// <value>Property <c>Y</c> represents the point's y-
26        coordinate.</value>
27        public int Y
28        {
29            get { return y; }
30            set { y = value; }
31        }
32
33        /// <summary>This constructor initializes the new Point to
34        /// (0,0).</summary>
35        public Point() : this(0,0) {}
36
37        /// <summary>This constructor initializes the new Point to
38        /// (<paramref name="xor"/>,<paramref name="yor"/>).</summary>
39        /// <param><c>xor</c> is the new Point's x-coordinate.</param>
40        /// <param><c>yor</c> is the new Point's y-coordinate.</param>
41        public Point(int xor, int yor) {
42            X = xor;
43            Y = yor;
44        }
45
46        /// <summary>This method changes the point's location to
47        /// the given coordinates.</summary>
48        /// <param><c>xor</c> is the new x-coordinate.</param>
49        /// <param><c>yor</c> is the new y-coordinate.</param>
50        /// <see cref="Translate"/>
51        public void Move(int xor, int yor) {
52            X = xor;
53            Y = yor;
54        }
55
56        /// <summary>This method changes the point's location by
57        /// the given x- and y-offsets.
58        /// <example>For example:
59        /// <code>
60        ///     Point p = new Point(3,5);
61        ///     p.Translate(-1,3);
62        /// </code>
63        /// results in <c>p</c>'s having the value (2,8).
64        /// </example>
65        /// <summary>
66        /// <param><c>xor</c> is the relative x-offset.</param>
67        /// <param><c>yor</c> is the relative y-offset.</param>
68        /// <see cref="Move"/>
69        public void Translate(int xor, int yor) {
70            X += xor;
71            Y += yor;
72        }
73    }
74 }

```

```

1      /// <summary>This method determines whether two Points have the same
2      /// location.</summary>
3      /// <param><c>o</c> is the object to be compared to the current
4      object.
5      /// </param>
6      /// <returns>True if the Points have the same location and they have
7      /// the exact same type; otherwise, false.</returns>
8      /// <seealso cref="operator==">
9      /// <seealso cref="operator!=">
10     public override bool Equals(object o) {
11         if (o == null) {
12             return false;
13         }
14         if (this == o) {
15             return true;
16         }
17         if (GetType() == o.GetType()) {
18             Point p = (Point)o;
19             return (X == p.X) && (Y == p.Y);
20         }
21         return false;
22     }
23     /// <summary>Report a point's location as a string.</summary>
24     /// <returns>A string representing a point's location, in the form
25     (x,y),
26     /// without any leading, training, or embedded whitespace.</returns>
27     public override string ToString() {
28         return "(" + X + "," + Y + ")";
29     }
30     /// <summary>This operator determines whether two Points have the same
31     /// location.</summary>
32     /// <param><c>p1</c> is the first Point to be compared.</param>
33     /// <param><c>p2</c> is the second Point to be compared.</param>
34     /// <returns>True if the Points have the same location and they have
35     /// the exact same type; otherwise, false.</returns>
36     /// <seealso cref="Equals">
37     /// <seealso cref="operator!=">
38     public static bool operator==(Point p1, Point p2) {
39         if ((object)p1 == null || (object)p2 == null) {
40             return false;
41         }
42         if (p1.GetType() == p2.GetType()) {
43             return (p1.X == p2.X) && (p1.Y == p2.Y);
44         }
45         return false;
46     }
47     /// <summary>This operator determines whether two Points have the same
48     /// location.</summary>
49     /// <param><c>p1</c> is the first Point to be compared.</param>
50     /// <param><c>p2</c> is the second Point to be compared.</param>
51     /// <returns>True if the Points do not have the same location and the
52     /// exact same type; otherwise, false.</returns>
53     /// <seealso cref="Equals">
54     /// <seealso cref="operator==">
55     public static bool operator!=(Point p1, Point p2) {
56         return !(p1 == p2);
57     }
58 }

```



```

1      /// <summary>This is the entry point of the Point class testing
2      /// program.
3      /// <para>This program tests each method and operator, and
4      /// is intended to be run after any non-trivial maintenance has
5      /// been performed on the Point class.</para></summary>
6      public static void Main() {
7          // class test code goes here
8      }
9  }
10 }
```

11 E.4.2 Resulting XML

12 Here is the output produced by one documentation generator when given the source code for class `Point`,
 13 shown above:

```

14 <?xml version="1.0"?>
15 <doc>
16   <assembly>
17     <name>Point</name>
18   </assembly>
19   <members>
20     <member name="T:Graphics.Point">
21       <remarks>Class <c>Point</c> models a point in a two-
22 dimensional
23       plane.
24     </remarks>
25   </member>
26   <member name="F:Graphics.Point.x">
27     <summary>Instance variable <c>x</c> represents the point's
28     x-coordinate.</summary>
29   </member>
30   <member name="F:Graphics.Point.y">
31     <summary>Instance variable <c>y</c> represents the point's
32     y-coordinate.</summary>
33   </member>
34   <member name="M:Graphics.Point.#ctor">
35     <summary>This constructor initializes the new Point to
36     (0,0).</summary>
37   </member>
38   <member name="M:Graphics.Point.#ctor(System.Int32,System.Int32)">
39     <summary>This constructor initializes the new Point to
40     (<paramref name="xor"/>,<paramref name="yor"/>).</summary>
41     <param><c>xor</c> is the new Point's x-coordinate.</param>
42     <param><c>yor</c> is the new Point's y-coordinate.</param>
43   </member>
44   <member name="M:Graphics.Point.Move(System.Int32,System.Int32)">
45     <summary>This method changes the point's location to
46     the given coordinates.</summary>
47     <param><c>xor</c> is the new x-coordinate.</param>
48     <param><c>yor</c> is the new y-coordinate.</param>
49     <see
50 cref="M:Graphics.Point.Translate(System.Int32,System.Int32)"/>
51   </member>
```

```

1      <member
2          name="M:Graphics.Point.Translate(System.Int32,System.Int32)">
3          <summary>This method changes the point's location by
4          the given x- and y-offsets.
5          <example>For example:
6          <code>
7          Point p = new Point(3,5);
8          p.Translate(-1,3);
9          </code>
10         results in <c>p</c>'s having the value (2,8).
11         </example>
12         </summary>
13         <param><c>xor</c> is the relative x-offset.</param>
14         <param><c>yor</c> is the relative y-offset.</param>
15         <see
16 cref="M:Graphics.Point.Move(System.Int32,System.Int32)"/>
17     </member>
18
19     <member name="M:Graphics.Point.Equals(System.Object)">
20         <summary>This method determines whether two Points have the
21         same
22         location.</summary>
23         <param><c>o</c> is the object to be compared to the current
24         object.
25         </param>
26         <returns>True if the Points have the same location and they
27         have
28         the exact same type; otherwise, false.</returns>
29         <seealso
30 cref="M:Graphics.Point.op_Equality(Graphics.Point,Graphics.Point)"/>
31     </member>
32
33     <member name="M:Graphics.Point.op_Inequality(Graphics.Point,Graphics.Point)"/>
34     </member>
35
36     <member name="M:Graphics.Point.ToString">
37         <summary>Report a point's location as a string.</summary>
38         <returns>A string representing a point's location, in the
39         form
40         (x,y),
41         without any leading, training, or embedded
42         whitespace.</returns>
43     </member>
44
45     <member
46         name="M:Graphics.Point.op_Equality(Graphics.Point,Graphics.Point)">
47         <summary>This operator determines whether two Points have the
48         same
49         location.</summary>
50         <param><c>p1</c> is the first Point to be compared.</param>
51         <param><c>p2</c> is the second Point to be compared.</param>
52         <returns>True if the Points have the same location and they
53         have
54         the exact same type; otherwise, false.</returns>
55         <seealso cref="M:Graphics.Point.Equals(System.Object)"/>
56         <seealso
57 cref="M:Graphics.Point.op_Inequality(Graphics.Point,Graphics.Point)"/>
58     </member>

```

```

1      <member
2
3      name="M:Graphics.Point.op_Inequality(Graphics.Point,Graphics.Point)">
4          <summary>This operator determines whether two Points have the
5          same
6          location.</summary>
7          <param><c>p1</c> is the first Point to be compared.</param>
8          <param><c>p2</c> is the second Point to be compared.</param>
9          <returns>True if the Points do not have the same location and
10         the
11         exact same type; otherwise, false.</returns>
12         <seealso cref="M:Graphics.Point.Equals(System.Object)"/>
13         <seealso
14
15     cref="M:Graphics.Point.op_Equality(Graphics.Point,Graphics.Point)"/>
16     </member>
17
18     <member name="M:Graphics.Point.Main">
19         <summary>This is the entry point of the Point class testing
20         program.
21         <para>This program tests each method and operator, and
22         is intended to be run after any non-trivial maintenance has
23         been performed on the Point class.</para></summary>
24     </member>
25
26     <member name="P:Graphics.Point.X">
27         <value>Property <c>X</c> represents the point's
28         x-coordinate.</value>
29     </member>
30
31     <member name="P:Graphics.Point.Y">
32         <value>Property <c>Y</c> represents the point's
33         y-coordinate.</value>
34     </member>
35 </members>
36 </doc>

```